

Sistemas Operativos - Teoría

Tabla de contenido

2 - Introducción a los sistemas operativos.....	7
<i>Sistemas Operativos.....</i>	<i>7</i>
<i>Una computadora.....</i>	<i>7</i>
<i>¿Qué es un Sistema Operativo?.....</i>	<i>8</i>
<i>Los Sistemas Operativos.....</i>	<i>8</i>
<i>La perspectiva del usuario.....</i>	<i>10</i>
¿Qué ve el usuario?.....	10
El sistema de archivos.....	10
El entorno de procesamiento.....	11
Bloques primitivos de construcción.....	11
3- Kernel: userland vs. kernelspace.....	12
<i>El kernel.....</i>	<i>12</i>
¿Por qué es necesario el Kernel?.....	12
Ejecución directa.....	12
Limitar la ejecución directa.....	12
Modo dual de operaciones.....	13
Ejecución directa Vs. Modo dual de operaciones.....	13
Kernel Land y User Land.....	13
Los modos.....	13
IOPL.....	14
Protección del sistema.....	14
Instrucciones privilegiadas.....	14
Instrucciones NO privilegiadas.....	15
Protección de memoria.....	15
Timer interrupts (Interrupciones por temporizador).....	15
Visión General.....	16
Tareas específicas del Kernel.....	16
<i>Modos de transferencia.....</i>	<i>17</i>
De Modo Usuario a Modo Kernel.....	17
Interrupciones.....	17
Excepciones del Procesador.....	17
System Calls.....	18
System Call.....	19
Llamada a una System Call.....	20

De Kernel Mode a User Mode.....	21
<i>El Sistema Operativo Unix y sus Distintos Sabores.....</i>	<i>21</i>
Linux.....	21
MINIX.....	22
BSD – Berkeley Software Distribution.....	23
Tipos de Kernel.....	23
Más sobre los tipos de Kernel.....	23
Más sobre el Kernel.....	23
Una versión moderna del Kernel de Unix.....	24
Alguna funcionalidad que provee el Kernel en el sistema operativo UNIX.....	24
<i>Iniciar al Sistema Operativo y el Kernel.....</i>	<i>25</i>
Booteo o Bootstrap.....	25
Booteo 1.....	25
Booteo 2.....	25
Booteo 3.....	25
Booteo 4.....	25
Fase de inicio del Kernel.....	26
El proceso de inicio.....	26
4 - La abstracción del proceso.....	28
<i>De programa a proceso.....</i>	<i>28</i>
El programa.....	28
El programa: Edición.....	28
El programa: Compilación.....	28
El programa: Formato ejecutable.....	29
Un programa en Linux: ELF.....	30
El comando readelf.....	30
De programa a proceso.....	30
<i>El proceso.....</i>	<i>31</i>
La virtualización.....	31
Definición.....	31
Virtualización de memoria.....	32
El proceso: Espacio de direcciones.....	32
Protección de memoria.....	32
Traducción de direcciones.....	33
Virtualización de procesador.....	33
Proceso: El contexto.....	34
1. El contexto User-Level.....	34
2. El contexto Register.....	35
3. El contexto System-level.....	35
La U area y estructura Proc.....	35
Process Table.....	36

Estados de un proceso.....	37
Estados en Unix System V.....	37
El API resumida.....	37
System Call getpid() y getppid().....	38
Creación de un proceso.....	38
system call _fork_.....	38
¿Qué hace fork?:.....	38
¿Qué hace _fork()_, el algoritmo?.....	38
Aspectos Curiosos de fork().....	39
System Call _exit().....	39
¿Qué hace exit(), el algoritmo?:.....	39
System Call brk().....	40
brk Vs. malloc() y free().....	41
Estados de un proceso en Linux.....	41
Desde el Kernel de Linux.....	42
5 - Signals.....	43
<i>Signals</i>	43
Signals o señales.....	43
Ciclo de vida.....	44
Disposition.....	45
Signal Handler.....	45
Envío de señales.....	45
kill().....	45
raise().....	45
6 - Virtualización de memoria.....	47
<i>La Abstracción del Espacio de Direcciones: Introducción</i>	47
Los primeros sistemas.....	47
Multiprogramación y Time Sharing.....	47
<i>El Espacio de Direcciones o Address Space</i>	49
Virtualización de Memoria.....	50
<i>Address Translation</i>	50
Formalización.....	51
<i>Dynamic Reallocation o Memoria Segmentada [DAH]</i>	51
Base and Bound.....	51
En x86.....	53
En x86 Real Mode.....	53
Address Translation con Tabla de segmentos.....	53
<i>Memoria Paginada</i>	55
Address translation con Page Table.....	55
Memoria paginada x86.....	56
Memoria paginada x86: Virtual Address.....	56

Memoria paginada x86: Page Directory Entry.....	57
Memoria paginada x86: Page Table Entry.....	57
Translation multinivel.....	57
Paged Segmentation (Segmentación paginada).....	58
Address Translation con Tres Niveles de Page Tables.....	59
Address Translation con Tabla de Hash por Software.....	59
Multilevel page segmentation.....	59
<i>Hacia una eficiente Address Translation.....</i>	<i>60</i>
Hacia una eficiente Address Translation: TLB.....	60
Translation-Lookaside buffer.....	60
La TLB.....	61
Todo completo.....	61
Consistencia de la TLB.....	62
La TLB.....	62
Que contiene la TLB.....	63
7 - Sistema de archivos (File system).....	64
<i>¿Qué es un file system?.....</i>	<i>64</i>
<i>El virtual File System.....</i>	<i>64</i>
<i>File System Abstraction Layer.....</i>	<i>65</i>
Linux Kernel Map.....	66
VFS: Sus objetos.....	67
VFS: dentry.....	67
Ejemplo.....	67
VFS: Operaciones.....	67
<i>Archivo.....</i>	<i>67</i>
<i>Directorios o Directories.....</i>	<i>68</i>
<i>El API.....</i>	<i>69</i>
Unix File Systems System Calls.....	69
open().....	69
creat().....	69
close().....	70
read().....	70
write().....	70
lseek().....	70
dup() y dup2().....	70
link().....	71
unlink().....	71
System Calls sobre Directorios.....	71
mkdir().....	71
rmdir().....	71
chdir().....	71

dirent.h.....	71
struct dirent.....	71
opendir().....	72
readdir().....	72
closedir().....	72
<i>System Call sobre los Metadatos.....</i>	<i>72</i>
stat().....	72
access().....	72
chmod().....	72
chown().....	72
umask().....	72
<i>Some File System Unix Commands.....</i>	<i>73</i>
<i>Implementación de File System.....</i>	<i>73</i>
8 - Scheduling.....	74
<i>Time sharing.....</i>	<i>74</i>
Multiprogramación.....	74
Utilización de la CPU.....	74
Ejemplo:.....	75
Multiple Fixed partitions.....	75
Multiple variable Partitions.....	75
Time sharing.....	76
Números y el workload.....	76
Métricas de planificación.....	77
<i>Políticas para sistemas mono-procesador.....</i>	<i>77</i>
First In, First Out (FIFO).....	77
Ejemplo.....	77
Shortest Job First (SJF).....	78
Ejemplo.....	78
Shortest Time-to-Completion (STCF).....	78
Ejemplo.....	78
Una nueva métrica: Tiempo de Respuesta.....	79
Ejemplo.....	79
Round Robin (RR).....	79
Ejemplo.....	79
<i>La planificación en la vida real.....</i>	<i>80</i>
Multi-Level Feedback Queue (MLFQ).....	80
MLQF: Las reglas básicas.....	80
9- Concurrencia.....	82
<i>La concurrencia.....</i>	<i>82</i>
¿A qué apuntamos?.....	82
La abstracción.....	83

Threads vs. procesos.....	83
Threads.....	83
Thread scheduler.....	83
<i>El API de Threads.....</i>	<i>84</i>
Creación de un thread.....	84
Terminación de un thread.....	85
<i>Estructura y ciclo de vida de un thread.....</i>	<i>85</i>
El Estado Per-thread y Threads Control Block (TCB).....	85
Metadata referente al thread que es utilizada para su administración.....	86
Shared State o Estado Compartido.....	86
Estados de un thread.....	86
Tabla de equivalencia entre procesos y threads.....	87

2 - Introducción a los sistemas operativos

Sistemas Operativos

Conceptualmente en el seno de las Ciencias de la Computación lo Sistemas Operativos ocupan un lugar destacado.

Por un lado, porque los **conceptos** que se manejan en el **ámbito de un sistema operativo** están categorizados cómo algunos de los **más complejos** en la informática.

Para tener una idea, un **Sistema Operativo** moderno está compuesto por unas 50 millones de líneas de código fuente.

Alguien en este momento está escribiendo algunas líneas de código para algún sistema operativo que necesariamente no tiene que ser para una PC.

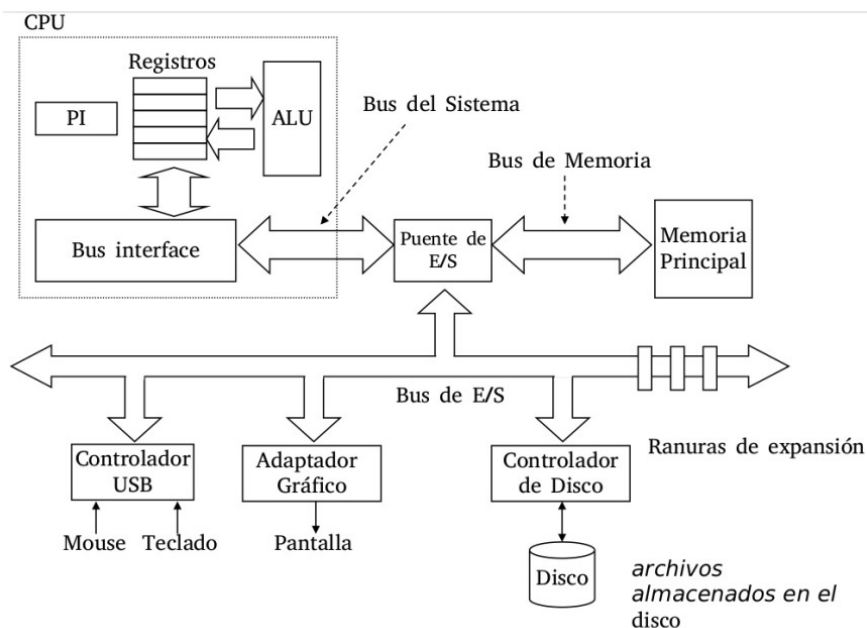
Por otro lado, los conceptos manejados en los sistemas operativos están dentro de la gama de los más utilizados en las Ciencias de la Computación y de hecho, muchos de ellos les van a parecer familiares y de la vida cotidiana:

- Tratar de hacer dos cosas al mismo tiempo.
- Haberse equivocado de cola en el supermercado.
- Tratar de que tu hermano, con el cual compartís la habitación, no se meta con tus cosas.

...entre otras.

Una computadora

Puede parecer casi trivial pero para poder dominar los conceptos que se manejan en sistemas operativos se debe tener bien en claro: ¿Qué es una Computadora? y además, ¿cuáles son sus partes?:



Las partes esenciales como todos sabemos... son:

- ❖ CPU: Central Process Unit.

- ❖ Main Memory: Memoria Principal.
- ❖ Devices: Dispositivos de Entrada o Salida.
- ❖ Buses: conexiones.

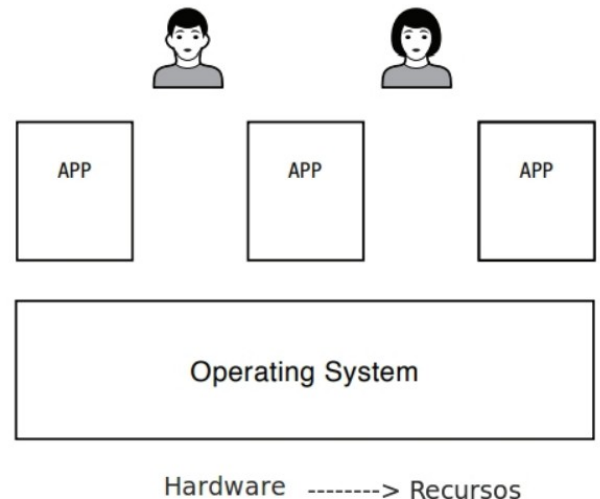
¿Qué es un Sistema Operativo?

Un **Sistema Operativo (OS)** es la **capa de software** que **maneja** los **recursos de una computadora** para sus **usuarios** y sus **aplicaciones**. [DAH]

Según Commers [COM]: Un **sistema operativo** está diseñado para **ocultar** detalles de **hardware de bajo nivel** y para crear una **máquina abstracta** que le proporcione a las **aplicaciones** servicios de **alto nivel**.

Según Maurice Bach [BCH]: El sistema operativo **interactúa directamente con el hardware**, proporcionando **servicios comunes** a los programas y **aislándolos** de idiosincrasia de hardware.

Otra definición dada por [STV]: Un Sistema Operativo puede definirse como el **software** que **controla** los **recursos de hardware** de una **computadora** y **provee** un **ambiente** bajo el cual los **programas pueden ejecutarse**.



Normalmente los sistemas operativos son **invisibles a los ojos de sus usuarios**. Por ejemplo alguien le prestó atención al sistema operativo de:

- ✚ Una consola de vídeo juegos
- ✚ Una Heladera
- ✚ Un teléfono
- ✚ Un karaoke
- ✚ Un coche
- ✚ Una cámara de fotos
- ✚ Un avión

De todas formas en este curso se estudiará a fondo a los sistemas operativos de propósito general, muchos de ustedes ya han interactuado con alguno como por ejemplo Linux, Mac OS o Windows.

Hoy en día cualquier persona que utilice una computadora debe lidiar, aun inconscientemente, con un sistema operativo.

En un sistema operativo de propósito general, los **usuarios** interactúan con **aplicaciones**, estas aplicaciones se **ejecutan** en un **ambiente** que es proporcionado por el **sistema operativo**. A su vez el sistema operativo **hace de mediador** para tener acceso al **hardware** del equipo.

Los Sistemas Operativos

Un sistema operativo es el software encargado de hacer que la ejecución de los programas parezca algo fácil. La forma principal para llegar a lograr esto es mediante el concepto de **virtualización**. Esto es, el sistema operativo toma un **recurso físico** (la memoria, el procesador, un disco) y lo transforma en algo virtual más general, poderoso, fácil de usar.

Un sistema operativo deberá virtualizar varios recursos físicos:

- ❖ El **procesador**
- ❖ La **memoria**
- ❖ La **persistencia** (dispositivos de almacenamiento).

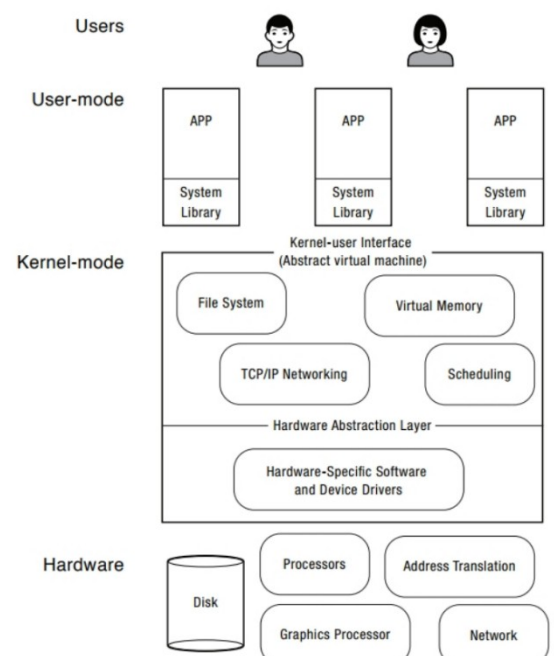
Por lo pronto se puede decir que un sistema operativo juega simultáneamente tres roles según varios autores DAH, ARP:

- ▣ **Referee:** Un OS gestiona recursos compartidos entre diferentes aplicaciones, que se encuentran ejecutándose en la misma máquina física. Por ejemplo:
 - o un OS puede frenar la ejecución de una aplicación e iniciar la ejecución de otra.
 - o los OS aíslan a cada aplicación de las demás que se encuentran corriendo en la misma computadora.
 - o Por ende un OS tiene que protegerse a sí mismo y a las demás aplicaciones que se están ejecutando en la misma computadora.
 - o Y dado que todas estas aplicaciones comparten los mismos recursos físicos el OS decide que aplicación usa un determinado recurso y cuando.
- ▣ **Ilusionista:** Un OS debe proveer una abstracción del hardware para simplificar el diseño de aplicaciones. Imaginen lo complejo que sería escribir el clásico hola mundo no se tiene que pensar en qué lugar de la memoria física este se encuentra almacenado, ni como esta memoria se comparte con otros datos y aplicaciones. El sistema operativo provee la **ilusión** de que se dispone de toda la memoria para almacenar al programa, cuando realmente se sabe que la memoria principal de la computadora es finita.
- ▣ **Pegamento:** Un OS debe proveer una serie de **servicios comunes** que faciliten un mecanismo que permita compartir, por ejemplo, información entre las aplicaciones.... “Cut & Paste” por ejemplo ... este mecanismo es **uniforme** en todo el sistema. Otro ejemplo es el “look and feel” de la interfaz de usuario. Tal vez uno de los más importantes sea el mecanismo de acceso a los dispositivos de entrada y salida del sistema, de forma tal que las aplicaciones puedan usarlos independientemente de la marca y modelo de los mismos.

En el **más bajo nivel** se encuentra el hardware o recursos de la máquina:

- CPU,
- memoria principal,
- dispositivos de red,
- dispositivos gráficos,
- dispositivos de almacenamiento, etc.

El hardware provee primitivas que el sistema operativo puede utilizar para aislar fallas y para sincronización.



- El Sistema Operativo se ejecuta como **la capa de software de más bajo nivel** en la computadora. Este contiene:
 - o por un lado una capa para la gestión de dispositivos específico
 - o y por otro una serie de servicios para la gestión de dispositivos agnósticos del hardware que son utilizados por las aplicaciones. Estas dos capas suelen ser conocidas como el **kernel** del sistema operativo. Cuando código fuente de esta capa es ejecutado la computadora pasa a un estado llamado **Modo Supervisor**.
- Las aplicaciones se ejecutan en un **contexto aislado, protegido y restringido** y mediante el uso de funciones que se encuentran bibliotecas pueden utilizar los servicios de acceso al hardware o recursos que el kernel proporciona. El contexto de ejecución de las aplicaciones se denomina **User Mode** o modo usuario, más restrictivo, aislado y controlado.

Por lo dicho anteriormente, existen **dos modos de ejecución**.

La perspectiva del usuario

¿Qué ve el usuario?

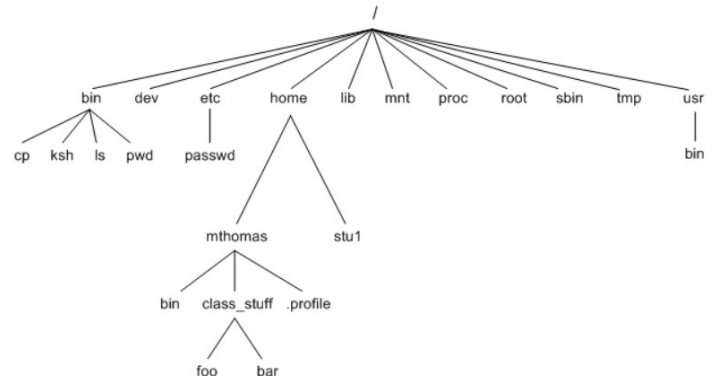
Desde el punto de vista del usuario lo que “se ve” es:

1. El **Sistema de Archivos** o File System.
2. El **Entorno de Procesamiento**.
3. Los **Bloques Primitivos de Construcción**.

El sistema de archivos

El sistema de archivos de Unix se caracteriza por:

- 1- Su **estructura jerárquica**
- 2- Tratamiento consistente archivos de datos
- 3- La habilidad de **crear y borrar archivos**
- 4- El **crecimiento dinámico** de los archivos
- 5- La **protección de los archivos** de datos
- 6- El **tratamiento de los dispositivos periféricos como si fueran archivos**



En Linux todo es un archivo

El Sistema de archivos de unix está organizado como un árbol con una única raíz (root) (que se escribe como “/”); cada nodo no hoja del sistema de archivo se denomina directorio de archivos, y las hojas del árbol pueden ser a su vez pueden ser archivos, directorios de archivos, archivos especiales, etc.

El nombre de un archivo está dado por el path (camino) para localizarlo dentro de la estructura del sistema de archivos. “/etc/passwd”

El sistema operativo no tiene noción de cómo es la estructura interna en el que se guardan.

El sistema operativo Unix provee una lista de programas que vienen junto al sistema operativo, estos programas normalmente se denominan comandos, para realizar operaciones sobre los componentes del sistema de archivos:

- **touch:** este comando permite crear un archivo o marcarlo como modificado.
- **rm:** este comando permite borrar un archivo.
- **mkdir:** este comando permite crear un directorio.
- **rmdir:** este comando permite eliminar un directorio.
- **ls:** este programa permite listar el contenido de un directorio.
- **pwd:** este comando permite saber en que parte del sistema de archivos se está.
- **cat:** este comando permite ver el contenido de un archivo.
- **cp:** copia un archivo.
- **mv:** cambia la locación de un archivo.

Existe una lista bastante extensa de comandos unix, aquí se deja una lista de los comandos que tienen que estar en una distribución de UNIX:

https://en.wikipedia.org/wiki/List_of_Unix_commands

El entorno de procesamiento

Un programa es un archivo ejecutable un proceso es **una instancia** de un programa en ejecución.

Muchos procesos pueden estar ejecutándose a la vez, no existe un límite lógico para la cantidad de estos.

Existen varios comandos relacionados con los procesos:

- **ps:** muestra los procesos que ha lanzado un determinado usuario.
- **top:** muestra la lista de comandos que utilizan mas recursos.
- **who:** muestra los usuarios conectados.
- **kill:** mata a un proceso.
- **free:** muestra la memoria libre del sistema.

El *intérprete de comandos* o *shell* es el primer programa que se ejecuta (en un unix) en modo usuario, este a su vez ejecuta el comando de login. El intérprete de comandos captura las teclas que son presionadas en el teclado y las traduce a nombres de comandos.

El shell puede interpretar tres tipos de comandos:

- 1) **Comandos Ejecutables** simples programas, por ejemplo el **ls**
- 2) **Shell scripts** que es un archivo ejecutable, consistente en líneas de comandos ejecutables
- 3) **Estructuras de control del Shell:** *if-then-else-f*

El shell ejecuta los comandos buscando en ciertos directorios en una determinada secuencia.

Dado que el shell es un programa no forma parte del **Kernel** del sistema operativo.

El sistema operativo provee también unas System Calls para la creación y manipulación de procesos:

- **fork():** crea un proceso hijo a partir de un proceso padre.
- **execl(), execve():** creación de procesos.

- **wait()**: sincronización

Bloques primitivos de construcción

La idea de unix es la de proveer ciertos mecanismos para que los programadores construyan a partir de pequeños programas otros más complejos.

Esta idea está basada en la ya vieja y conocida estrategia de **Divide y Conquista**.

Estos mecanismos son usables desde el shell:

- El redireccionamiento de entrada y salida de datos. Los procesos acceden normalmente a tres archivos estándares: `_stdin_`, `_stdout_` y `_stderr_`
- El operador `>` redirecciona el standar output a un archivo. `ls >output`
- El operador `<` redirecciona el standar input tomando los datos de un archivo. `iconv -f ISO-8859-15 -t UTF-8 < Introduccion_a_la_programacion.tex > intro.tex`
- Las tuberías o **los pipes**. Permites que un stream de datos sean pasados entre dos procesos uno escritor y el otro lector. Del extremo izquierdo del caño se redirecciona el standar output de ese proceso al extremo derecho del caño el cual redirecciona el standar input del otro proceso.

3- Kernel: userland vs. kernelspace

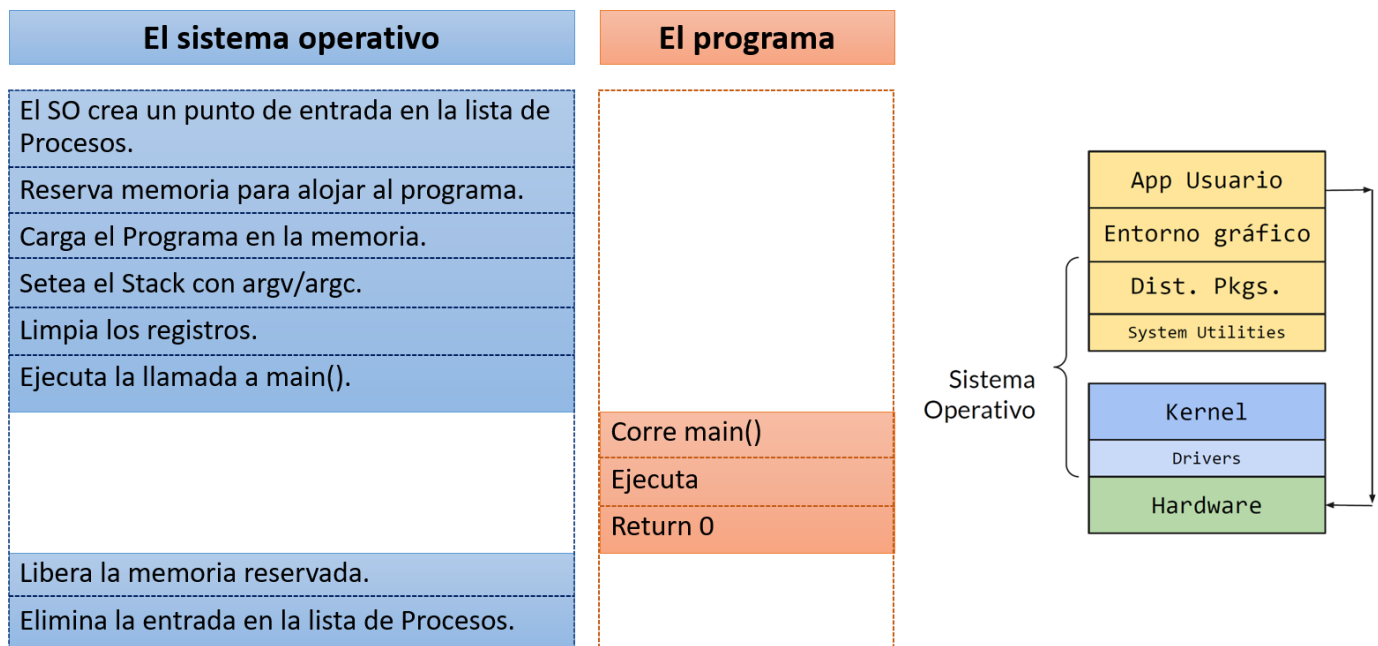
El kernel

¿Por qué es necesario el Kernel?

El rol central de un sistema operativo es la protección

Ejecución directa

La ejecución directa de un programa es un concepto simple, significa, correr el programa directamente en la CPU:



¿Tiene alguna ventaja la ejecución directa? Si, la rapidez.

Pero.... ¿cuáles son los problemas de este mecanismo tan simple?:

1. Cuando se corre un programa, ¿cómo se asegura el Sistema Operativo, que el programa no va a hacer nada que el usuario no quiere que sea hecho?
2. ¿Cómo hace el sistema Operativo para pausar la ejecución de ese programa y hacer que otro sea ejecutado?

Justamente se necesita Limitar la Ejecución Directa

Limitar la ejecución directa

La ejecución directa es PELIGROSA

Para poder limitar la ejecución directa se necesitan ciertos mecanismos de hardware:

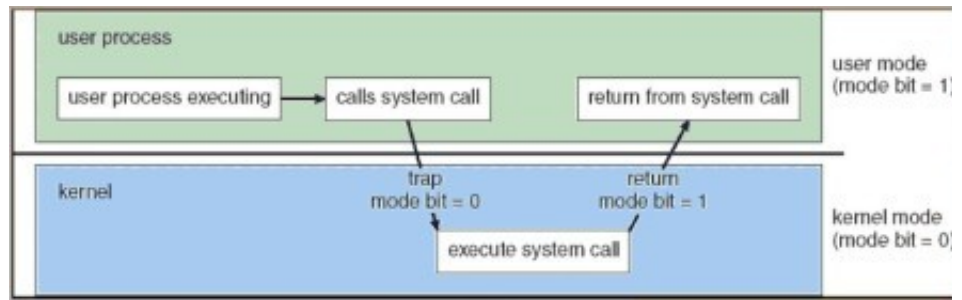
1. Dual Mode Operation - Modo de operación dual
2. Privileged Instructions - Instrucciones Privilegiadas

3. Memory Protection - Protección de Memoria

4. Timer Interrupts - Interrupciones por temporizador

Modo dual de operaciones

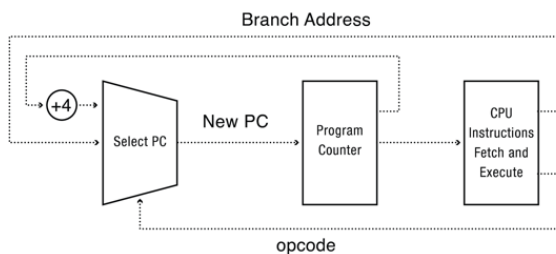
- 1) User-Mode
- 2) Kernel-Mode



Ejecución directa Vs. Modo dual de operaciones

Ejecución directa

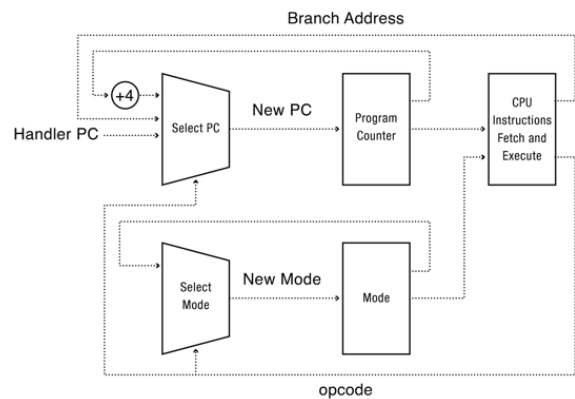
El modo de operación dual, es un mecanismo que proveen todas los procesadores y algunos microprocesadores modernos. En un mundo Ideal... el esquema básico de un microprocesador podría ser el siguiente:



Intel 8088 no tenía bit de modo - no tenía dual mode - (esa es la historia detrás de MSDOS).

Modo dual de operaciones

La realidad:

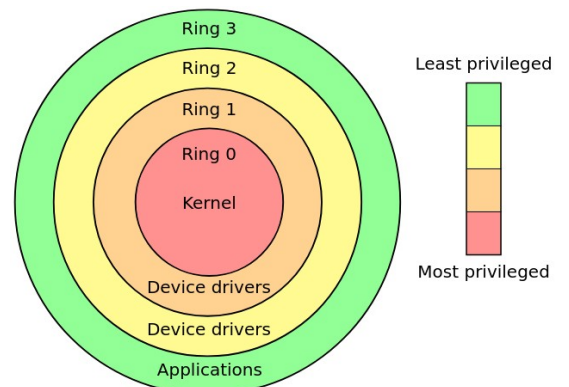


Existen ciertos estados en el que el procesador puede encontrarse.

Kernel Land y User Land

Los modos

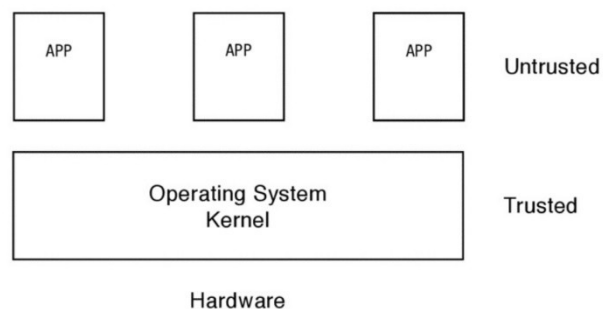
En el caso de la arquitectura x86 la misma provee 4 modos de operaciones vía el hardware. Estos modos están numerados entre 0 y 3, y se denominan **rings**.



Los modos más utilizados por los sistemas operativos son el **modo 0** (modo supervisor o modo Kernel) y el **modo 3**, llamado modo "Usuario" o "User Mode" dado que este mecanismo es proveído por el hardware cada instrucción a ser ejecutada el mismo chequea en qué modo de operación se encuentra.

Existen dos modos operacionales utilizados de la CPU:

- **Modo Usuario o User Mode:** que ejecuta instrucciones en nombre del usuario.
- **Modo Supervisor o Kernel o Monitor:** ejecuta instrucciones en nombre del Kernel del S.O. y estas son instrucciones privilegiadas.



Esta diferencia entre los modos operacionales de un procesador equivale a un bit en el registro de control del procesador. Este bit permite diferenciar entre tipos de instrucciones: instrucciones privilegiadas o instrucciones normales.

A cada uno de estos niveles de privilegio también se lo denominan **Ring** o Anillo.

Se protege:

1. La memoria
2. Los Puertos de I/O
3. La Posibilidad de ejecutar ciertas instrucciones

IOPL

El indicador IOPL (nivel de privilegio de E / S) es un indicador que se encuentra en todas las CPU x86 compatibles con IA-32. Ocupa los bits 12 y 13 en el registro FLAGS. En modo protegido y modo largo, muestra el nivel de privilegio de E/S del programa o tarea actual. El nivel de privilegio actual (CPL) (CPL0, CPL1, CPL2, CPL3) de la tarea o programa debe ser menor o igual que el IOPL para que la tarea o el programa accedan a los puertos de E / S.

El IOPL se puede cambiar usando POPF (D) e IRET (D) solo cuando el nivel de privilegio actual es Ring 0.

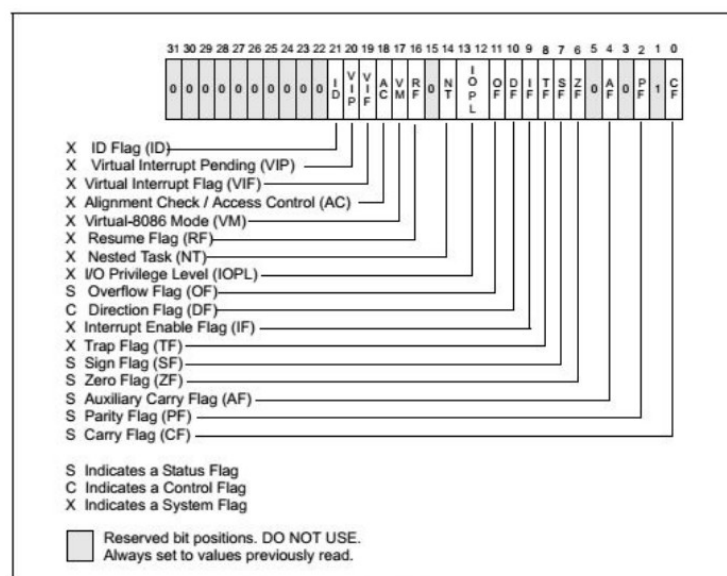


Figure 3-8. EFLAGS Register

Además de IOPL, los permisos de puerto de E/S en el TSS (Task State Segment) también participan en la determinación de la capacidad de una tarea para acceder a un puerto de E / S.

Protección del sistema

¿Cuál es el hardware necesario para que el kernel del sistema operativo pueda proteger a **Usuarios y aplicaciones de otros usuarios?**

Instrucciones privilegiadas

La existencia del mecanismo llamado modo dual permite que los distintos modos posean cada uno su propio set de instrucciones. Con lo cual el bit de modo de operación indica al procesador si la instrucción ejecutada puede ser o no ejecutada, según el modo en que se encuentre el mismo.

Este conjunto de instrucciones sólo puede ser ejecutado en **modo kernel**.

Instrucción	Descripción
LGDT	Loads an address of a GDT into GDTR
LLDT	Loads an address of a LDT into LDTR
LTR	Loads a Task Register into TR
MOVCR	Copy data and store in Control Registers
LMSW	Load a new Machine Status WORD
CLTS	Clear Task Switch Flag in Control Register CRO
MOV	Debug Register Copy data and store in debug registers
INVD	Invalidate Cache without writeback
INVLPG	Invalidate TLB Entry
WBINVD	Invalidate Cache with writeback
HLT	Halt Processor
RDMSR	Read Model Specific Registers (MSR)
WRMSR	Write Model Specific Registers (MSR)
RDPMSR	Read Performance Monitoring Counter
RDTSC	Read time Stamp Counter

Estas instrucciones no pueden ser accedidas en *user mode*.

Instrucciones NO privilegiadas

- Reading the status of Processor
- Reading the System Time
- Generate any Trap Instruction
- Sending the final printout of Printer

Protección de memoria

El sistema operativo y los programas que están siendo ejecutados por el mismo deben residir **ambos en memoria al mismo tiempo**.

El sistema operativo tiene que estar ahí para cargar el programa y hacer que comience a ejecutarse.

El programa tiene que residir en memoria para poder ejecutarse, es más, *todos* los programas que se están ejecutando deben estar cargado en la memoria de la máquina.

Debido a esto el y para que la memoria sea compartida de forma segura, el sistema operativo debe poder configurar al hardware de forma tal que cada proceso pueda leer y escribir su propia porción de memoria.

Existen mecanismos de hardware que permiten hacer este control.

PID: 12391				Heap
grep				Stack
				Data
				Code
File Descriptors				
0	1	2	3	

stdin
stdout
stderr
file.txt

Timer interrupts (Interrupciones por temporizador)

Para que el kernel pueda tomar el control de la computadora debe haber algún mecanismo que periódicamente le permita al kernel **desalojar** al proceso de usuario en ejecución y volver a tomar el control del procesador, y así de toda la máquina.

Cuando un programa se está ejecutando este cree que tiene todo el tiempo del mundo para hacer lo que quiera... pero eso es sólo una ilusión.

En la actualidad todos los procesadores poseen un mecanismo de hardware llamado **hardware counter**, el cual puede ser seteado para que luego del transcurso de un determinado tiempo el procesador sea interrumpido. Cada CPU posee su propio timer.

Cuando una interrupción por tiempo ocurre, el proceso en modo usuario que se esté ejecutando le transfiere el control al kernel ejecutándose en modo kernel. De esta forma el kernel tiene asegurado el uso del procesador.

Todos los procesadores poseen un dispositivo de hardware llamado **Interrupciones de Tiempo**. Una Interrupción de Tiempo permite que el procesador sea interrumpido después de un determinado lapso de tiempo. Cada procesador tiene su propio contador de tiempo. Si el procesador es multicore cada core tiene sus propio contador de tiempo individual.

El reseteo del timer es una instrucción privilegiada que puede ser utilizada en modo kernel.

```
$ sudo cat /proc/timer_list
```

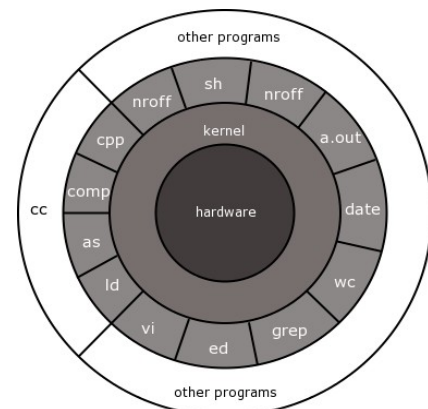
Visión General

Si se define al sistema operativo como el *software que maneja y dispone de los recursos de una computadora*, entonces el término **kernel** puede ser equivalente al de **sistema operativo**.

Viendo al sistema como un conjunto de capas, el **sistema operativo** se denomina comúnmente **kernel del sistema**, o simplemente **kernel**, lo que enfatiza su aislamiento de los programas de los usuarios (sh, vi, wc, ls, etc.).

Gracias a la existencia del **kernel** los programas son **independientes del hardware subyacente**, es fácil moverlos entre sistemas UNIX que se ejecutan en hardware diferente dado que los programas no hacen suposiciones sobre el hardware subyacente, sino que se comunican con el kernel y no con el hardware directamente.

El **kernel** es entonces **la capa de software de más bajo nivel** en la computadora.



Este contiene por un lado una capa para la gestión de dispositivos específico y por otro una serie de servicios para la gestión de dispositivos agnósticos del hardware que son utilizados por las aplicaciones.

Cuando código fuente de esta capa es ejecutado la computadora pasa a un estado llamado **Modo Supervisor**.

Un Sistema Operativo puede definirse como el software que controla los recursos de hardware de una computadora y provee un ambiente bajo el cual los programas pueden ejecutarse. Habitualmente a este software se lo llama el Kernel.

Una de las cosas a tener en cuenta es que el **kernel es un programa**.

Tareas específicas del Kernel

Este programa *hace cosas*, entre ellas:

- ♦ **Planificar la ejecución de las aplicaciones**
- ♦ **Gestionar la Memoria**
- ♦ **Proveer un sistema de archivos**
- ♦ **Creación y finalización de procesos**
- ♦ **Acceder a los dispositivos**
- ♦ **Comunicaciones**
- ♦ **Proveer un API**

Modos de transferencia

Una vez que el hardware posee los mecanismos necesarios para que pueda ejecutarse un kernel tiene que haber una o varias formas de alternar entre **modo usuario** y **modo kernel**.

Este tipo de transición no son eventos raros y por ende deben tener un mecanismo que sea seguro y rápido y que además no de lugar para programas maliciosos o con errores que pueden intencionalmente ser insertados y corromper el Kernel.

De Modo Usuario a Modo Kernel

Existen tres formas por las cuales se debería pasar de pasar de modo usuario a modo kernel:

1. Interrupciones: Un evento externo
2. Excepciones del procesador: Un evento inesperado
3. Mediante la ejecución de system calls (llamadas al sistema): Un evento intencional generado por una system call (int 0x80).

Interrupciones

Una **interrupción** es una **señal asincrónica** enviada hacia el procesador de que **algún evento externo ha sucedido** y pueda **requerir de la atención** del mismo.

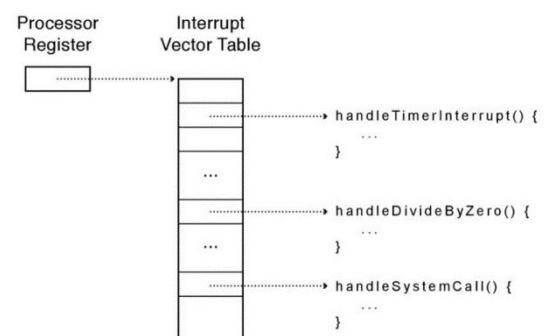
El procesador está continuamente chequeando por si una interrupción se dispara. Si así es, este **completa o detiene** cualquier instrucción que se esté ejecutando y en vez de ejecutar la siguiente instrucción, el procesador **guarda** todo el contexto en el que se estaba ejecutando la instrucción y comienza a ejecutar el manejador de esa interrupción en el kernel.

IRQ	Usage
0	system timer (cannot be changed)
1	keyboard controller (cannot be changed)
2	cascaded signals from IRQs 8–15
3	second RS-232 serial port (COM2: in Windows)
4	first RS-232 serial port (COM1: in Windows)
5	parallel port 2 and 3 or sound card
6	floppy disk controller
7	first parallel port
8	real-time clock
9	open interrupt
10	open interrupt
11	open interrupt
12	PS/2 mouse
13	math coprocessor
14	primary ATA channel
15	secondary ATA channel

El orden de prioridad de las interrupciones según [BCH], ordenados por prioridad decreciente:

1. Errores de la Máquina
2. Timers
3. Discos Network device
4. Terminales
5. Interrupciones de Software

```
$ sudo cat /proc/interrupts
```



Excepciones del Procesador

La otra forma por la cual se necesitaría pasar de modo usuario a modo kernel es por un **evento de hardware causado por un programa de usuario**. El funcionamiento es igual de la una interrupción.

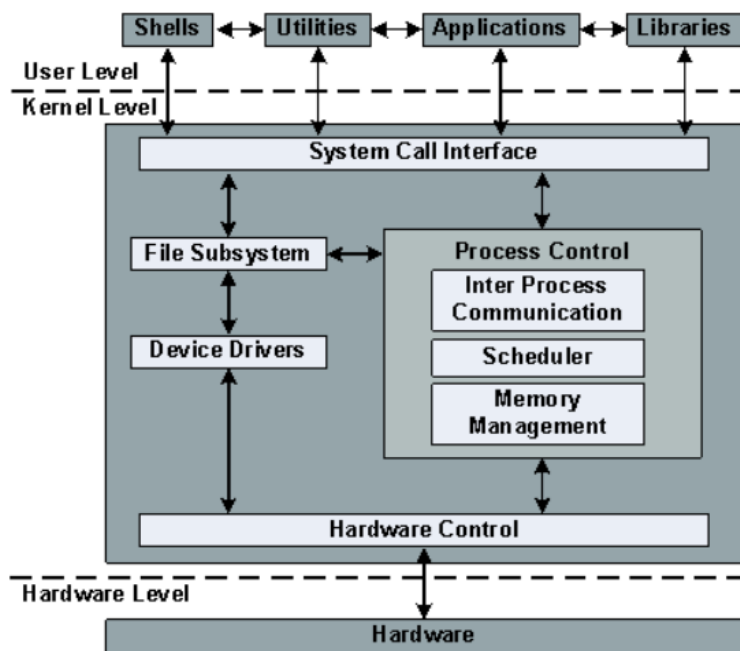
Una excepción podría ser:

- ! Acceder fuera de la memoria del proceso
- ! Intentar ejecutar una instrucción privilegiada en modo usuario
- ! Intentar escribir en memoria de solo lectura
- ! Dividir por cero

```
set_trap_gate(0,&divide_error);
set_intr_gate(1,&debug);
set_intr_gate(2,&nmi);
set_system_intr_gate(3, &int3); /* int3/4 can be called from all */
set_system_gate(4,&overflow);
set_trap_gate(5,&bounds);
set_trap_gate(6,&invalid_op);
set_trap_gate(7,&device_not_available);
set_task_gate(8,GDT_ENTRY_DOUBLEFAULT_TSS);
set_trap_gate(9,&coprocessor_segment_overrun);
set_trap_gate(10,&invalid_TSS);
set_trap_gate(11,&segment_not_present);
set_trap_gate(12,&stack_segment);
set_trap_gate(13,&general_protection);
set_intr_gate(14,&page_fault);
set_trap_gate(15,&spurious_interrupt_bug);
set_trap_gate(16,&coprocessor_error);
set_trap_gate(17,&alignment_check);
```

System Calls

Las System Calls son funciones que permiten a los procesos de usuario pedirle al kernel que **realice operaciones** en su nombre. Una System Call es cualquier función que el kernel **expone** que puede ser utilizada por un proceso a nivel usuario.



Las System Calls conforman un API:

GENERAL CLASS	SPECIFIC CLASS	SYSTEM CALL
File Structure Related Calls	• Creating a Channel • Input/Output • Random Access • Channel Duplication • Aliasing and Removing • Files • File Status • Access Control • Device Control	• creat() • open() • close() • read() • write() • lseek() • dup() • link() • unlink() • stat() • fstat() • access() • chmod() • chown() • umask() • ioctl()
Calls	• Process Creation and Termination • Process Owner and Group • Process Identity • Process Control • Change Working Directory	• exec() • fork() • wait() • exit() • getuid() • geteuid() • getgid() • getegid() • getpid() • getppid() • signal() • kill() • alarm() • chdir()
Interprocess Communication	• Pipelines • Messages • Semaphores • Shared Memory	• pipe() • msgget() • msgsnd() • msgrcv() • msgctl() • semget() • semop() • shmget() • shmat() • shmdt()

System Call

Una **system call** (llamada al sistema) es un punto de entrada controlado al kernel, permitiendo a un proceso solicitar que el **kernel** realice alguna operación en su nombre [KER] (cap. 3). El kernel expone una gran cantidad de servicios accesibles por un programa vía el **application programming interface (API) de system calls**.

Algunas características generales de las system calls son:

- Una system call cambia el modo del procesador de *user mode* a *kernel mode*, por ende la CPU podrá acceder al área protegida del kernel.
- El conjunto de system calls es fijo. Cada system call esta identificada por un único número, que por supuesto no es visible al programa, este sólo conoce su nombre.
- Cada system call debe tener un conjunto de parámetros que especifican información que debe ser transferida desde el *user space* al *kernel space*.

Las system calls de unix v6 syscalls Las system calls de linux para arquitectura x86 syscalls x86

Llamada a una System Call

Desde el punto de vista de un programa llamar a una system call es más o menos como invocar a una función de C. Por supuesto, detrás de bambalinas muchas cosas suceden:

1. El programa realiza un llamado a una system call mediante la invocación de una **función wrapper** (envoltorio) en la biblioteca de C.
2. Dicha función **wrapper** tiene que proporcionar todos los argumentos al **system call trap_handling**. Estos argumentos son pasados al wrapper por el stack, pero el kernel los espera en determinados registros. La función wrapper copia estos valores a los registros.
3. Dado que todas las system calls son accedidas de la misma forma, el kernel tiene que saber identificarlas de alguna forma. Para poder hacer esto, la función *wrapper* copia el número de la system call a un determinado registro de la CPU (%eax).
4. La función *wrapper* ejecuta una instrucción de código maquina llamada **trap machine instruction** (int 0x80), esta causa que el procesador pase de *user mode* a *kernel mode* y ejecute el código apuntado por la dirección 0x80 (128) del vector de traps del sistema.

		Registers								Definition	
#	Name	eax	ebx	ecx	edx	esi	edi				
0	sys_restart_syscall	0x00	-	-	-	-	-	-	-	kernel/signal.c:2475	
1	sys_exit	0x01	int error_code	-	-	-	-	-	-	kernel/exit.c:935	
2	sys_fork	0x02	-	-	-	-	-	-	-	kernel/fork.c:2116	
3	sys_read	0x03	unsigned int fd	char __user *buf	size_t count	-	-	-	-	fs/read_write.c:566	
4	sys_write	0x04	unsigned int fd	const char *buf	size_t count	-	-	-	-	fs/read_write.c:581	
5	sys_open	0x05	const char __user *filename	int flags	umode_t mode	-	-	-	-	fs/handle.c:257	
6	sys_close	0x06	unsigned int fd	-	-	-	-	-	-	fs/open.c:1153	
7	sys_waitpid	0x07	pid_t pid	int __user *stat_addr	int options	-	-	-	-	kernel/exit.c:1692	
8	sys_creat	0x08	const char __user *pathname	umode_t mode	-	-	-	-	-	fs/open.c:1115	
9	sys_link	0x09	const char __user *oldname	const char __user *newname	-	-	-	-	-	fs/namei.c:4313	
10	sys_unlink	0x0a	const char __user *pathname	-	-	-	-	-	-	fs/namei.c:4097	
11	sys_execve	0x0b	const char __user *filename	const char __user *argv	const char __user *envp	-	-	-	-	fs/exec.c:1919	

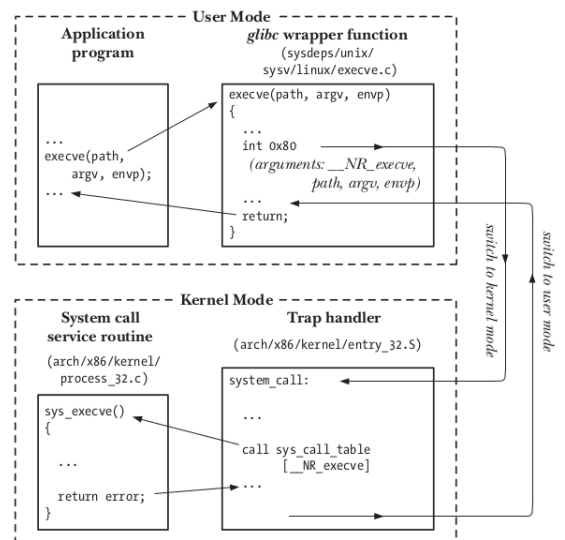
5. En respuesta al trap de la posición 128, el kernel invoca su propia función llamada `system_call()` (arch/i386/entry.s) para manejar esa trap. Este manejador:
 - a. graba el valor de los registros en el stack del kernel.
 - b. verifica la validez del número de system call.
 - c. invoca el servicio correspondiente a la system call llamada a través del vector de system calls, el servicio realiza su tarea y finalmente le devuelve un resultado de estado a la rutina `system_call()`.
 - d. se restauran los registros almacenado en el stack del kernel y se agrega el valor de retorno en el stack.
 - e. se devuelve el control al wrapper y simultáneamente se pasa a user mode.
6. Si el valor de retorno de la rutina de servicio de la system call da error, la función wrapper setea el valor en *errno*.

Strace

Ltrace

```
strace ls testdir/
```

```
strace -o trace.log ls testdir/
```



De Kernel Mode a User Mode

Así como hay varias formas de pasar de modo Usuario a modo Kernel también hay varias formas de pasar de **modo kernel** a **modo usuario**:

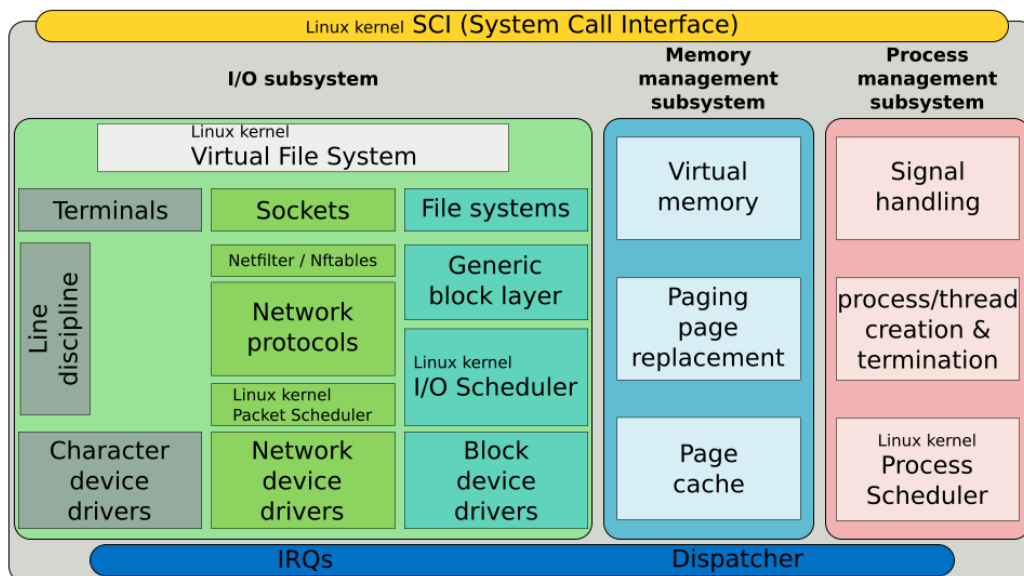
- ★ Un **nuevo proceso**. Cuando se inicia un nuevo proceso el Kernel copia el programa en la memoria, setea el contador de programa apuntando a la primera instrucción del proceso setea el stack pointer a la base del stack de usuario y switchea a modo usuario.
- ★ **Continuar después de una interrupción una excepción del procesador o de un system call**. Una vez que el Kernel termino de manejar el pedido, este continua con la ejecución de proceso interrumpido mediante la restauración de todos los registros y cambiando el modo a nivel usuario.
- ★ **Cambio entre diferentes procesos**. En algunos casos puede pasar que el Kernel decida ejecutar otro proceso que no sea el que se estaba ejecutando, en este caso el Kernel carga el estado del proceso nuevo a través de la PCB y cambia a modo usuario.

El Sistema Operativo Unix y sus Distintos Sabores

Linux

En el caso de linux y contrariamente a lo que la mayoría de los usuarios creen, linux no es un sistema operativo sino que Linus desarrolló solo un kernel. El resto de los componentes de linux son desarrollos de terceros como por ejemplo GNU.

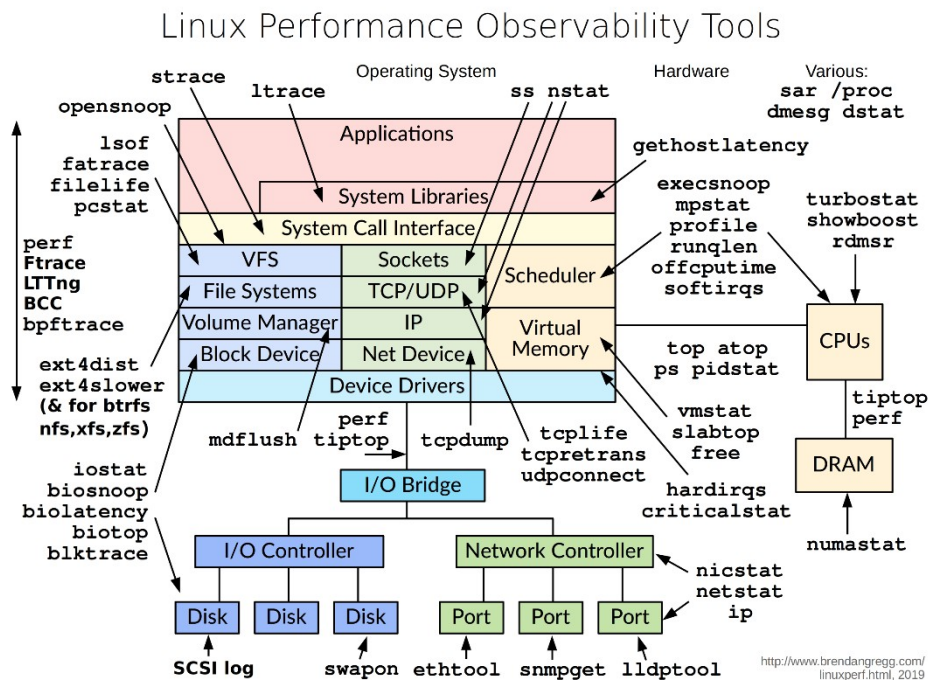
by Shmuel Csaba Otto Traian Xerxes; GFDL 1.3 & CC-BY-SA 4.0; 2016-02-20



El kernel de linux posee ciertas características a la hora de ser comparado con otros programas:

- o El kernel no tiene acceso a la biblioteca C ni a los encabezados C estándar.
- o El kernel está codificado en GNU C.
- o El kernel carece de la protección de memoria internamente, no se protege de sí mismo [^4].
- o El kernel no puede ejecutar fácilmente operaciones de punto flotante.
- o El kernel tiene una pequeña pila de tamaño fijo y no es dinámica [^5].

- o Debido a que el kernel tiene interrupciones sincrónicas, es preemptive y admite SMP[^6], la sincronización y la concurrencia son preocupaciones importantes dentro del kernel.
- o La portabilidad es importante.



MINIX

MINIX es un sistema operativo de la familia de UNIX, creado por Andrew S. Tanenbaum con fines didácticos para su curso de sistemas Operativos de la Universidad Libre de Amsterdam. Este venía distribuido en diskettes junto a su libro de texto Operating Systems: Design and Implementation (1987). Versiones de MINIX:

- MINIX 1.0 (1987)
- MINIX 2.0 (1997)
- MINIX 3.0 (2005)

El Kernel de MINIX utiliza una filosofía distinta que el kernel de Linux o de un UNIX tradicional. El kernel se basa en el concepto de microkernel pequeño que maneja interrupciones, gestión de procesos de bajo nivel e IPC, y no mucho más (unas 4000 líneas de código fuente). La mayor parte del sistema operativo se ejecuta como una colección de procesos en modo de usuario.

El concepto clave aquí es el “sistema operativo multiservidor”: el diseño del sistema como un colección de controladores y servidores en modo de usuario con modos de falla independientes.

Mientras el término “microkernel” llama mucho la atención, éstos son ampliamente utilizados en teléfonos celulares, aviónica, automotriz y otros sistemas integrados donde la fiabilidad es crucial.

El microkernel se ejecuta en modo kernel, pero casi todos los demás componentes del sistema operativo se ejecutan en modo de usuario. La capa más baja de procesos en modo de usuario consiste en el dispositivo de E/S controladores, cada conductor completamente aislado en un proceso separado protegido por la MMU y comunicarse con el núcleo a través de una API simple y con otros procesos de paso de mensajes. La siguiente capa está compuesta por servidores, incluido el archivo virtual servidor, el servidor de archivos MINIX, el administrador de procesos, el administrador de memoria virtual, y el servidor de

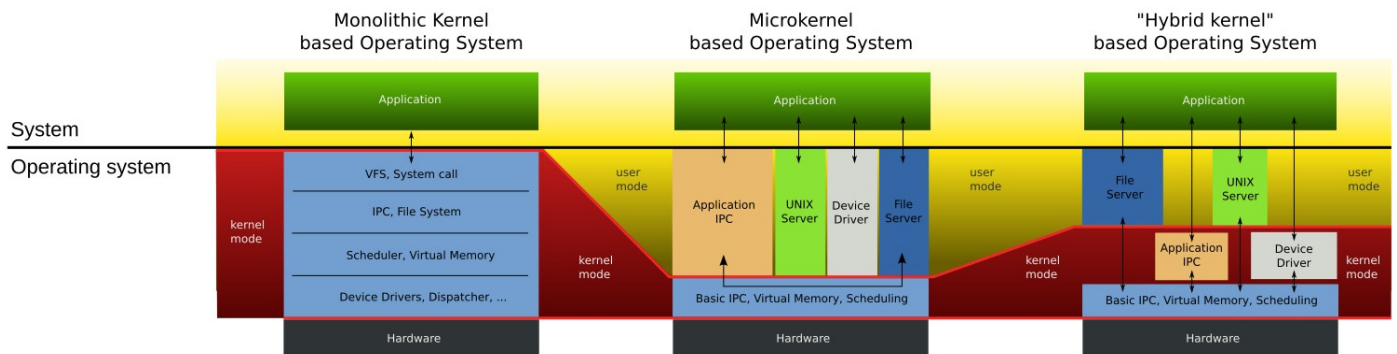
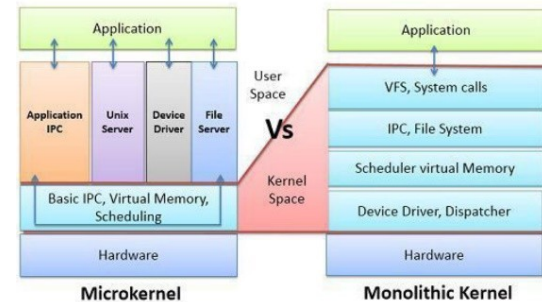
reencarnación. Sobre esta capa están los procesos normales de usuario, tales como X11, shells y programas de aplicación.

BSD – Berkeley Software Distribution

Tipos de Kernel

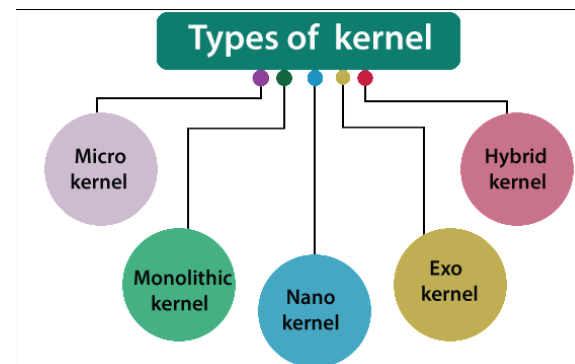
Existen básicamente dos tipos de estructuras de kernel:

- ♦ **Kernel Monolítico:** El kernel es un único programa (en realidad proceso) que se ejecuta continuamente en la memoria de la computadora intercambiándose con la ejecución de los procesos de usuario.
- ♦ **Micro Kernel:** el kernel sigue existiendo pero sólo implementa funcionalidad básica en el ring 0. Otros servicios se implementan en ring 1 o ring 2, estos servicios no son imprescindibles que se ejecuten en modo kernel exclusivamente.



Más sobre los tipos de Kernel

- ✚ **Monolithic Kernel:** - In a monolithic kernel, the kernel and operating system, both run in the same memory, and it is mainly used where security is not a major concern. The result of the monolithic kernel is fastly accessible. But in some situations, like if a device driver has a bug, then there may be chances of a whole system crash.
- ✚ **Microkernel:** - A Microkernel is the derived version of the monolithic kernel. In microkernel, the kernel itself can do different jobs, and there is no requirement of an additional GUI.
- ✚ **Nano kernel:** - Nano kernel is the small type of kernel which is responsible for hardware abstraction, but without system services. Nano kernel is used in those cases where most of the functions are set up outside.
- ✚ **Exo kernel:** - Exo kernel is responsible for resource handling and process protection. It is used where you are testing out an inhouse project, and in up-gradation to an efficient kernel type.



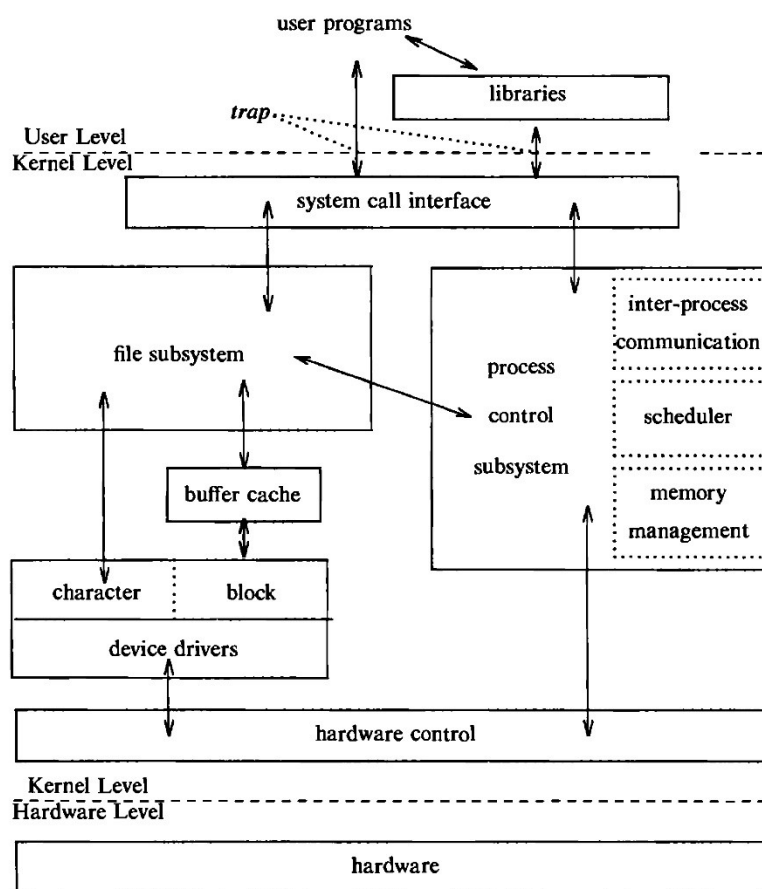
- ✚ Hybrid kernel: - Hybrid kernel is a mixture of microkernel and monolithic kernel. The Hybrid kernel is mostly used in Windows, Apple's macOS. Hybrid kernel moves out the driver and keeps the services of a system inside the kernel.

Más sobre el Kernel

Según VAHALIA, Uresh en su libro *"UNIX internals: the new frontiers"* de editorial Pearson Education India, 2008:

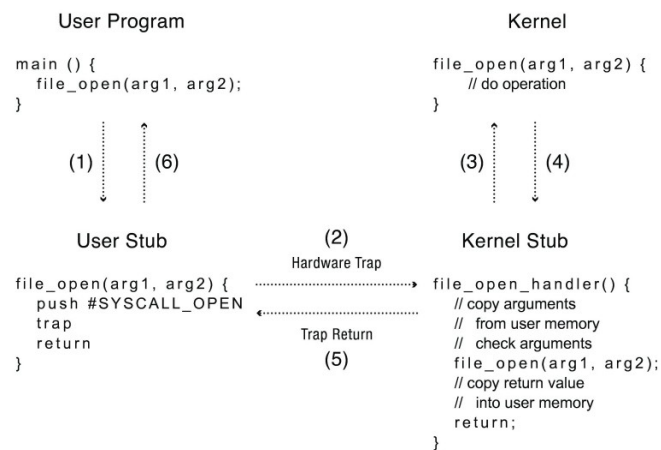
- Implementa el **Modelo de Procesos y otros servicios** del sistema.
- Reside en el disco en un archivo típicamente llamado **/vmlinuz**, **/vmunix** o **/unix** (dependiendo del vendedor de UNIX).
 1. Cuando el sistema se inicia, éste carga al Kernel desde el disco usando un procedimiento especial llamado bootstrapping.
 2. El Kernel inicializa el sistema y setea el ambiente para correr procesos.
 3. Entonces, el Kernel crea algunos procesos iniciales, los cuales a su vez crean otros procesos.
 4. Una vez cargado en la memoria el Kernel permanece allí hasta que se apague la computadora.
 5. Él administra los procesos y les provee de varios servicios (acceso a disco, acceso a network, acceso a usb, etc).

Una versión moderna del Kernel de Unix



Alguna funcionalidad que provee el Kernel en el sistema operativo UNIX

Los procesos de Usuario explícitamente requieren servicios al Kernel a través del uso de las **System Call**, que es el componente central del API (Application Programming Interface) de UNIX. El Kernel ejecuta los pedidos en nombre de los procesos que utilizan las System Calls.



Iniciar al Operativo y el

Sistema Kernel

El **kernel** es la **capa de software de más bajo nivel** en la computadora. Este contiene por un lado una **capa para la gestión de dispositivos específicos** y por otro una **serie de servicios para la gestión de dispositivos agnósticos del hardware** que son utilizados por las aplicaciones.

Estas dos capas suelen ser conocida como el **kernel** del sistema operativo. Cuando código fuente de esta capa es ejecutado la computadora pasa a un estado llamado **Modo Supervisor**.

El proceso de inicio de una computadora se divide esencialmente en 3 partes:

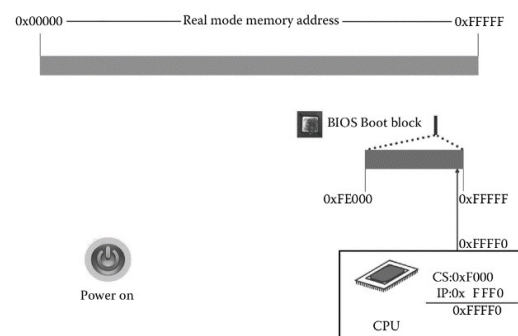
1. Booteo
2. Carga del Kernel
3. Inicio de las aplicaciones de Usuarios

Booteo o Bootstrap

Este proceso es denominado bootstrap, y generalmente depende del hardware de la computadora. En el se realizan los chequeos de hardware y se carga el bootloader, que es el programa encargado de cargar el Kernel del Sistema Operativo. Este proceso consta de 4 partes.

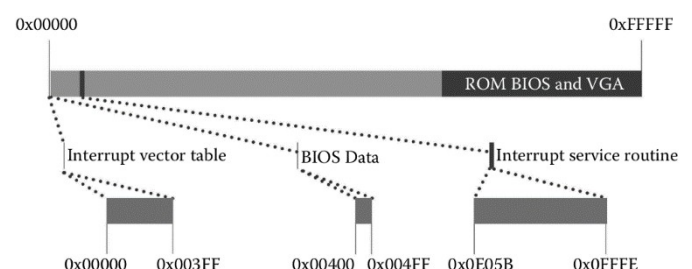
Booteo 1

Cargar el **BIOS (Basic Input/Output System)**: para eso apenas se enciende la PC, se carga CS con 0xFFFF y IP con 0x0000; por ende, la dirección de CS:IP es 0xFFFF0, justamente la dirección de memoria de la BIOS.



Booteo 2

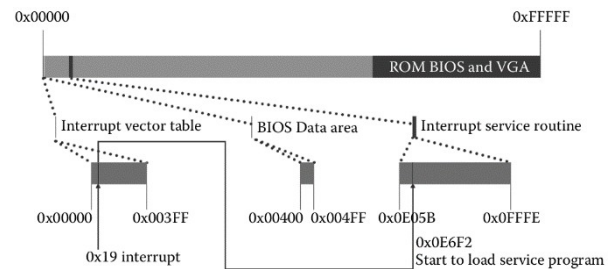
Crear la **Interrupt Vector Table** y cargar las rutinas de manejo de interrupciones en **Modo Real**: el BIOS pone la tabla de interrupciones en el inicio de la memoria 1 KB (0x00000-0x003FF), son 256 entradas de 4 bytes.



El área de datos del BIOS de unos 256 B (0x00400-0x004FF), y el servicio de atención de interrupciones (8 KB), 56 KB, después de la dirección 0x0E05B.

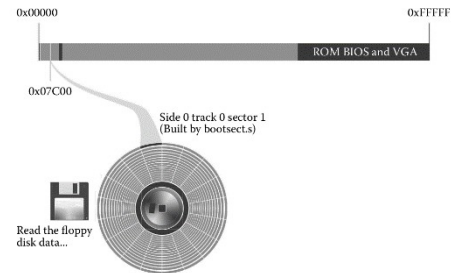
Booteo 3

La BIOS genera una interrupción 19 (INT 19) de la tabla de interrupciones la cual hace apuntar a CS:IP a 0x0E6F2.



Booteo 4

Lo cual hace ejecutar el servicio de interrupciones, el handler de dicha interrupción, que es leer el primer sector de 512 bytes del disco a memoria, y ahí termina.



Fase de inicio del Kernel

La función de arranque para el kernel (también llamado intercambiador o proceso 0) establece la gestión de memoria (tablas de paginación y paginación de memoria), detecta el tipo de CPU y cualquier funcionalidad adicional como capacidades de punto flotante, y después cambia a las funcionalidades del kernel para arquitectura no específicas de Linux, a través de una llamada a la función `start_kernel()`.

`start_kernel()` ejecuta una amplia gama de funciones de inicialización:

1. Establece el manejo de interrupciones (IRQ)
2. configura memoria adicional
3. comienza el proceso de inicialización (procesa el espacio del primer usuario y luego comienza la tarea inactiva a través de `cpu_idle()`).

En particular, el proceso de inicio del kernel también monta el disco RAM inicial ("initrd") que se ha cargado anteriormente como el sistema raíz temporal durante la fase de arranque.

Esto permite que los módulos controladores se carguen sin depender de otros dispositivos físicos y drivers y mantiene el kernel más pequeño.

El sistema de archivos raíz es cambiado más tarde a través de la llamada a `pivot_root()`, que desmonta el sistema de archivos temporal y lo reemplaza por el real una vez que éste sea accesible. La memoria utilizada por el sistema de archivos temporal es entonces recuperada.

Por lo tanto, el núcleo **inicializa los dispositivos, monta el sistema de archivos raíz** especificado por el gestor de arranque como de sólo lectura, **y se ejecuta Init** (/sbin/init), que es designado como el **primer proceso ejecutado** por el sistema (PID=1). También puede ejecutar opcionalmente `initrd` para permitir instalar y cargar dispositivos relacionados (disco RAM o similar), para ser manipulados antes de que el sistema de archivos raíz está montado.

En este punto, con las interrupciones habilitadas, el programador puede tomar el control de la gestión general del sistema, para proporcionar multitarea preventiva, e iniciar el proceso para continuar con la carga del entorno de usuario en el espacio de usuario.

El proceso de inicio

El trabajo de **Init** es “**conseguir que todo funcione como debe ser**” una vez que el kernel está totalmente en funcionamiento. En esencia, establece y opera todo el espacio de usuario. Esto incluye:

1. la comprobación y montaje de sistemas de archivos
2. la puesta en marcha los servicios de usuario necesarios y, en última instancia, cambiar al entorno de usuario cuando el inicio del sistema se ha completado.

Es similar a los procesos Init de Unix y BSD, de la que deriva, pero en algunos casos se ha apartado o se hicieron a la medida.

En un sistema Linux estándar, Init se ejecuta con un parámetro, conocido como nivel de ejecución, que tiene un valor entre 1 y 6, y que determina que subsistemas pueden ser operacionales.

Cada nivel de ejecución tiene sus propios scripts que codifican los diferentes procesos involucrados en la creación o salida del nivel de ejecución determinado, y son estas secuencias de comandos los necesarios en el proceso de arranque. Los scripts de Init se localizan normalmente en directorios con nombres como “/etc/rc...”.

El archivo de configuración de más alto nivel para Init es /etc/inittab.

Durante el arranque del sistema, se verifica si existe un nivel de ejecución predeterminado en el archivo /etc/inittab, si no, se debe introducir por medio de la consola del sistema. Después se procede a ejecutar todos los scripts relativos al nivel de ejecución especificado.

Después de que se han dado lugar todos los procesos especificados, Init se aletarga, y espera a que uno de estos tres eventos suceda:

- que procesos comenzados finalicen o mueran;
- un fallo de la señal de potencia (energía);
- una petición a través de /sbin/telinit para cambiar el nivel de ejecución.

Habitualmente en una instalación desktop se ejecuta comúnmente /sbin/init

4 - La abstracción del proceso

De programa a proceso

El programa

1. Programador edita código fuente
2. El compilador compila el source code en una secuencia de instrucciones de máquina y datos llamada.
3. El compilador genera esa secuencia y posteriormente se guarda en disco: programa ejecutable.

El programa: Edición

Una vez que se edita un programa en cualquier lenguaje de programación se debe compilar para poder obtener un programa ejecutable.

Compilar que en la actualidad suele considerarse como una actividad atómica, está separada en varias etapas en las cuales se utilizan distintas herramientas y se obtienen distintos archivos en distintos formatos hasta alcanzar el formato ejecutable. Sin el compilador el programa se limita a ser:

	#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
	35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
#include<stdio.h>	h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
int main(){	104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
printf("hello,world\n");	\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
}	10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
	l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
	108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

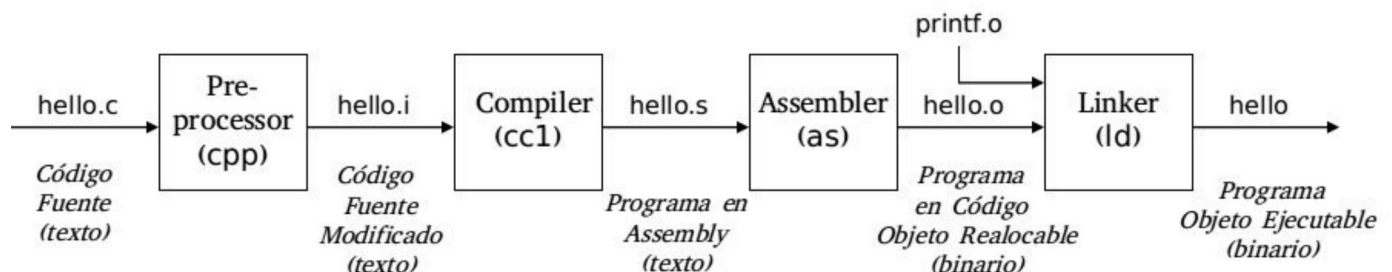
Las fase de compilación son 4 etapas, que utilizan 4 herramientas para llevar a cabo cada una de estas etapas. Estas herramientas son:

1. **El preprocesador.**
2. **El compilador**
3. **El ensamblador**
4. **El link editor o linker.**

Estas en conjunto son conocidas como un **sistema de compilación**.

El programa: Compilación

- ❖ **La fase de procesamiento:** El preprocesador (cpp) modifica el código de fuente original de un programa escrito en C de acuerdo a las directivas que comienzan con un caracter(#). El resultado de este proceso es otro programa en C con la extinción .i
`gcc -save-temps ( -s, -c and -E)`
- ❖ **La fase de compilación:** El compilador (cc) traduce el programa .i a un archivo de texto .s que contiene un programa en lenguaje assembly.
- ❖ **La fase de ensamblaje:** A continuación el ensamblador (as) traduce el archivo .s en instrucciones de lenguaje de máquina empaquetándolas en un formato conocido como *programa objeto realocable*. Este es almacenado en un archivo con extensión .o
- ❖ **La fase de link edición:** Generalmente los programas escritos en lenguaje C hacen uso de funciones que forman parte de la **biblioteca estándar de C** que es provista por cualquier compilador de ese lenguaje. Por ejemplo la función printf(), la misma se encuentra en un archivo objeto pre compilado que tiene que ser mezclado con el programa que se está compilando, para ello el linker realiza esta tarea teniendo como resultado un archivo objeto ejecutable.



El programa: Formato ejecutable

Un **programa** es un archivo que posee toda la información de cómo construir un proceso en memoria [KER](cap. 6). Un programa contiene:

- **Formato de Identificación Binaria:** Cada archivo ejecutable posee META información describiendo el formato ejecutable. Esto permite al kernel interpretar la información contenida en el mismo archivo.

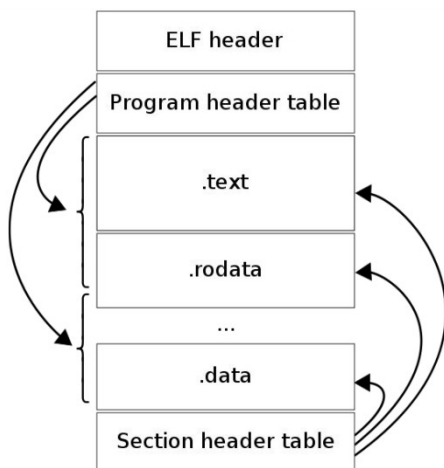
Recordar que por algo la salida del compilador C es por defecto *a.out* que viene de “**assembler output**”. Posteriormente, dos formatos han sido utilizados en la familia de sistemas operativos unix:

- o **COFF** (Common Object File Format”) utilizado en la mayoría de las versiones de System V.

- o **ELF** (Executable and Linking Format) utilizado en la actualidad.

- **Instrucciones de Lenguaje de Máquina:** Almacena el código del algoritmo del programa.
- **Dirección del Punto de Entrada del Programa:** Identifica la dirección de la instrucción con la cual la ejecución del programa debe iniciar.
- **Datos:** El programa contiene valores de los datos con los cuales se deben inicializar variables, valores de constantes y de literales utilizadas en el programa.
- **Símbolos y Tablas de Realocación:** Describe la ubicación y los nombres de las funciones y variables de todo el programa, así como otra información que es utilizada por ejemplo para debugg.
- **Bibliotecas Compartidas:** describe los nombres de las bibliotecas compartidas que son utilizadas por el programa en tiempo de ejecución así como también la ruta del linker dinámico que debe ser usado para cargar dicha biblioteca.
- **Otra información:** El programa contiene además otra información necesaria para terminar de construir el proceso en memoria.

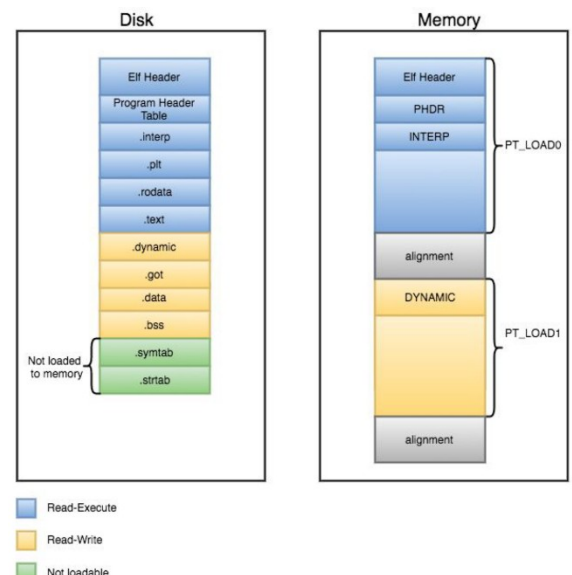
Un programa en Linux: ELF



ELF: Extensible Linking Format

Magic number 0x7F 'E' 'L' 'F'

<https://greek0.net/elf.html>



- ▢ **readelf** is a Unix binary utility that displays information about one or more ELF files. A free software implementation is provided by GNU Binutils.
- ▢ **elfutils** provides alternative tools to GNU Binutils purely for Linux.[11]
- ▢ **objdump** provides a wide range of information about ELF files and other object formats. objdump uses the Binary File Descriptor library as a back-end to structure the ELF data.
- ▢ The Unix **file** utility can display some information about ELF files, including the instruction set architecture for which the code in a relocatable, executable, or shared object file is intended, or on which an ELF core dump was produced.

El comando readelf

```

hexdump -C ./compile_me.elf | head -n 10
00000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00 00 |.ELF.....|
00000010 02 00 3e 00 01 00 00 00 30 04 40 00 00 00 00 00 00 |>.....0.0...|
00000020 40 00 00 00 00 00 00 00 00 0a 00 00 00 00 00 00 00 |.....0.0.0...|
00000030 00 00 00 00 40 00 38 00 00 00 40 00 1f 00 1c 00 00 |.....0.0.0...|
00000040 00 00 00 00 05 00 00 00 40 00 00 00 00 00 00 00 00 |.....0.0.0...|
00000050 45 00 00 00 00 00 00 00 40 00 40 00 00 00 00 00 00 |.....0.0.0...|
00000060 f8 01 00 00 00 00 00 00 f8 01 00 00 00 00 00 00 00 |.....0.0.0...|
00000070 00 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00 00 |.....0.0.0...|
00000080 38 02 00 00 00 00 00 00 38 02 40 00 00 00 00 00 00 |.....0.0.0...|
00000090 38 02 40 00 00 00 00 00 1c 00 00 00 00 00 00 00 00 |.....0.0.0...|
kh3n@kh3n-nacht:~/Research/ELF/tests/baseline/compile_options$

readelf -l ./compile_me.elf | head -n 20

Elf file type is EXEC (Executable file)
Entry point 0x400430
There are 9 program headers, starting at offset 64

Program Headers:
Type           Offset             VirtAddr           PhysAddr          Flags          Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040 R E 8
INTERP         0x0000000000000238 0x0000000000000238 0x0000000000000238 R 1
LOAD           0x000000000000001c 0x000000000000001c 0x000000000000001c R 1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000 R E 8
LOAD           0x0000000000000010 0x0000000000000010 0x0000000000000010 R E 10
LOAD           0x0000000000000228 0x0000000000000228 0x0000000000000228 RW 20000

```

Ver el SHT (Section Header Table) readelf -S <archivo>
Ver el PHT (Program Header table) readelf -l <archivo>
Ver el ELF Header readelf -h <archivo>
Ver la Relocation Table readelf -r <archivo>
Ver el dump hexa hexdump -C archivo | head -n 10

De programa a proceso

Un **programa es algo _sin vida_**, un conjunto de instrucciones y datos que esperan en algún lugar del disco para saltar a la acción!!!

- Y el Sistema Operativo es quien toma ese puñado de bytes y es el SO que transforma ese programa en algo útil, mediante el **Kernel**.
- Uno nunca tiene que ser consciente si la CPU está disponible, etc. sólo ejecuta el programa.

En este punto se tiene un programa capaz de ser ejecutado por una computadora, es función del sistema operativo hacer que este se ejecute, para ello existe el concepto de proceso.

El **Sistema Operativo** más precisamente el **Kernel** se encarga de:

1. Cargar instrucciones y Datos de un programa ejecutable en memoria.
2. Crear el Stack y el Heap
3. Transferir el Control al programa
4. Proteger al SO y al Programa

El proceso

“Un proceso es **la ejecución de un programa** de aplicación con derechos restringidos; el proceso **es la abstracción** que provee el Kernel del sistema operativo para la ejecución protegida” - [DAH]

“Es **simplemente** un programa que se está ejecutando en un instante dado” - [ARP]

“Un Proceso **es la instancia de un programa en ejecución**” - [VAH]

“Un proceso **es un programa en medio de su ejecución**” - [LOV]

Por supuesto que no está más lejos de la verdad, decir que un proceso es sólo un programa en ejecución. Un proceso incluye:

- Los Archivos abiertos
- El estado completo del procesador
- las señales(signals) pendientes
- Un espacio de direcciones de memoria
- Datos internos del kernel

➤ **Uno o más hilos de Ejecución. Cada thread contiene:**

- o Un único contador de programa

- o Un Stack
- o Un Conjunto de Registros
- o Una sección de datos globales

“Un **Proceso** es una entidad abstracta, definida por el Kernel, en la cual los recursos del sistema son asignados” [KER]

La virtualización

Definición

Crear una abstracción que haga que un dispositivo de hardware sea mucho más fácil de utilizar .

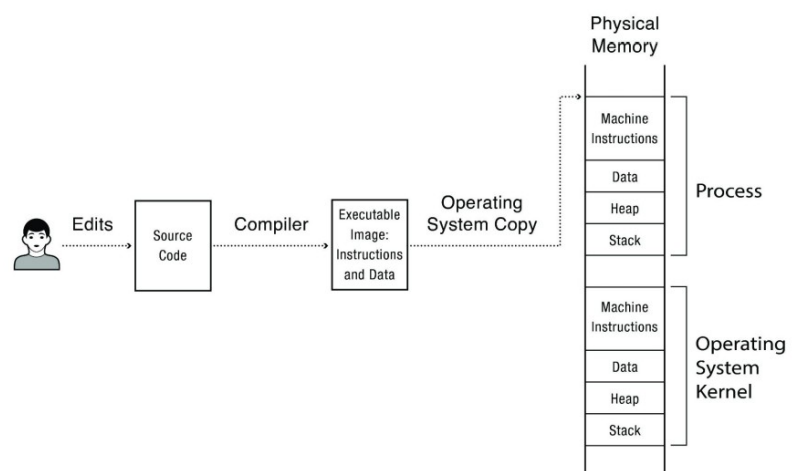
Particularmente el concepto de proceso es el más importante de todos los conceptos que se ven en sistemas operativos. En los sistemas operativos modernos proporciona dos tipos de virtualización:

1. Virtualización de memoria
2. Virtualización de procesador

Virtualización de memoria

La virtualización de memoria le hace creer al proceso que este tiene toda la memoria disponible para ser reservada y usada como si este estuviera siendo ejecutado sólo en la computadora (*ilusión*). Todos los procesos en Linux, está dividido en 4 segmentos:

- **Text:** Instrucciones del Programa.
- **Data:** Variables Globales (extern o static en C)
- **Heap:** Memoria Dinámica Alocable
- **Stack:** Variable Locales y trace de llamadas

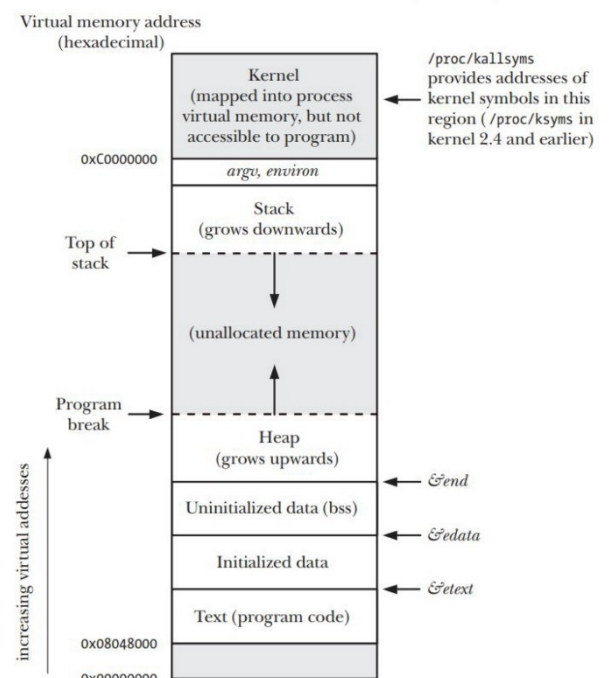


El proceso: Espacio de direcciones

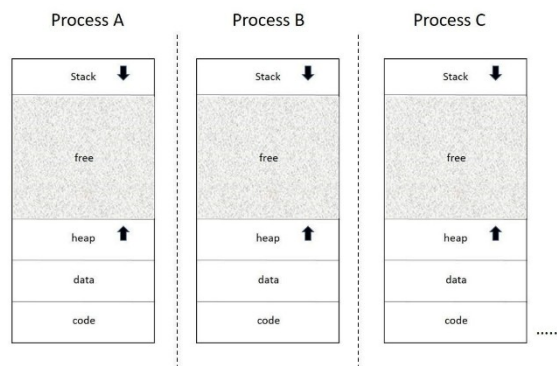
Todas estas secciones pertenecientes a un proceso se denominan **espacio de direcciones** del proceso.

Para ejecutar un programa el sistema operativo:

1. Copia las instrucciones en la sección .code y los datos en la sección .data, desde el programa ejecutable residente en disco hacia la memoria física,



- Además el sistema operativo setea una región de memoria llamada **execution stack (.stack)**, que mantiene el estado de las variables locales durante las llamadas a los procedimientos.
- El sistema operativo también setea una región de memoria llamada **heap**, destinada a alojar cualquier estructura de datos alocada en forma dinámica que el programa pueda necesitar.



Todos los procesos tienen la misma estructura del Address Space.

Protección de memoria

Para que un proceso se ejecute tiene que estar residente en memoria, pero a su vez el sistema operativo tiene que estar residente en memoria.

- El **proceso** tiene que estar en memoria para poder ejecutarse.

Mientras que:

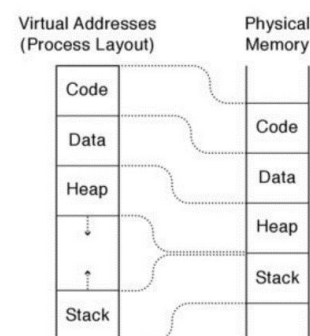
- El **sistema operativo** tiene que estar ahí para:
 - iniciar la ejecución del programa
 - manejar las interrupciones
 - y/o atender las systems call.

Es más, otros procesos podrían estar simultáneamente en memoria para poder compartir la memoria de forma segura, para ello el sistema operativo tiene que poder configurar el hardware de forma tal que cada proceso pueda leer y escribir solo su propia memoria (No la memoria del sistema operativo tampoco la de otros procesos. Ya que sino el proceso en cuestión podría incluso modificar al Kernel del sistema operativo. Para ello el Hardware debe proveer un mecanismo de protección de memoria (que se verán detalladamente más adelante)).

Uno de estos mecanismos es denominado **Memoria Virtual**, la memoria virtual es una **abstracción** por la cual la **memoria física** puede ser compartida por diversos procesos.

Un componente clave de la memoria virtual son las **direcciones virtuales**, con las direcciones virtuales, para cada proceso su memoria inicia en el mismo lugar, la dirección 0.

Cada proceso piensa que tiene toda la memoria de la computadora para sí mismo, si bien obviamente esto en la realidad no sucede. El hardware traduce la dirección virtual a una dirección física de memoria.



Traducción de direcciones

Se traduce una **Dirección Virtual** (emitida por la CPU) en una **Dirección Física** (la memoria). Este mapeo se realiza por hardware, más específicamente por **Memory Management Unit (MMU)**.



Virtualización de procesador

La **virtualización de procesamiento** es la forma de virtualización más primitiva, **consiste en dar la ilusión de la existencia de un único procesador para cualquier programa que requiera de su uso**. De esta forma, se provee:

■ Simplicidad en la programación

- o Cada proceso cree que tiene toda la CPU.
- o Cada proceso cree que todos los dispositivos le pertenecen.
- o Distintos dispositivos parecen tener el mismo nivel de interfaces.
- o Las interfaces con los dispositivos son más potentes que el bare metal.

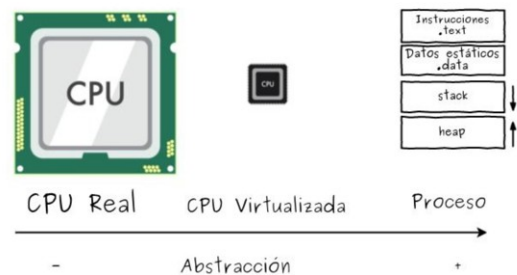
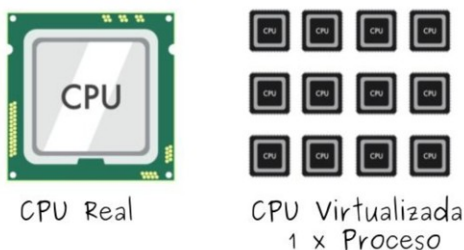
■ Aislamiento frente a Fallas:

- o Los procesos no pueden directamente afectar a otros procesos.
- o Los errores no colapsan toda la máquina.

¿Cómo se provee la ilusión de tener varios CPUs?

El SO crea esta **ilusión** mediante la **virtualización** de la CPU a través del kernel:

Viéndolo desde el punto de vista de la abstracción y virtualización:



Proceso: El contexto

Cada **proceso** tiene un **contexto** bien definido que comprende toda la información necesaria para describir completamente al mismo.

El **contexto** de un proceso **es la información necesaria para describir al proceso** [VAH p.26]

El contexto de un proceso consiste:

1. **User Address Space:** normalmente está dividido en varias áreas: text, data, stack y heap,
2. **Control Information:** el kernel utiliza dos estructuras principales para mantener información de control de un proceso- la **u area** y la estructura *proc*. Cada proceso

además tiene su propio **kernel stack** y **mapas de traducción de direcciones**.

3. **Credentials**: Las credenciales del proceso incluyen los groups IDs y user id, asociados con el.
4. **Variables de entorno**: son un conjunto de strings del formato variable=valor que son heredadas del [proceso padre].
5. **Hardware Context: Esto contine el contenido de los registros de proposito general, y de un conjunto especial de registros del system**:
 - El Program Counter (*PC*)
 - El Stack Pointer (*SP*)
 - El Processor Status Word (*PWD*)
 - Memory Management Registers
 - Los Registros de la Unidad de Punto Flotante.

Según bach: “el contexto de un proceso comprende, el contenido del address space, el contenido de los registros de hardware y las estructuras de datos que pertenecen al kernel relacionadas con el proceso”.

En forma resumida, el contexto de un proceso consiste según [BCH] en la unión de **user-level context**, **register context** y **system level context**:

1. El contexto User-Level

Consiste en las secciones que forman parte de **Virtual Address Space** del proceso.

- a. **text**,
- b. **data**,
- c. **stack**,
- d. **heap**

2. El contexto Register

El contexto de registro consiste de los siguientes componentes:

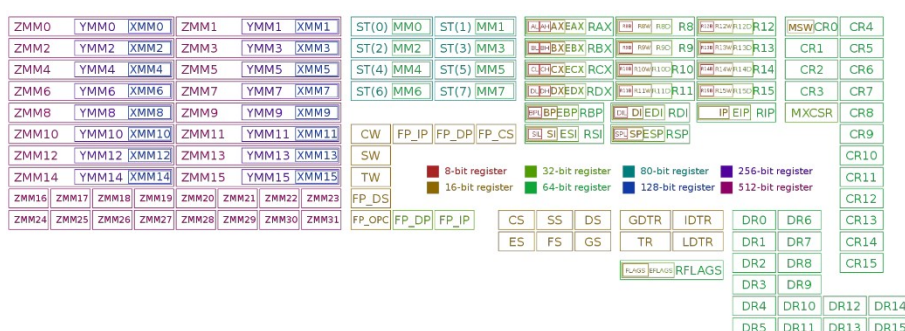
a. Contador de Programa

b. Registro de Estado del Procesador (PS)

c. Stack Pointer

d. Los Registro de propósito General

i. En x86:



3. El contexto System-level

Consiste en :

- a. La entrada en la **Process Table Entry**
- b. La **u area**
- c. La **Process Region Entry**, **Region Table** y **Page Table** que definen el mapeo de la **memoria virtual** vs **memoria física** del proceso.

- d. El Stack del Kernel, contiene los stack frames de las llamadas al kernel hechas por el proceso.

La U area y estructura Proc

La información de control sobre un proceso es mantenida en dos **per process data structures**. Estas estructuras dependen de la implementación del kernel mismo, la **proc structure** es una entrada en una tabla conocida como **process table**.

La **U area o user area** es parte del espacio del proceso, se mapea y es visible por el proceso solo cuando este está siendo ejecutado. La user area contiene información que solo es necesario acceder cuando el proceso se está ejecutando. Por el contrario la proc structure posee información necesaria incluso cuando el proceso no se está ejecutando.

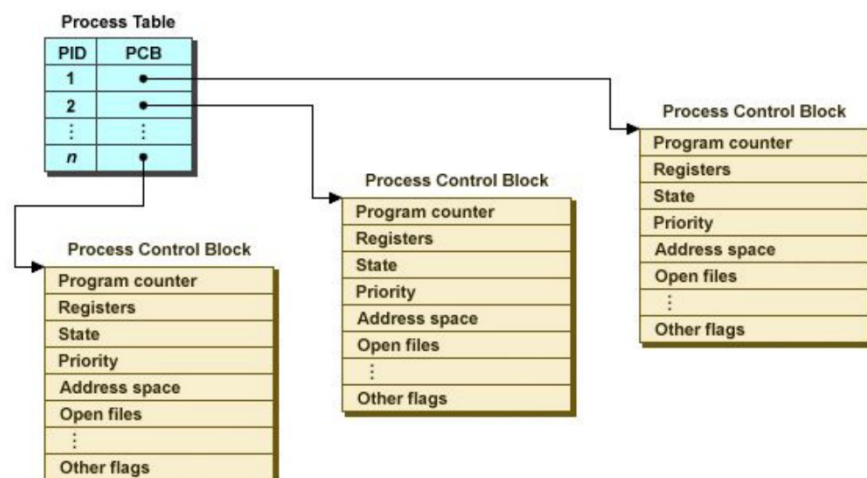
Contenido de la user area <arch/x86/include/asm/user.h>:

- La Process Control Block - guarda el hardware context cuando el proceso no se está ejecutando
- Un puntero a la proc structure del proceso
- El UID y GID real
- Argumentos para, y valores de retorno o errores hacia, la system call actual
- Manejadores de Señales
- Información sobre las áreas de memoria text, data, stack, heap y otra información.
- La tabla de descriptores de archivos abiertos (Open File dscriptor Table).
- Un puntero al directorio actual
- Datos estadísticos del uso de la CPU, información de perfilado, uso de disco y límites de recursos.

Process Table

Contenidos de la estructura **Proc**:

- Identificación: cada proceso tiene un identificador único o *process ID* (PID) y además perteneces a un determinado grupo de procesos.
- Ubicación del mapa de direcciones del Kernel del u area del proceso.
- Estado actual del proceso
- Un puntero hacia el siguiente proceso en el planificador y al anterior.
- Prioridad
- Información para el manejo de señales.
- Información para la administración de memoria.



En linux la *proc structure* se conoce como la *task structure*.

En C esto se traduce a:

```
// the registers xv6 will save and restore
//to stop and subsequently restart a process
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;           // Start of process memory
    uint sz;             // Size of process memory
    char *kstack;        // Bottom of kernel stack
    // for this process
    enum proc_state state; // Process state
    int pid;             // Process ID
    struct proc *parent;  // Parent process
    void *chan;          // If non-zero, sleeping on chan
    int killed;          // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;    // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf; // Trap frame for the
    // current interrupt
};
```

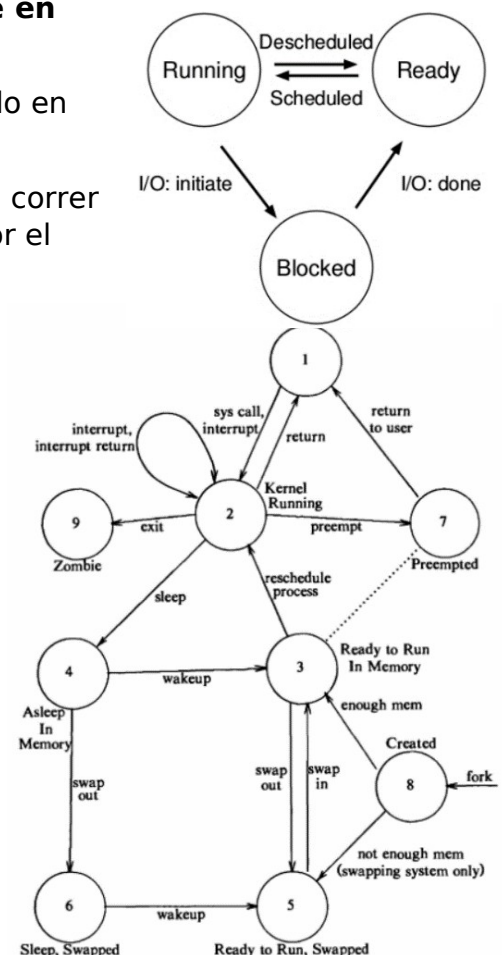
Estados de un proceso

En una visión simplificada un proceso puede encontrarse en los siguientes estados:

- **Corriendo (Running):** el proceso se encuentra corriendo en un procesador. Está ejecutando instrucciones.
- **Listo (Ready):** en este estado el proceso está listo para correr pero por algún motivo el SO ha decidido no ejecutarlo por el momento.
- **Bloqueado (Blocked):** en este estado el proceso ha ejecutado algún tipo de operación que hace que éste no esté listo para ejecutarse hasta que algún evento suceda.

Estados en Unix System V

1. **Corriendo User Mode (Running User Mode):** El proceso se encuentra corriendo en un procesador. Está ejecutando instrucciones.
2. **Corriendo kernel Mode (Running Kernel Mode):** d



3. **Listo para Correr en Memoria (Ready to Run on Memory):** En este estado el proceso está listo para correr pero por algún motivo el SO ha decidido no ejecutarlo por el momento.
4. **Durmiendo en Memoria (Asleep In Memory):** El proceso está bloqueado en memoria.
5. **Listo para Correr pero Swapeado (Ready to Run but wapped):** El proceso está bloqueado en memoria secundaria.
6. **Durmiendo en Memoria Secundaria (Asleep Swapped):** El proceso se encuentra bloqueado en memoria secundaria.
7. **Desalojado (Preempt):** Es igual a 1 pero un proceso que pasó antes por Kernel mode solo puede pasar a preentive.
8. **Creado (Created):** El proceso está recién creado y en un estado de transición.
9. **Zombie (Zombie):** El proceso ejecutó la S.C. exit(), ya no existe más, lo único que queda es el exit state

El API resumida

1. **fork():** Crea un proceso y devuelve su id.
2. **exit():** Termina el proceso actual.
3. **wait():** Espera por un proceso hijo.
4. **kill(pid):** Termina el proceso cuyo pid es el parámetro.
5. **getpid():** Devuelve el pid del proceso actual.
6. **exec(filename, argv):** Carga un archivo y lo ejecuta.
7. **sbrk(n):** Crece la memoria del proceso en n bytes.

System Call getpid() y getppid()

Esta system call `_getpid()` permite determinar el PID (Process IDentification).

Originalmente en UNIX el PID es un número positivo hasta 32,768, una vez alcanzado ese número se resetea a 300 y se vuelven a asignar esos valores nuevamente. Esto pasaba en Linux, hasta la versión 2.6 del kernel, en la cual dependiendo de la plataforma ese valor es ajustable, dependiendo de la plataforma:

- o La versión `getppid()` devuelve el PID del proceso padre del proceso.
- o Para validar esto se puede ejecutar el comando **pstree**

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

Creación de un proceso

system call `_fork_`.

La única forma de que un usuario cree un proceso en el sistema operativo `_UNIX_` es llamando a la system call `_fork_`.

El proceso que invoca a `_fork_` es llamado proceso **padre**, el nuevo proceso creado es llamado **hijo**.

¿Qué hace fork?:

1. Crea y asigna una nueva entrada en la **Process Table** para el nuevo proceso.
2. Asigna un número de ID **único** al proceso hijo.
3. Crea una copia lógica del contexto del proceso padre, algunas de esas partes pueden ser compartidas como la sección **text**
4. Realiza ciertas operaciones de I/O.
5. Devuelve el número de ID del hijo al proceso padre, y un 0 al proceso hijo

¿Qué hace _fork(), el algoritmo?

La implementación de esta system call no es para nada trivial ya que cuando el proceso hijo inicia a ejecutarse parece hacerlo casi en el aire:

- chequear que haya recursos en el kernel;
- obtener una entrada libre de la Process Table, como un PID único;
- chequear que el usuario no esté ejecutando demasiados procesos;
- macar al proceso hijo en estado “siendo creado”;
- copiar los datos de la entrada en la Process Table del padre a la del hijo;
- incrementar el contador del current directoty inode;
- incrementar el contador de archivos abiertos en la File Table;
- hacer una copia del contexto del padre en memoria;
- crear un contexto a nivel sistema falso para el hijo;
 - o el contexto falso contiene datos para que el hijo se reconozca a sí mismo
 - o y para que tenga un punto de inicio cuando el planificador lo haga ejecutarse;

```
if (el proceso en ejecución es el padre) {  
    cambiar el estado del hijo a "ready to run";  
    return ( ID del hijo);  
} else { /* se esta ejecutando el hijo */  
    inicializar algunas cosas;  
    return 0;  
}
```

Aspectos Curiosos de fork()

- ★ **Una llamada dos valores de retorno:** fork se llama una vez desde el proceso padre, pero devuelve dos valores uno valor al proceso padre y otro valor al proceso hijo.
- ★ **Ejecución concurrente:** ambos procesos se ejecutan en forma concurrente por lo cual no hay determinismo en el orden de ejecución.
- ★ **Address space duplicados pero separados:** si se pudieran para el **padre** y el **hijo** después de la ejecución de fork, se vería que el address space de cada proceso son idénticos.
- ★ **Archivos compartidos:** la **user file descriptor table** es heredada con todos sus archivos en el mismo estado

System Call _exit()

Generalmente un proceso tiene dos formas de terminar:

1. La **anormal**: a través de recibir una señal cuya acción por defecto es terminar el programa.
2. La **normal**: a través de invocar a la system call *exit()*

Esta system call generalmente no es utilizada, en su lugar se utiliza *exit()* de la biblioteca estándar de C.

¿Qué hace exit(), el algoritmo?:

- Ignora todas las signals.
- Cierra todos los archivos abiertos
- En consecuencia se liberan todos los locks mantenidos por este proceso sobre esos archivos
- Libera el directorio actual
- Los segmentos de memoria compartida del procesos se separan
- los contadores de los semáforos son actualizados
- Libera todas las secciones y memoria asociada al proceso
- Registra información sobre el proceso (accounting record)
- Pone el estado del proceso en “zombie”
- Le asigna el parent PID de los procesos hijos al PID de *init*
- le manda una signal o señal de muerte al proceso padre
- context switch

Esta función de la biblioteca estándar de C :

```
#include <stdlib.h>

void exit(int status);
```

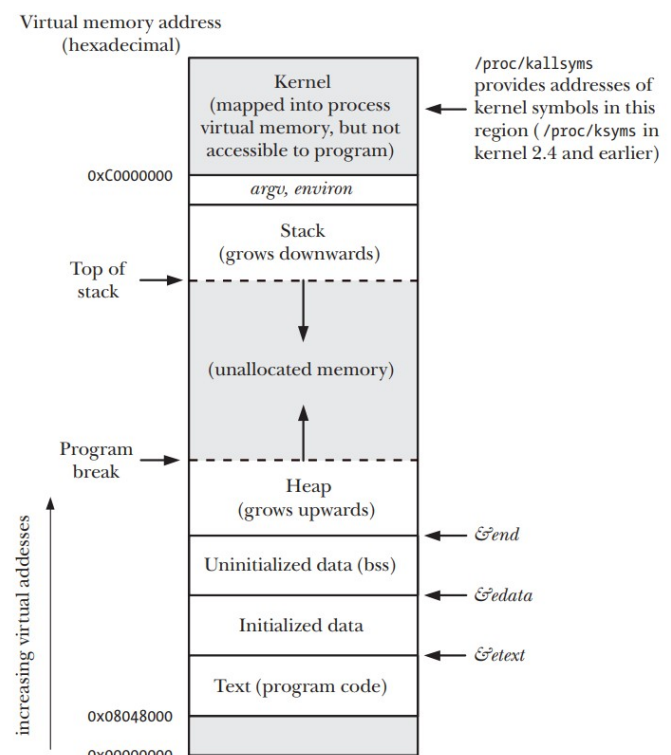
- llama a los Exit Handler que son dos funciones llamadas *on_exit()* y *atexit()*
- los streams de stdio son flushados (buffer->disco)
- se llama a la system call *_exit()*.

System Call brk()

Para entender esta system call es necesario entender el layout o estructura real de la memoria de un proceso en linux, [KER] (cap 6, pag.119):

Como puede apreciarse el final del heap se denomina *break*. El heap aumenta hacia direcciones altas, crece hacia arriba (grows upwards) y el stack se crece hacia direcciones bajas (grows downwards).

Un proceso puede reservar memoria para sí mismo incrementando el tamaño del heap, recordar que el



límite actual del heap se denomina *break*. Para reservar memoria, un programa en C normalmente utiliza la familia de funciones *malloc()*. Esta función está basada en la system call **brk()**.

Redimensionar el heap (reservando o liberando memoria) es tan simple como pedirle al kernel que ajuste su idea de **dónde** el break del proceso está.

Inicialmente el *break* del programa está ubicado justo en el final de datos no inicializados.

Después que *brk()* se ejecuta, el *break* es incrementado, el proceso puede acceder a cualquier memoria en la nueva área reservada, pero no accede directamente a la memoria física.

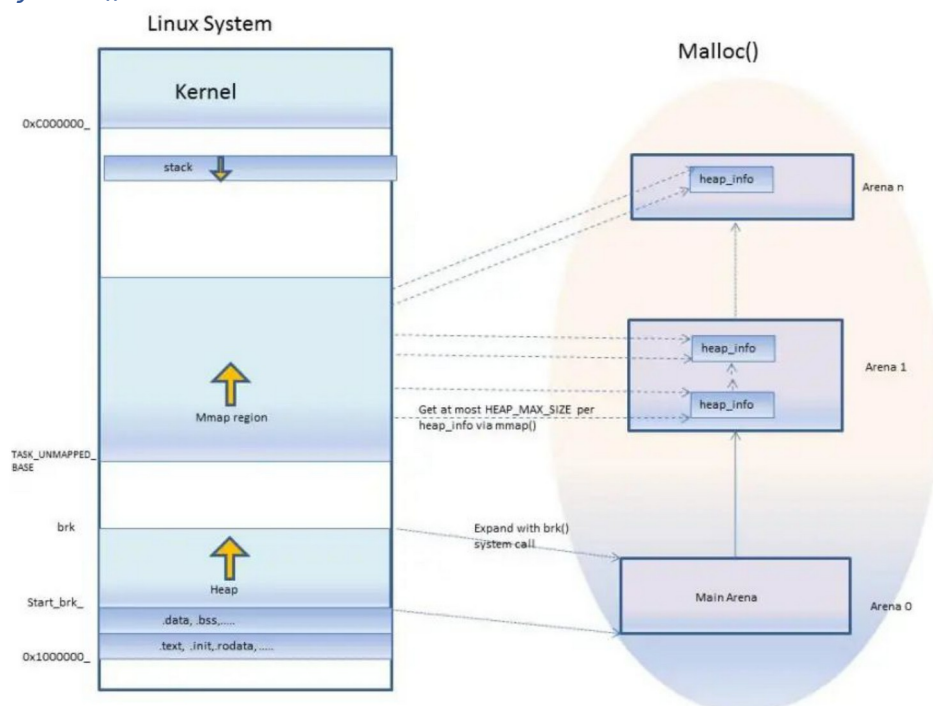
Esto se realiza automáticamente por el kernel en el primer intento del proceso en acceder al área reservada.

```
#include <unistd.h>

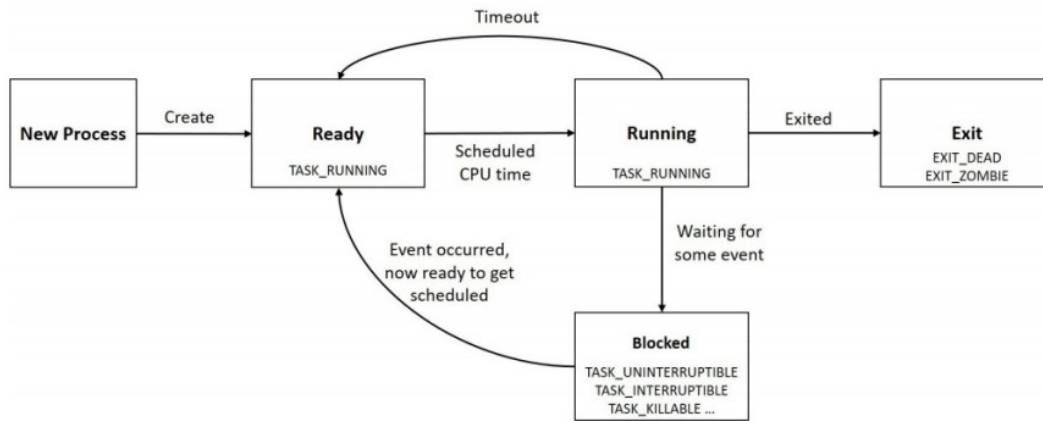
int brk(void *addr);
void *sbrk(intptr_t increment);
```

El parámetro de *sbrk()* es la dirección exacta donde el nuevo *break* debe estar. Por otro lado en **sbrk()** se pasa el incremento al cual se le sumará al viejo *break* para setear el nuevo *break*. Si se ejecuta *sbrk(0)* se obtiene la dirección del *break* actual [KER cap 7].

brk Vs. *malloc()* y *free()*

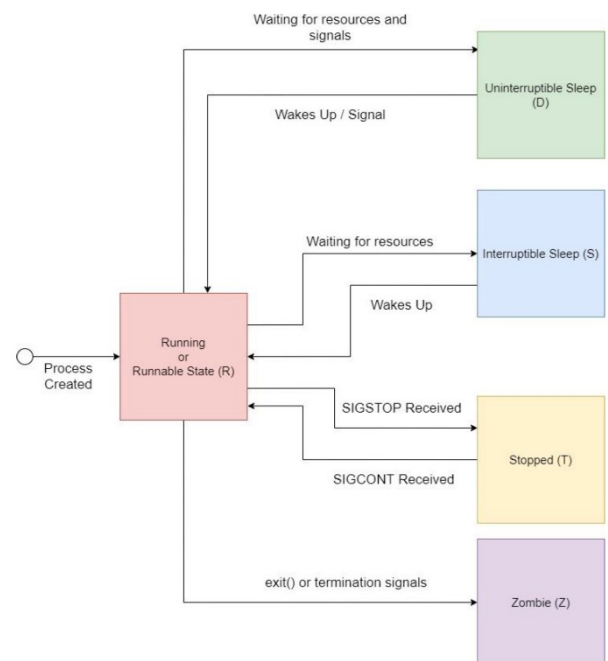


Estados de un proceso en Linux



Dado que en Linux los procesos son denominados **tasks**, los estados de una task pueden ser:

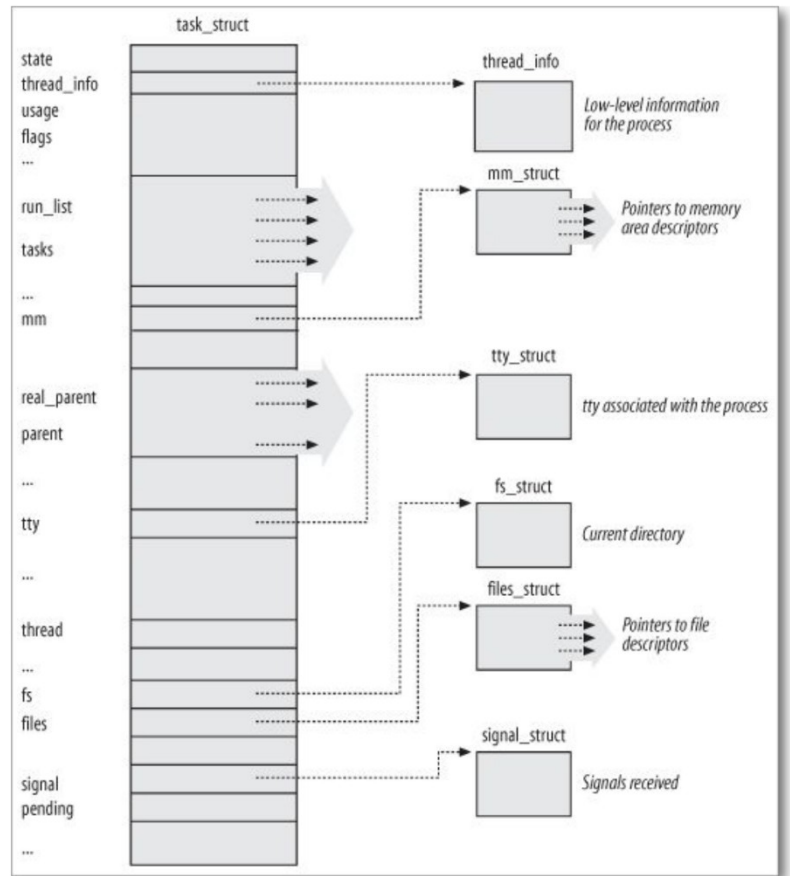
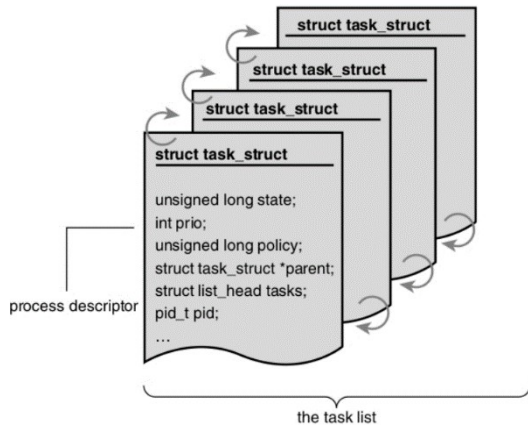
1. **TASK_RUNNING (0):** El proceso está o ejecutándose o peleando por CPU en la cola de run del planificador.
2. **TASK_INTERRUPTIBLE (1):** El proceso se encuentra en un estado de espera interrumpible; este queda en este estado hasta que la condición de espera eventualmente sea verdadera, por ejemplo un dispositivo de I/O está listo para ser utilizado, comienza de su time slice, etc. Mientras el proceso está en este estado, cualquier señal (signal) generada para el proceso es entregada al mismo, causando que este se despierte antes que la condición de espera se cumpla.
3. **TASK_KILLABLE:** este estado es similar al TASK_INTERRUPTIBLE, con la excepción que las interrupciones pueden ocurrir en fatal signals.
4. **TASK_UNINTERRUPTIBLE (2):** El proceso está en un estado de ininterrupción similar al anterior pero no podrá ser despertado por las señales que le lleguen. Este estado es raramente utilizado.
5. **TASK_STOPPED (4):** El proceso recibió una señal de STOP. Volverá a running cuando reciba la señal para continuar (SIGCONT).
6. **TASK_TRACED (8):** Un proceso se dice que está en estado de trace, cuando está siendo revisado probablemente por un debugger.
7. **EXIT_ZOMBIE (32):** El proceso está terminado, pero sus recursos aún no han sido solicitados.
8. **EXIT_DEAD (16):** El proceso Hijo ha terminado y todos los recursos que este mantenía para sí se han liberado, el padre posteriormente obtiene el estado de salida del hijo usando wait.



Desde el Kernel de Linux

El kernel almacena la lista de procesos en una lista circular doblemente enlazada llamada **task list**.

Cada elemento de la task list es un descriptor de proceso (*process descriptor*) del tipo task_struct, definida en in <linux/sched.h>.



5 - Signals

Signals

Signals o señales

Una señal (signal) es una notificación que envía a un proceso cuando un determinado evento ocurre.

A las señales se les dice a veces *interrupciones por software*.

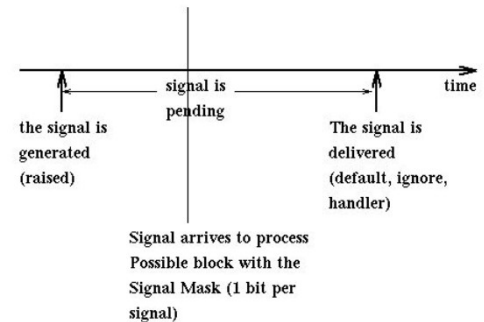
Son análogas a las interrupciones por hardware, interrumpen el flujo normal de la ejecución de un programa, pero en la mayoría de los casos no es posible predecir el arribo de una señal.

Son análogas a las interrupciones por hardware, interrumpen el flujo normal de la ejecución de un programa, pero en la mayoría de los casos no es posible predecir el arribo de una señal.

Un proceso puede enviar una señal a otro proceso.

Un proceso puede enviarse señales a él mismo.

Normalmente la fuente de envío de señales hacia un proceso es el Kernel.



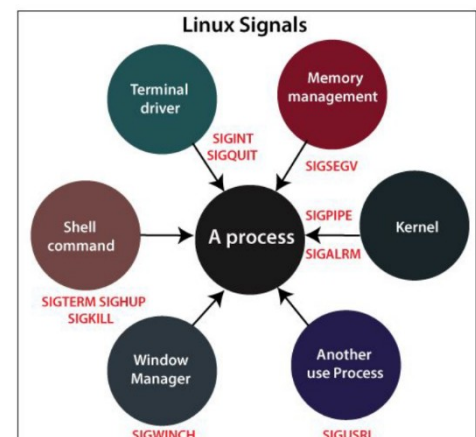
Motivos por los que el Kernel envía señales a un proceso:

- Una excepción de software (div por 0).
- El usuario presiona algún carácter especial en la terminal (CTRL-C).
- Un evento que tiene que ver con el software ocurrió, un proceso hijo terminó.

Cada señal está definida como un valor entero (corto) único en <signal.h> con un nombre simbólico que comienza con SIGxxxx.

Las señales se agrupan en dos categorías:

1. Tradicionales o estándares, cuyos valores van de 1 a 31.
2. De Tiempo Real.



Name	Signal Number	Default	Description	SUSv3
SIGHUP	1	Term	Cuelgue detectado en la terminal de control o muerte del proceso de control	•
SIGINT	2	Term	Interrupción procedente del teclado	•
SIGQUIT	3	Core	Terminación procedente del teclado	•
SIGILL	4	Core	Instrucción ilegal	•
SIGTRAP	5	Core	Trace/breakpoint trap	•
SIGABRT	6	Core	Señal de aborto procedente de abort(3)	•
SIGBUS	7 (SAMP=10)	Core	Memory Access error	•
SIGFPE	8	Core	Excepción de coma flotante	•
SIGUSR1	10 (SA=30, MP=16)	Term	Señal definida por usuario 1	•
SIGKILL	9	Term	Señal de matar	•

SIGSEGV	11	Core	Referencia inválida a memoria	•
SIGUSR2	12 (SA=31, MP=17)	Term	Señal definida por usuario 2	•
SIGPIPE	13	Term	Tubería rota: Escritura sin lectores	•
SIGALRM	14	Term	Señal de alarma de alarm(2)	•
SIGTERM	15	Term	Señal de terminación	•
SIGSTKFLT	16 (SAM=undef, P=36)	Term	Stack fault on coprocessor	
SIGCHLD	17 (SA=20, MP=18)	Ignore	Proceso hijo terminado o parado	•
SIGCONT	18 (SA=19, M=25, P=26)	Cont	Continuar si estaba parado	•
SIGSTOP	19 (SA=17, M=23, P=24)	Stop	Parar proceso	•
SIGTSTP	20 (SA=18, M=24, P=25)	Stop	Parada escrita en la tty	•
SIGTTIN	21 (M=26, P=27)	Stop	E. de la tty para un proc. de fondo	•
SIGTTOU	22 (M=27, P=28)	Stop	S. a la tty para un proc. de fondo	•
SIGURG	23 (SA=16, M=21, P=29)	Ignore	Urgent data on socket	•
SIGXCPU	24 (M=30, P=33)	Core	CPU time limit exceeded	•
SIGXFSZ	25 (M=31, P=34)	Core	File size limit exceeded	•
SIGVTALRM	26 (M=28, P=20)	Term	Virtual timer expired	•
SIGPROF	27 (M=29, P=21)	Term	Profiling timer expired	•
SIGWINCH	28 (M=20, P=23)	Ignore	Terminal window size change	
SIGIO /	29 (SA=23, MP=22)	Term	I/O possible	•
SIGPWR	30 (SA=29, MP=19)	Term	Power about to fail	
SIGSYS	31 (SAMP=12)	Core	Invalid system call	•
SIGEMT	undef (SAMP=7)	Term	Hardware fault	
SIGPOLL				

Ciclo de vida

Estados de una señal:

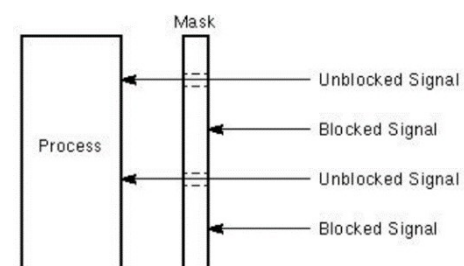
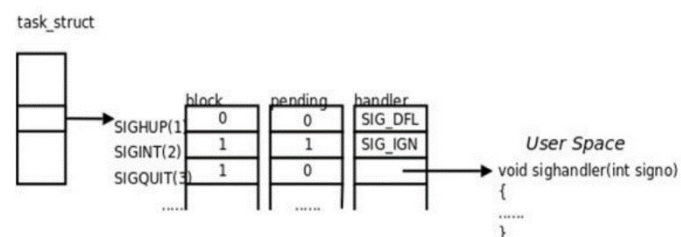
- ❖ Generada
- ❖ Entregada
- ❖ Pendiente

Una señal es **generada** por un evento, una vez generada, esta es **entregada** a un determinado proceso, que tomará en algún momento una acción.

El tiempo entre la generación y la entrega la señal está **pendiente**.

Una señal pendiente es entregada al proceso tan pronto como este esté planificado para ser el próximo en correr o inmediatamente si este se encuentra en ejecución.

A veces es necesario que un proceso no sea interrumpido por la entrega de una señal. Para ello se agrega una máscara de señal.



- **SIG_BLOCK** Adds the set of signals specified in the second argument to the process's signal mask
- **SIG_UNBLOCK** Subtracts the set of signals specified in the second argument from the process's signal mask
- **SIG_SETMASK** Replaces the process's signal mask with the set of signals specified in the third argument

Una vez que la señal se entrega, el proceso puede llevar a cabo una de las siguientes acciones por defecto:

1. La señal es **ignorada (ignored)**, es descartada por el kernel y no tiene efecto sobre el proceso.
2. El proceso es **terminado** (killed), muchas veces se denomina abnormal process termination.
3. Un archivo de **core dump** es generado y el proceso terminado. (imagen de la memoria virtual)
4. El proceso es **detenido** (stopped) - ejecución suspendida.
5. El proceso **continúa** (resumed) su ejecución.

Disposition

Un programa puede modificar la acción por defecto de una determinada señal cuando esta es entregada.

Esto se denomina definir la **disposition**:

1. Que ocurra la acción por defecto. ¿Cuándo es bueno esto?
2. La señal es ignorada. ¿Cuándo es bueno esto?.
3. Un manejador de la señal (signal handler) es ejecutado.

Signal Handler

Un signal handler es una función, escrita por el programador, que realiza la tarea apropiada en respuesta a la entrega de una señal.

Por Ejemplo, la Shell tiene un signal handler para la señal SIGINT (Ctrl-C). ¿Cuál es?

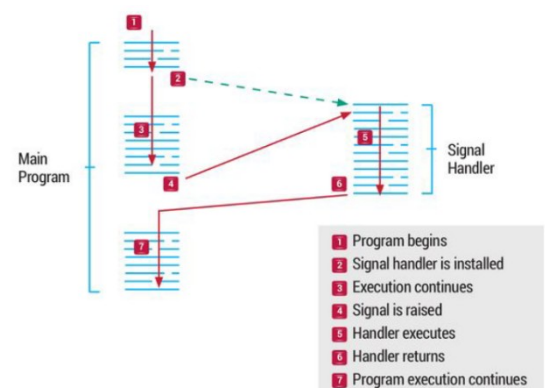
```
#include <signal.h>
```

```
void ( *signal(int sig, void (*handler) (int)) ) (int);
```

Returns previous signal disposition on success, or SIG_ERR on error

El primer argumento de signal() , identifica la señal a la cual se le quiere cambiar la disposition.

El segundo argumento handler es un puntero a una función que debe ser llamada cuando la señal se entrega.



signal() no tiene una implementación estándar.

El valor de retorno de Signal es un puntero a la antigua disposition

Envío de señales

kill()

A través de la system call kill() un proceso puede enviar señales a otro proceso.

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

Returns 0 on success, or -1 on error

raise()

La system call raise() permite a un proceso enviar señales a sí mismo..

```
#include <signal.h>
```

```
int raise(int sig);
```

Returns 0 on success, or nonzero on error

6 - Virtualización de memoria

La Abstracción del Espacio de Direcciones: Introducción

El tema fundamental sobre la administración de memoria se encuentre en la forma adecuada de representarla. La memoria física puede ser imaginada como un arreglo de direcciones de memoria una detrás de otra.

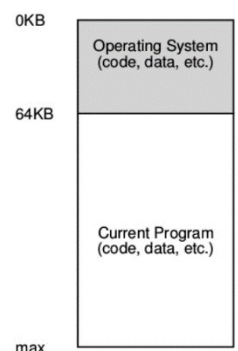


Esta “**abstracción**” es manejable mientras la cantidad de memoria necesitada está en un rango “manejable” y, ¿cuál es ese rango? En DOS (Disk Operating System) ese rango lo daba la elección de una arquitectura determinada, la del 8086, que permite tener direcciones de memoria de hasta 2^{20} bits, es decir 1 MegaByte de memoria.

Los primeros sistemas

En los primeros Sistemas Operativos, en “**the Early Days**”, en los cuales el mismo S.O. se desplegaba en solo 64 KiloBytes de memoria, el resto de la misma estaba disponible para un único proceso ejecutándose a partir de los 64Kb de memoria física.

- No había mucha ilusión que este pudiera crear en esta época.
- Los usuarios no esperaban mucho del sistema operativo.
- No había muchos usuarios de sistemas operativos.



El sistema operativo era más o menos **un conjunto de funciones o rutinas** (específicamente una biblioteca) que se alojaba en la memoria empezando en la dirección física 0, y después existía un único programa en ejecución (un proceso) que se encontraba en la memoria física de la computadora, esta memoria era el resto no utilizado por el sistema operativo.

Multiprogramación y Time Sharing

Después de un tiempo y debido a que las computadoras eran muy caras la gente empezó a compartir las computadoras de una manera más efectiva. La “**era de la multiprogramación**” había visto la luz del día (*saw the lighth of the day*).

En la cual más de un proceso estaba preparado para ser ejecutado en algún determinado momento, y el sistema operativo intercalaba dicha ejecución según la circunstancia. Haciendo esto se mejoró efectivamente el uso de la CPU, tal mejora en la eficiencia fue particularmente decisiva en esos días en la cual una computadora costaba cientos de miles o tal vez millones de dólares

En esta era, múltiples procesos están listos para ser ejecutados un determinado tiempo¹ según el S.O. lo decidiese en base a ciertas políticas de planificación o scheduling. De todas formas había una computadora para varias personas aún.

Rápidamente la demanda de más computadoras, es decir, una computadora para una persona, dio el impulso para que la “**era del time sharing**” naciese.

¹ Recordar el concepto de time slice.

Específicamente, muchos se dieron cuenta de las limitaciones de la computación batch, particularmente los programadores, los que estaban cansados de largos ciclos de programar - debuggear.

La noción de interacción empezó a tomar importancia, esta interacción es pensada teniendo en cuenta que varios usuarios deberían poder **concurrentemente** utilizar una misma computadora, y cada uno de estos estaría esperando un determinado tiempo para obtener cierta respuesta de las tareas una respuesta del proceso que estaba ejecutando en ese momento, era vital.

Una forma de implementar el time sharing es aquella de:

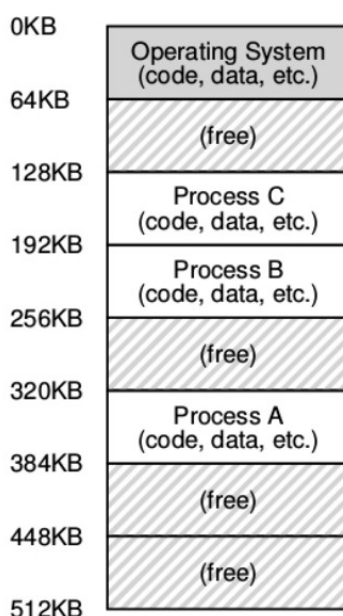
1. Un proceso por un determinado time slice o quantum de tiempo al cual se le da acceso a toda la memoria y los recursos.
2. Hasta que éste debe detener su ejecución, grabar su estado²: en algún lugar como por ejemplo el disco.
3. Cargar otro proceso.
4. Ejecutarlo por un rato, y así sucesivamente.

De esta forma se implementa una especie de time sharing rudimentario...

Desafortunadamente este método tiene un gran problema, es **demasiado lento** particularmente cuando la memoria de la computadora comienza a crecer.

Teniendo en cuenta que hacer un cambio de contexto a nivel registros es relativamente rápido hacer lo mismo con el contenido de la memoria hacia el disco es muy poco eficiente, por eso, es más fácil mantener los procesos en memoria, mientras se realizan los cambios de quien se está ejecutando, de esa forma se permite implementar eficientemente time sharing.

La idea de los sistemas operativos **time sharing** apuntó a darle a cada proceso una pequeña parte de la memoria física:



A medida que el time sharing se empezó a hacer cada vez más popular, uno se podría preguntar ¿cuáles fueron las siguientes demandas sobre los sistemas operativos? La respuesta a esta pregunta se basa en la **protección**. Varios procesos ejecutándose en memoria podrían interactuar entre ellos y... ¿Por qué sería eso un problema?

² y toda su memoria física

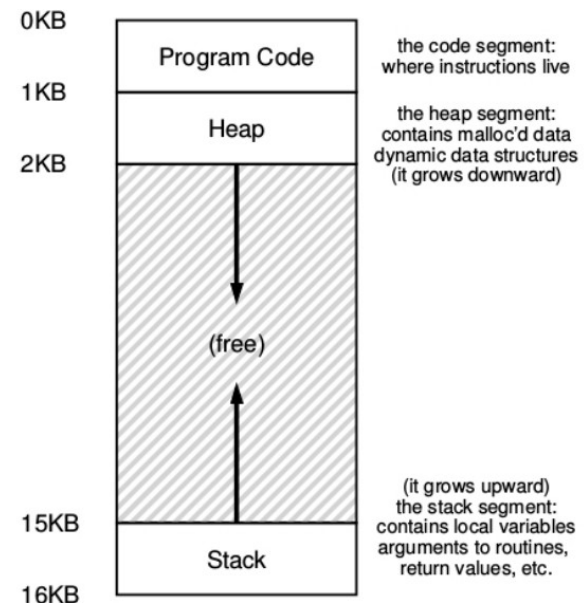
El Espacio de Direcciones o Address Space

Crear un mecanismo que permita que la memoria física de una computadora sea utilizada de forma fácil y eficiente llevo paulatinamente a concebir el concepto de espacio de direcciones, la abstracción para la memoria.

El **Address Space** de un proceso contiene **todo** el estado de la memoria de un programa en ejecución.

Por ejemplo:

- El código del programa tiene que estar alojado en la memoria en algún lugar.
- El programa mientras se está ejecutando usa el stack para mantener registro de donde se encuentra en la cadena de llamadas a funciones o procedimientos para reservar espacios para las variables locales, para pasar parámetros y a su vez devolver valores de y hacia una rutina.
- Finalmente se utiliza el heap para reservar memoria de forma dinámica.
- Existen por supuesto otras cosas en la memoria: Variables estáticas, constantes, etc.



En el ejemplo, se presenta un pequeño espacio de direcciones de 16kb.

El **código fuente** del programa vive en lo **alto del espacio de direcciones** empezando de 0 en este ejemplo y está empaquetado en la primer 1k del espacio de direcciones. El **código fuente es estático** por ende se puede poner al principio del espacio de direcciones y **no necesitará más espacio** mientras que el programa se ejecute.

Por otro lado hay dos regiones del espacio de direcciones que pueden **crecer o achicarse** mientras el programa se esta ejecutando. Como ya se sabe son el **Heap** y el **Stack**, debido a que ambas en algún momento van a querer crecer siempre se ponen en los **extremos** del espacio de direcciones **enfrentadas** entre sí.

De esta forma se permite tal crecimiento solamente que el mismo se dirige a **direcciones opuestas**:

- El heap empieza justo después del código fuente y crece hacia abajo.
- El stack empieza al final del espacio de direcciones y crece hacia arriba.

Por supuesto que cuando se describe de esta forma el **espacio de direcciones**, lo que se está describiendo es la **abstracción** que el sistema operativo provee para ejecutar un programa.

En realidad el **programa** no se encuentra en el **rango** de las **direcciones de memoria física** entre 0 y 16k.

Cuando el sistema operativo realiza esto, se dice que el sistema operativo está **virtualizando memoria** porque el programa que se está ejecutado piensa que está cargado en una dirección de memoria particular, en este caso la 0 y tiene potencialmente un espacio de direcciones muy grande; la realidad es bastante diferente.

Cuando por ejemplo un proceso trata de cargar el contenido de la dirección 0, que a partir de ahora llamaremos la **dirección virtual 0**, de alguna forma el **sistema operativo** con ayuda de algún tipo de **mecanismo de hardware** que tendrá que asegurarse que no se cargue la

dirección física 0 real más bien que se cargue la **dirección física** en la cual el espacio de direcciones de ese proceso se encuentra alojado.

El **Espacio de Direcciones o Address Space** es la abstracción fundamental sobre la memoria de una computadora. Consiste en dar un mecanismo fácil de usar a los usuarios de la computadora.

Cuando se describe el **espacio de direcciones** se está describiendo la **abstracción** que el **Sistema Operativo** le proporciona al **programa en ejecución** sobre la memoria de la computadora.

Cuando el Sistema Operativo implementa esta abstracción, se dice que el O.S. está **Virtualizando la Memoria** ya que el programa en ejecución cree que está cargado en un lugar particular de la memoria (la posición 0 **dirección virtual o virtual address**) y tiene potencialmente toda la memoria para él³.

Virtualización de Memoria

El **sistema operativo tiene que virtualizar memoria...** pero lo tiene que hacer **con estilo**, para ello una de las **metas** principales de la **virtualización es la transparencia**.

El sistema operativo debería implementar la virtualización de memoria de forma tal que sea **invisible** al programa que se está ejecutando; es más el programa debe comportarse como si el estuviera alojado en su propia área de memoria física privada.

Pero detrás de escena, el sistema operativo y el hardware hacen todo el trabajo para multiplexar memoria a lo largo de los diferentes procesos y por ende implementa la ilusión.

Otra meta importante de la virtualización es la **eficiencia**. El sistema operativo debe esforzarse para hacer que la virtualización sea lo más **eficiente** posible en términos de **tiempo** (no hacer que los programas corran más lentos) **y espacio** (no usar demasiada memoria para las estructuras necesarias para soportar la virtualización).

Por último, una tercera meta de la virtualización de memoria es la **protección**. El sistema operativo tiene que asegurarse de **proteger a los procesos unos de otros como también proteger al sistema operativo de los procesos**. Cuando un proceso realiza un load, un store, o un fetch de una instrucción este no tiene que ser capaz de hacerlo o afectar de ninguna forma al contenido de la memoria del proceso o del sistema operativo.

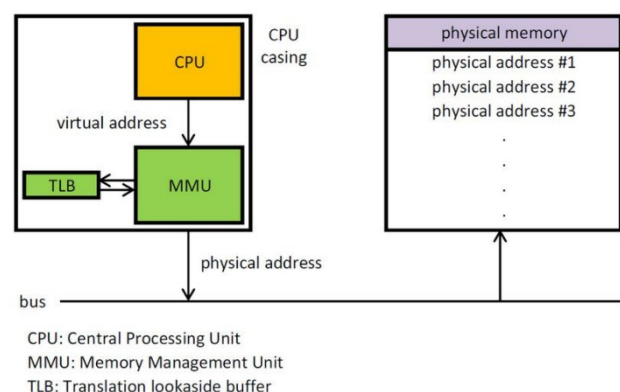
La protección por ende habilita una propiedad llamada aislamiento entre proceso; cada proceso tiene que ejecutarse en su propio caparazón aislado y seguro de los avatares de otros procesos con fallas o incluso maliciosos.

Address Translation

Existen dos puntos importantes a la hora de virtualizar memoria:

1. la flexibilidad
2. la eficiencia

Para llegar a ellos un buen mecanismo de virtualización de memoria debe ser lo más flexible y eficiente posible. ¿Cómo se logra esto? Mediante la utilización de una técnica llamada **Hardware-Based Address**



³ Se sabe que esto no es cierto

Translation o como más comúnmente se conoce **Address Translation**.

Con esta técnica el hardware **transforma** cada acceso a memoria, transformando la **virtual address** que es provista desde dentro del espacio de direcciones en una **physical address** en la cual la información deseada se encuentra realmente almacenada.

Entonces, en todos y por cada una de las referencias a memoria un **address translation** es realizada por el **hardware** para redireccionar las referencias a la **memoria de la aplicación** hacia las posiciones reales en la **memoria física**.

Es evidente, que el **hardware por sí solo** no puede virtualizar la memoria este **solo provee un mecanismo de bajo nivel** para poder hacerlo eficientemente.

El sistema operativo tiene que involucrarse en los puntos claves para:

- setear al hardware de forma correcta para que esta traducción se de lugar;
- por eso debe entonces gerenciar la memoria, manteniendo información de en qué lugar hay áreas libres y en qué lugar hay áreas en uso;
- y por último intervenir de forma criteriosa como mantener el control sobre toda la memoria usada.

Otra vez, todo esto se trata de crear una bella **ilusión**:

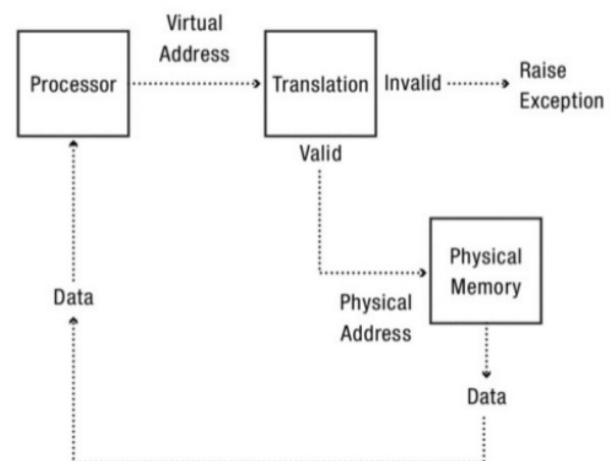
- o que el programa tenga su propia memoria privada donde reside su propio código y sus propios datos.
- o Detrás de esa realidad virtual existe o yace una fea verdad física que muchos programas se encuentran realmente compartiendo memoria a mismo tiempo que la CPU los cambia de estado ejecutando un único programa y el siguiente.

Address translation es el término por el cual se conoce al mecanismo, proporcionado por el hardware, que permite a partir de una dirección virtual obtener la dirección física correspondiente. Esa traducción es realizada por el hardware. El S.O. está en el medio de ese proceso y determina si el mismo se ha realizado correctamente. Lo que hace es gerenciar el manejo de la memoria:

1. Manteniendo registro de que parte está libre.
2. Que parte está en uso.
3. Manteniendo el control de la forma en la cual la memoria está siendo utilizada.

Desde la perspectiva del programador el proceso de address translation es completamente transparente, si bien, el hecho es que en realidad la memoria es almacenada en un lugar completamente diferente de donde él cree que está siendo almacenada.

Si el proceso de address translation se considera como una **caja negra**, sería una función.



Formalización

Formalmente, el proceso de address translation es un *mapeo* entre los elementos de un **espacio de direcciones virtuales de N-elementos (VAS)** y un **espacio de direcciones físicas de M-elementos (PAS)**:

$$MAP: VAS \Rightarrow PAS \cup \emptyset$$

donde

$$MAP(A) = \begin{cases} A' & \text{si los datos en la dir. virtual } A \text{ están presentes en la dir. física de } A' \text{ de PAS} \\ \emptyset & \text{si los datos en la dir. virtual } A \text{ no están presentes en la memoria física} \end{cases}$$

Dynamic Reallocation o Memoria Segmentada [DAH]

Base and Bound

Para ir ganando comprensión sobre el **hardware-based address translation**, se verá su primera implementación, introducida en las primeras máquinas que utilizaban time-sharing hacia el fin de los años 50, es una idea simple llamada **base y segmento** también puede ser vista como **dynamic reallocation**.

Específicamente solo se necesitan dos registros de hardware dentro de cada CPU:

1. **Registro base**
2. **Registro límite o Segmento.**

Este par **base-límite** va a permitir que el address space pueda ser ubicado en cualquier lugar deseado de la memoria física, y se hará mientras el sistema operativo se asegura que el proceso solo puede acceder a su address space.

En esta configuración, cada programa es escrito y compilado como si fuera cargado en la dirección física 0. A su vez cuando se inicia la ejecución del programa el OS decide en qué lugar va a cargar el mismo y para hacerlo setea el registro base en un determinado valor. Algunas cosas interesantes pasan cuando el proceso se está ejecutando. A partir de ahora cuando cualquier referencia es generada por el proceso es traducida por el procesador de la siguiente manera:

$$\text{physical address} = \text{virtual address} + \text{base}$$

Cada referencia de memoria generada por el procesador es una **dirección virtual**; el hardware cada vez que se hace referencia a esta dirección tiene que sumar el contenido del registro base y su resultado es la **dirección física** que tiene que ser utilizada en la memoria del sistema.

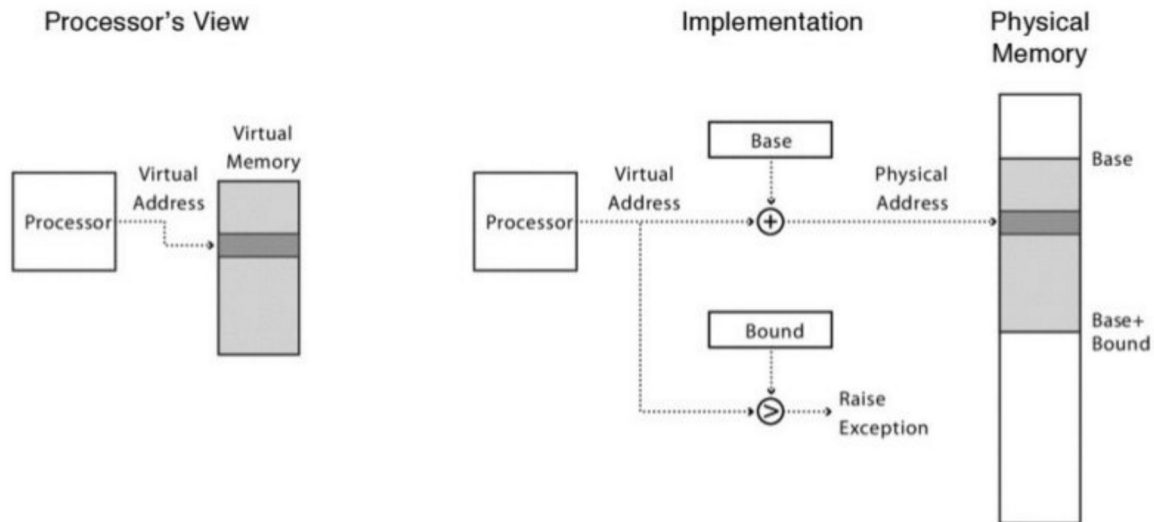
El proceso que transforma la **virtual address en la physical address** es exactamente la técnica conocida como **address translation**. Esto es el hardware toma una **dirección virtual** a la cual el proceso cree que está referenciando y la transforma en una **dirección física** que es el lugar en el cual realmente los datos están ubicados. Debido a que esta ubicación sucede en run-time y debido a que se puede mover el **address space** incluso una vez que el proceso empezó a ejecutarse la técnica habitualmente es referenciada como **dynamic reallocation** o **realocación dinámica**.

Ahora la pregunta es: ¿qué pasa con los bound registers o registros de segmento?

El **registro bound** está allí para ayudar con la protección. Específicamente, el procesador antes que hacer nada va a chequear que la **referencia de memoria** esté dentro de los **límites** del **address space** para asegurarse que la misma sea legal.

Si un proceso genera una **dirección virtual** que es mayor que los límites o una dirección que es negativa la CPU va a generar una excepción y el proceso va a terminarse.

El punto es que los límites están ahí para asegurarse que todas las direcciones generadas por el proceso sean legales y estén dentro del límite del mismo.



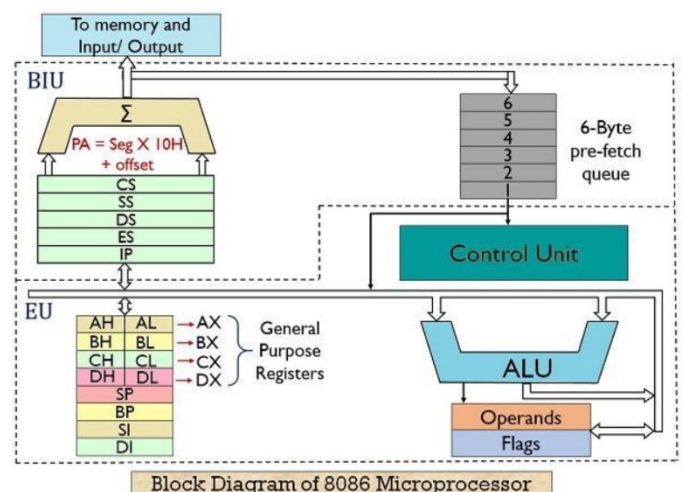
Se habrá notado que el registro base y el límite son estructuras de hardware mantenidas dentro del mismo procesador o chip. Antiguamente a esa parte del procesador que ayudaba a la address translation se la llamaba **memory management unit (MMU)**. A medida que se fueron sofisticando las técnicas de gerenciamiento de memoria se fue agregando más circuitería a lo que se llamaba MMU.

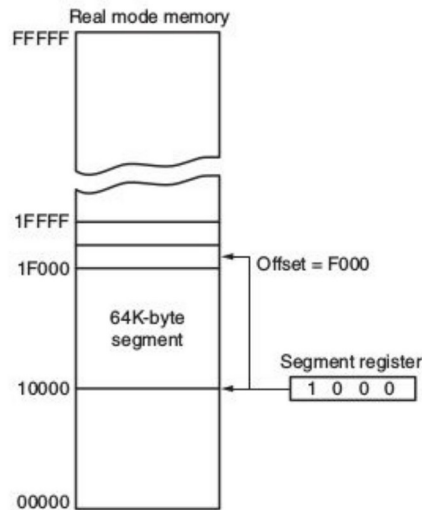
Un pequeño detalle sobre el bound register, puede ser definidos en dos formas diferentes:

- 1°) la primera: este registro mantiene el tamaño del address space entonces el hardware chequea la dirección virtual contra el bound register, sumándole primero el registro base.
- 2°) la segunda es mediante el bound register almacena la dirección física del fin del espacio de direcciones.

En x86

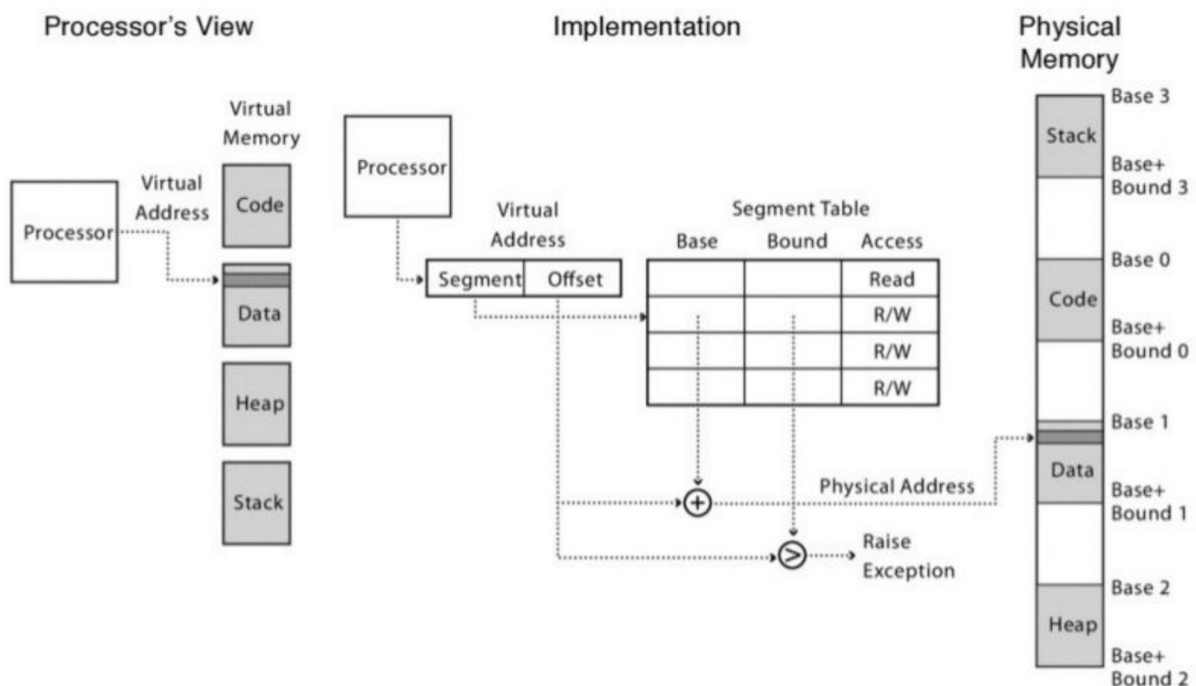
- AX, BX, CX, DX: Registros Generales
- CS, DS, SS, ES: Registros de Segmentos, manejan la traducción en modo real. (Code Segment, Data Segment, Extra Data Segment)
- SI, DI, BP, SP, IP: Registros de Punteros y Registro de Índices. (Base Pointer, Stack Pointer and Instruction Pointer)
- **CS:IP** localiza la próxima instrucción a ser ejecutada en modo real.
- **SS:SP** localiza la dirección del puntero al stack, a veces también puede ser SS:BP.
- **DS: BX,DI,SI** localizan el puntero a una dirección de memoria dentro del data address.
- **ES:DI** puntero al extra data address donde van los strings.





Address Translation con Tabla de segmentos

El problema de la técnica anterior es que se tiene un solo registro base y solo un segmento. La mejora a este método es mediante la aplicación de un pequeño cambio: en vez de tener un solo registro límite, se tiene un arreglo de pares de (registro base, segmento) por cada proceso:



cada entrada en el arreglo controla una porción del virtual address space. La memoria física de cada segmento es almacenada continuamente, pero distintos segmentos pueden estar ubicados en distintas partes de la memoria física.

Una dirección virtual tiene dos componentes:

1. un número de segmento
2. un offset de segmento

El número de segmento es el índice de la tabla para ubicar el inicio del segmento en la memoria física. El registro bound es chequeado contra la suma del registro base+offset para prevenir que el proceso lea o escriba fuera de su región de memoria.

En una dirección virtual utilizando esta técnica, los bits de más alto orden son utilizados como índice en la tabla de segmentos. El resto se toma como offset y es sumado al registro base y comparado contra el registro bound. El número de segmentos depende de la cantidad de bits que se utilizan como índice.

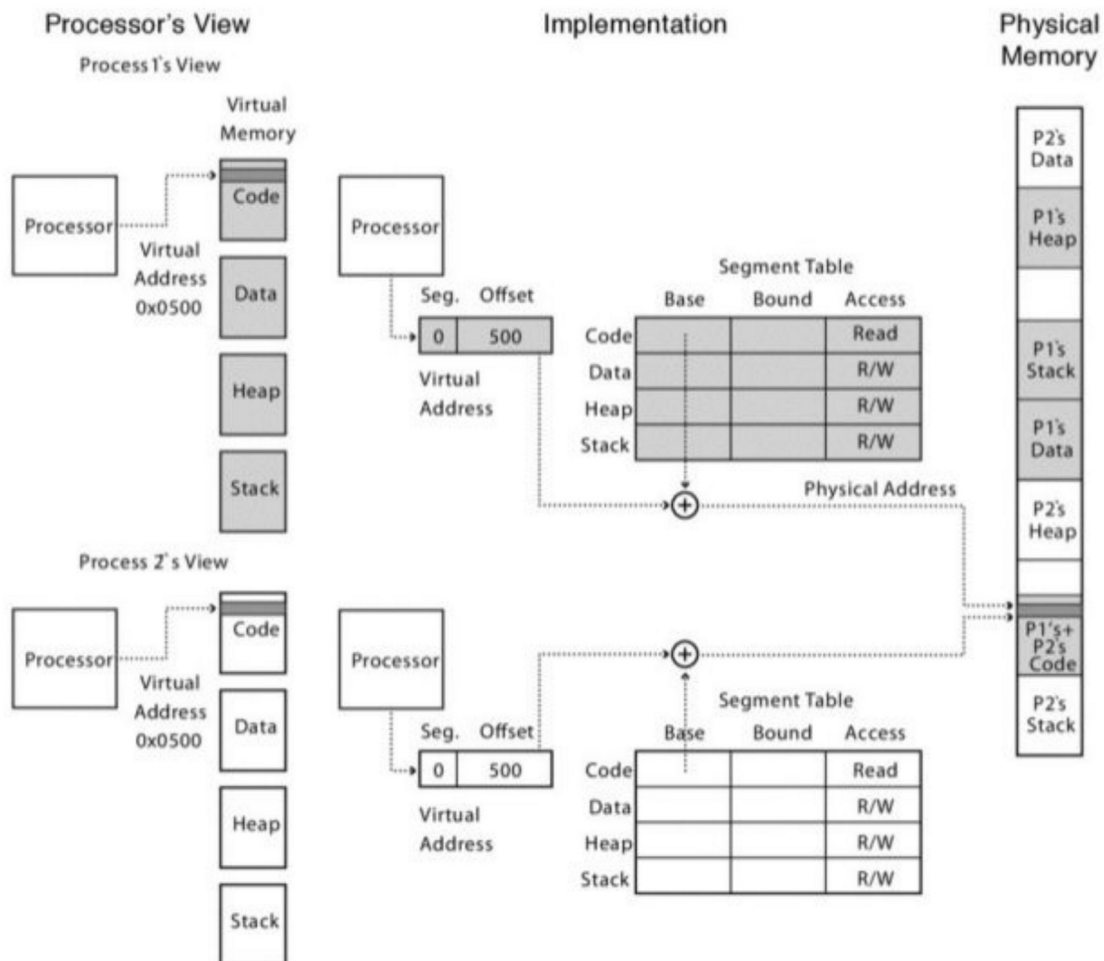
El error de **Segmentation Fault** era un error que se daba cuando, en las máquinas que implementaban segmentación, se quería direccionar una posición fuera del espacio direccionable. Increíblemente este error aún se utiliza en sistemas operativos que no usan segmentación.

Algunas notas:

1. El stack crece hacia atrás o backward, por ende el hardware necesita saber sobre eso en un bit de información
2. Compartir: a veces es necesario poder compartir ciertos segmentos entre varios procesos, para ello el hardware necesita proporcionar un mecanismo , otro bit que permita saber si se puede ejecutar, escribir o leer por varios procesos a la vez. Para ello se usan un par de bits llamados protection bits.
3. Todo lo hace el hardware en este caso la MMU (Memory Mangement Unit).
4. Segmentación de grano fino vs segmentación de grano grueso. La primera consiste en tener muchos segmentos pequeños y la segunda consiste en tener pocos segmentos grandes.

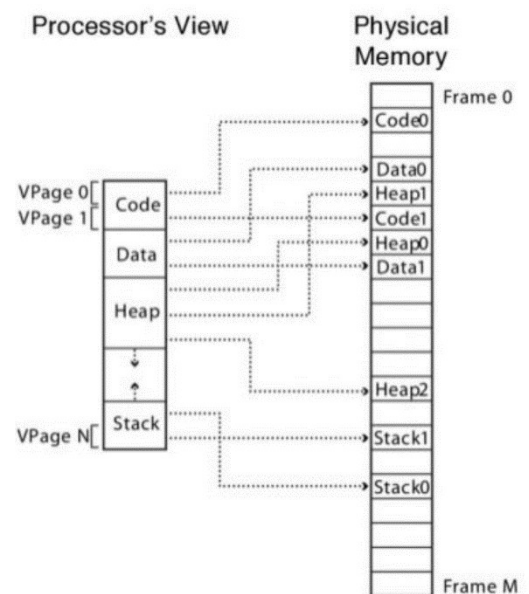
El problema de la segmentación es la fragmentación, precisamente la fragmentación externa.

Ejemplo de dos procesos que comparten el code segment, pero con stack y data segment separados:



Memoria Paginada

Una alternativa a la **memoria segmentada** es la **memoria paginada**. Con la **paginación**, la memoria es **reservada en pedazos de tamaño fijo** llamados **page frames**. El address translation es similar a como se trabaja con la segmentación. En vez de tener una página de segmentos cuyas entradas contienen punteros a segmentos, hay una tabla de páginas por cada proceso cuyas entradas contienen punteros a las page frames. Teniendo en cuenta que los page frames tienen un tamaño fijo, y son potencia de 2, las entradas en la page table sólo tienen que proveer los bit superiores de la dirección de la page frame. De esta forma van a ser más compactos. No es necesario tener un límite; la página entera se reserva como una unidad.



Address translation con Page Table

La virtual address tiene dos componentes:

1. El número de página virtual.
2. El offset dentro de esa página.

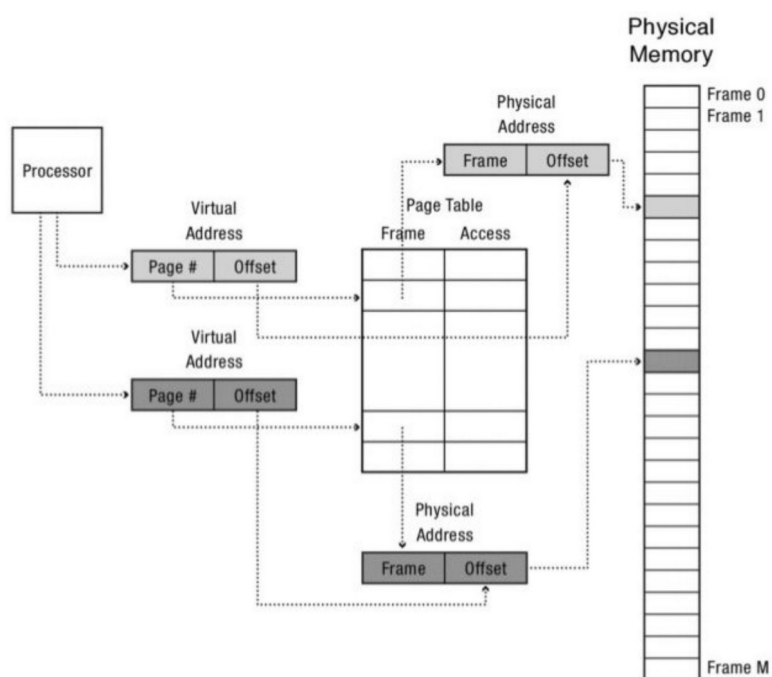
El número de la página virtual es el índice en la page table para obtener el page frame en la memoria física. La dirección física está compuesta por la dirección física del **Frame Page** que

se obtiene de la page table concatenado con el offset de la página que se obtiene de la virtual address. El sistema operativo maneja los accesos a la memoria.

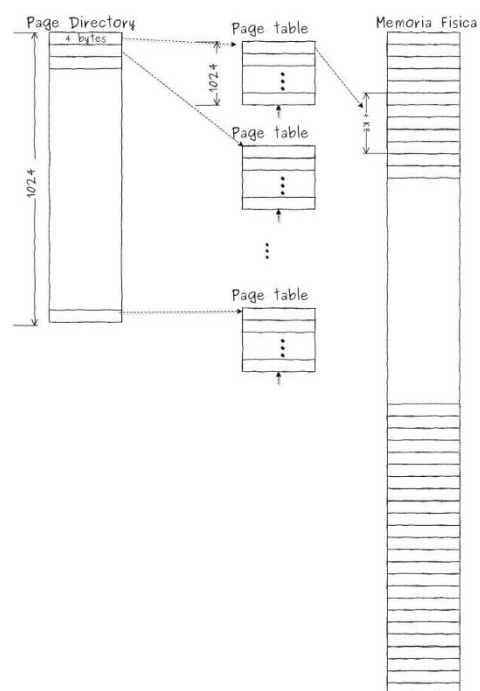
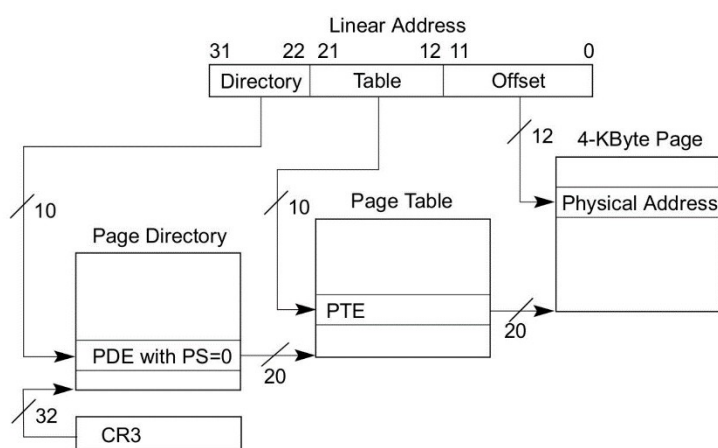
Una de las cosas que pueden parecer raras sobre la paginación es que si bien el programa cree que su memoria es lineal, de hecho, su memoria está desparramada por toda la memoria física como si fuera un mosaico.

El procesador va a ejecutar una instrucción después de otra usando direcciones virtuales y esas direcciones virtuales son lineales. Si bien la instrucción ubicada en el final de una página podría estar ubicada en una región completamente diferente de la memoria física de la próxima instrucción que debe empezar en la siguiente página.

Las estructuras de datos que también son contiguas usando direcciones virtuales, en la realidad, una matriz muy grande podría estar desparramada por distintos page frames físicos. La paginación encuentra el mismo problema que la segmentación, saber qué espacio de la memoria está vacío es muy complicado.



Memoria paginada x86

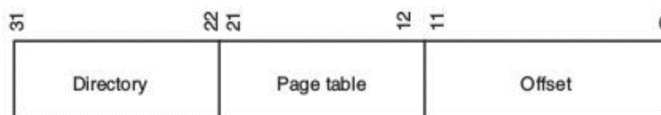


Memoria paginada x86: Virtual Address

Page Directory bits 31-22 (10 bits)

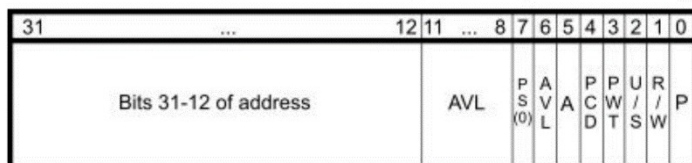
Page Table Entry bits 21-12 (10 bits)

Memory Page offset address bits 11-0 (12 bits)



Memoria paginada x86: Page Directory Entry

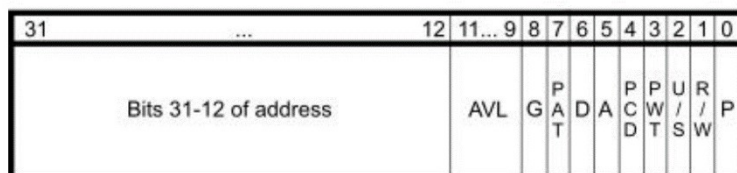
Page Directory Entry



P: Present	D: Dirty
R/W: Read/Write	PS: Page Size
U/S: User/Supervisor	G: Global
PWT: Write-Through	AVL: Available
PCD: Cache Disable	PAT: Page Attribute Table
A: Accessed	

Memoria paginada x86: Page Table Entry

Page Table Entry

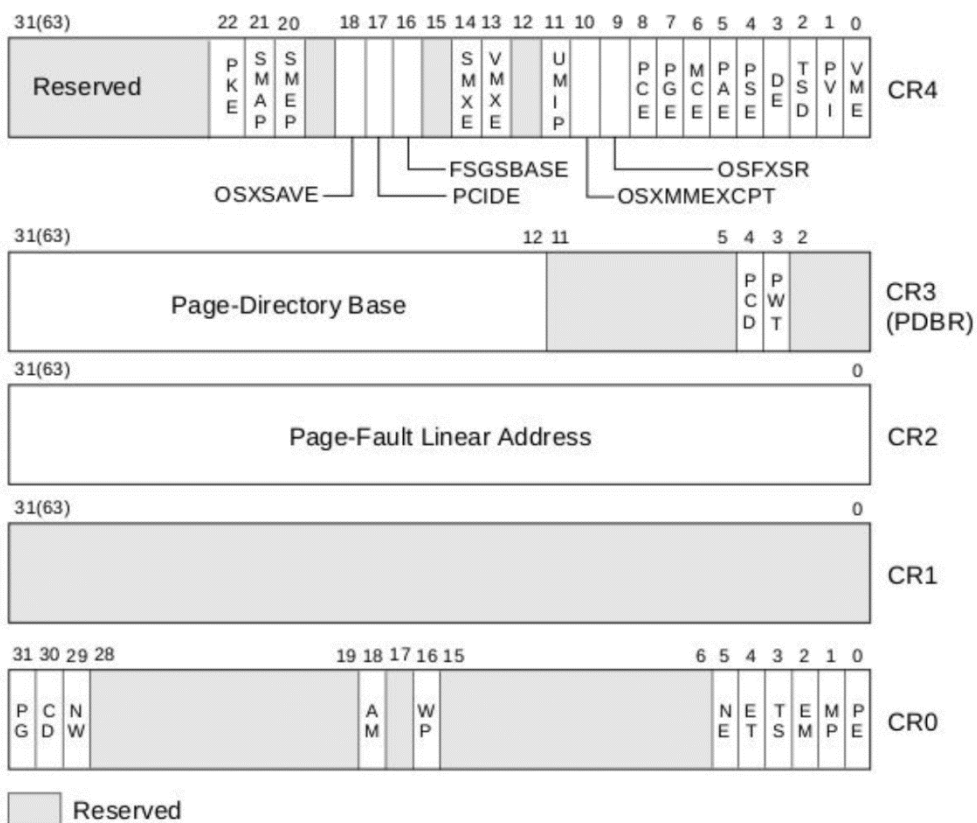


P: Present	D: Dirty
R/W: Read/Write	G: Global
U/S: User/Supervisor	AVL: Available
PWT: Write-Through	PAT: Page Attribute Table
PCD: Cache Disable	
A: Accessed	

Estos registros se encuentran a partir del microprocesador Intel 80386 hasta Core2. A partir del Pentium4 un registro llamado CR4 extiende la arquitectura x86. Los registros importantes para la paginación son: CR0 y CR3.

El bit más a la izquierda de el registro CR0 si está en 0 determina que la lineal address se convierte directamente en physical address para acceder a la memoria. Si PG está en 1 la lineal address debe ser convertida en physical address a través del mecanismo de paginación.

CR3 contiene page directory base que contiene 1024 entradas de 4 bytes cada una, cada entrada en el page directory ocupa 4 bytes y direcciona a una page table que contiene 1024 entradas.



Translation multinivel

Si se desea implementar un mecanismo eficiente de paginación, posiblemente elegir un arreglo no sea la solución más correcta. Un árbol o un hash table son más apropiados ya que son usados en sistemas más modernos.

Muchos sistemas utilizan técnicas de address translation basados en árboles, si bien los detalles varían de sistema en sistema y la terminología también es un poco confusa, sin embargo estos sistemas que se describirán a continuación tienen propiedades bastantes similares. Estas técnicas soportan:

- Protección de memoria de grano fino
- Memoria Compartida
- Ubicación de memoria flexible
- Reserva eficiente de memoria
- Un sistema de búsqueda de espacio de direcciones eficiente

Si bien todos los sistemas multinivel de address translation usan paginación en el nivel más bajo del árbol; la principal diferencia entre ellos es como se llega a la page table a nivel de las hojas del árbol, ya sea usando segmentación mas paginación, múltiples niveles de paginación, segmentación mas múltiples niveles de paginación.

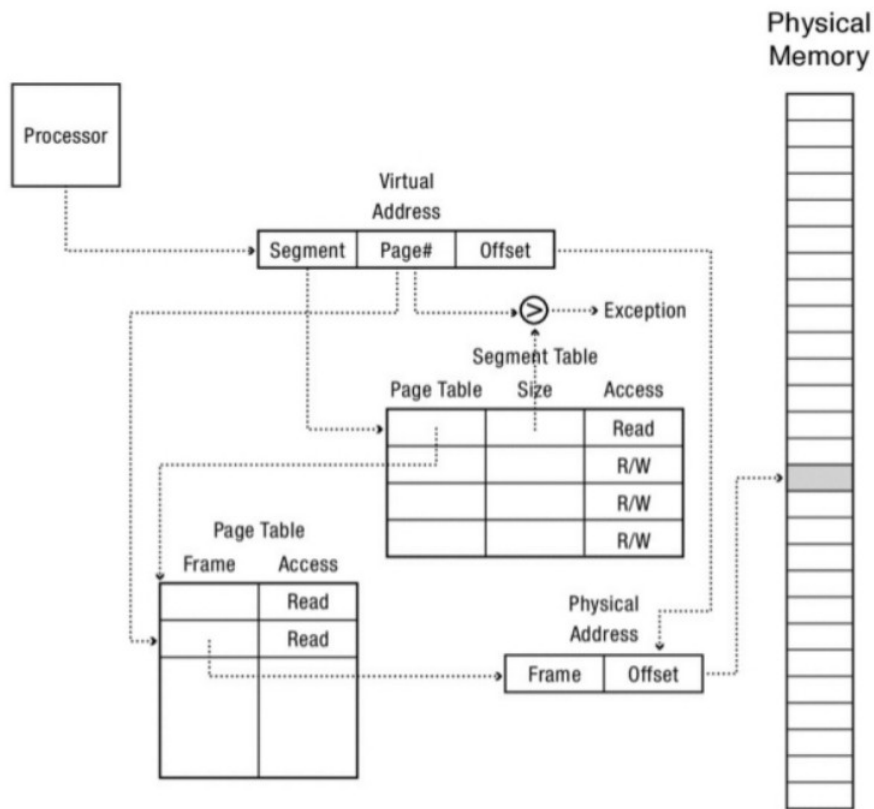
Paged Segmentation (Segmentación paginada)

Este sistema utiliza dos niveles de un árbol. Con paged segmentation, la memoria esta segmentada, pero en vez de que cada entrada en la página de segmentos apunte directamente a una región continua de la memoria física, cada entrada en la tabla de segmentos apunta a una tabla de páginas, que a su vez apunta a la memoria correspondiente a ese segmento. La tabla de segmentos tiene una entrada llamada limite o tamaño que describe la longitud de la página de tablas, o sea, la longitud de los segmentos en las páginas. El proceso de traducción de virtual address a physycal address es el siguiente.

La virtual address tiene 3 componentes:

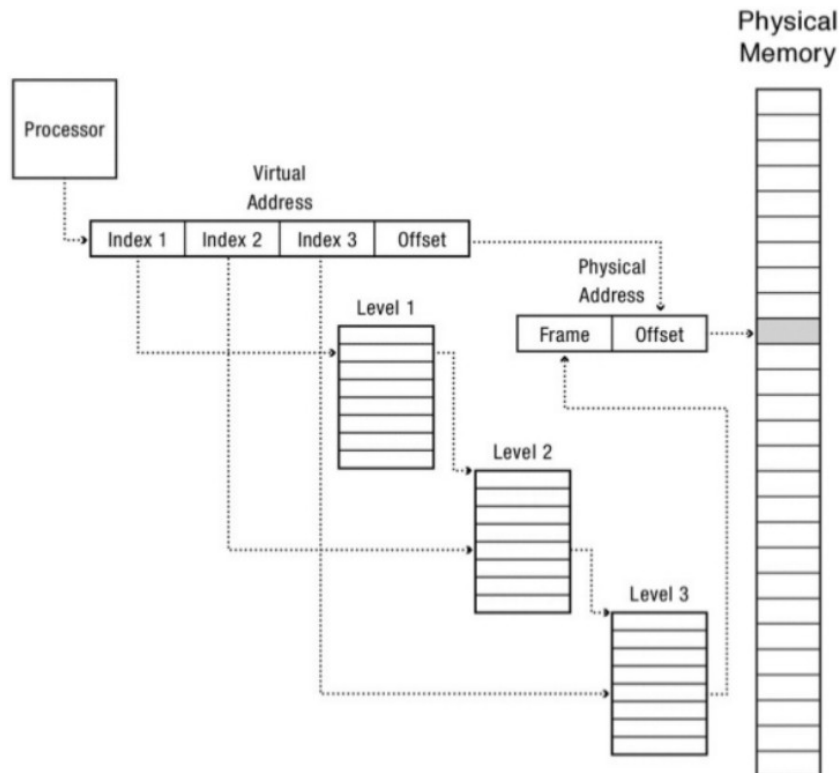
1. un numero de segmento
2. un numero de página virtual dentro de ese segmento
3. el offset dentro de la página.

El número de segmento es el índice para la segment table que aloja la page table para se segmento. La page number de esa virtual address es el índice de la page table (que pertenece a un segmento de la segment table) que contiene una page frame en la memoria física. La physical address está compuesta por el physical page frame que pertenece a la page table concadenado con el offset de la virtual address.

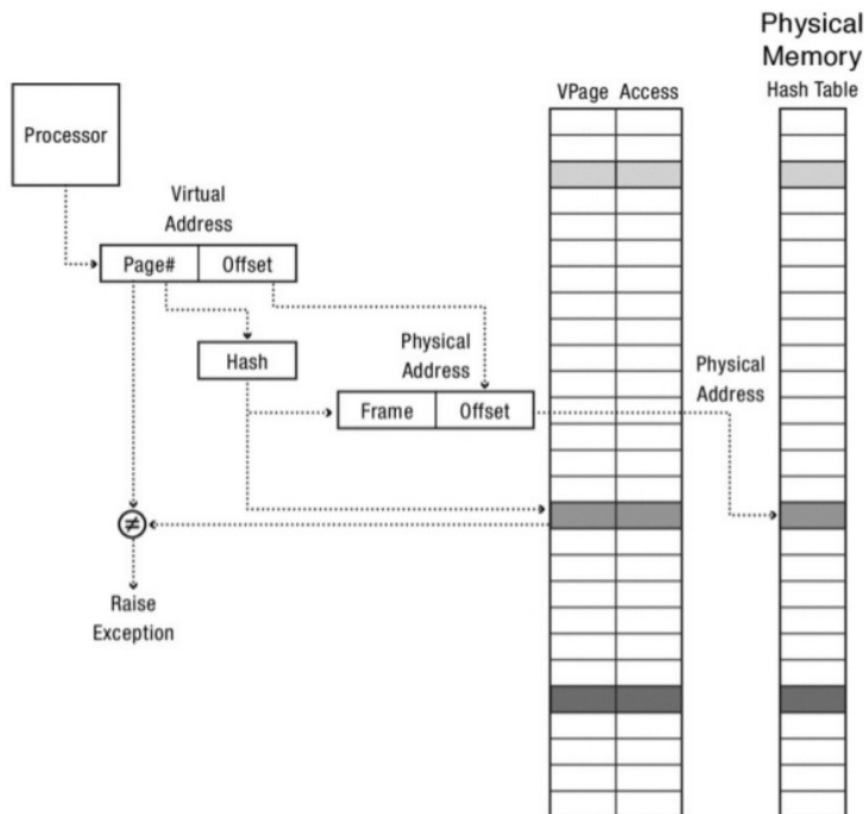


Address Translation con Tres Niveles de Page Tables

Este es un ejemplo concreto del sistema conocido como multi-level paging es similar a la paged segmentation, pero implementa multiples niveles de page tables. El procesador de sun microsystem sparc usa un sistema de 3 niveles de tablas de páginas.



Address Translation con Tabla de Hash por Software



Multilevel page segmentation

Este es el sistema utilizado por las arquitecturas x86 para 64 y 32 bit.

En la arquitectura x86 cada proceso posee una **Global Descriptor Table (GDT)**, que es equivalente a la segment table. La GDT es almacenada en la memoria; cada entrada a esta tabla apunta a una tabla de paginas (multinivel) para ese segmento. Para inicializar el proceso de address translation el sistema operativo setea la GDT e inicializa un registro llamado gdtr GDTR que contiene la dirección y la longitud de la GDT.

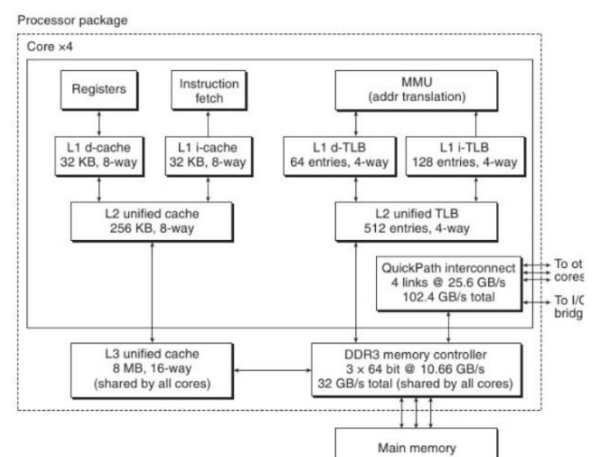
Por cada 32 bit en la arquitectura x86 la virtual address posee un segmento a una tabla de 2 niveles:

- Los primeros 10 bit de la virtual address son el índice de la paged table de primer nivel, llamada **page directory**,
- los otros 10 bit son el índice de una page table de segundo nivel
- y finalmente los 12 bit restantes son el offset dentro de la página.

Cada entrada en la page table ocupan 4 bytes y existen 1024 entradas por ende el tamaño total de la page directory es de 4k, sucede lo mismo con la page table de segundo nivel y justo coincide con el tamaño de una página en la memoria física.

Hacia una eficiente Address Translation

Hasta aquí ya uno puede estar un poco cansado de todos los mecanismos de hardware para realizar la traducción de direcciones de memoria. El tema es que muchos de estos métodos tienen varios niveles, hasta 4 en algunos casos, para alcanzar una dirección física, entonces eso lo hace realmente poco práctico para el procesador. A



continuación se mostrarán mecanismos para mejorar el rendimiento de la traducción de las direcciones.

Para ello se usará un caché (o escondrijo), que consiste en una copia de ciertos datos que pueden ser accedidos más de una vez más rápidamente. El concepto de **Cache** es ampliamente utilizado en muchas ramas de las ciencias de la computación: arquitectura de computadoras, sistemas operativos, sistemas distribuidos.

Hacia una eficiente Address Translation: TLB

Uno de los problemas del **address translation** reside en la **velocidad de la traducción** para ello se utilizan técnicas que mejoran la velocidad de esta traducción. Para mejorar el address translation se utiliza un mecanismo de hardware llamado **Translation-Lookaside Buffer**; o también conocido como **TLB**. La TLB es parte de la MMU y es simplemente un mecanismo de cache de las traducciones más utilizadas entre los pares virtual to physical address. Por ende un mejor nombre para este mecanismo podría ser address translation cache.

Por cada referencia a la memoria virtual, el hardware primero chequea la TLB para ver si esa traducción esta guardada ahí; si es así la traducción se hace rápidamente sin tener que consultar a la page table (la cual tiene todas las traducciones).

Este mecanismo tiene un tremendo impacto en la velocidad de la traducción.

Translation-Lookaside buffer

Si uno se pone a pensar la forma en que se programa, ésta tiene dos características: es secuencial y localizada:

```
...  
mov r1,r2  
mult r1,r2  
...
```

El hardware tiene que hacer el fetch de la instrucción add, después pasear por todos los multiniveles de las tablas de traducción para encontrar la dirección física de la instrucción add; ejecutar la instrucción , incrementar el contador de programa y volver a hacer todo esto otra vez para la próxima instrucción y además para sus datos pero esto es muy ineficiente.

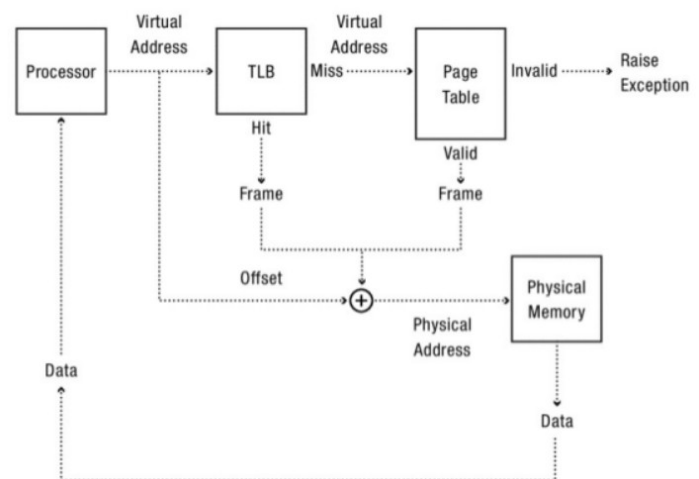
La **Translation Lookaside Buffer (TLB)** es una pequeña tabla a nivel hardware que contiene los resultados de la recientes traducciones de memorias realizadas. Cada entrada de la tabla mapea una **virtual page a** una **physical page**

```
TLB entry = {  
    virtual page number,  
    physical page frame number,  
    access permissions  
}
```

Normalmente se chequean todas las entradas de la TLB contra la virtual page, si existe matcheo el procesador utiliza ese matcheo para formar la physical address, ahorrándose todos los pasos de la traducción.

Esto se llama un **TLB hit**

Cuando del proceso anterior no existe matcheo en la TLB, se dice que se tiene un **TLB miss**

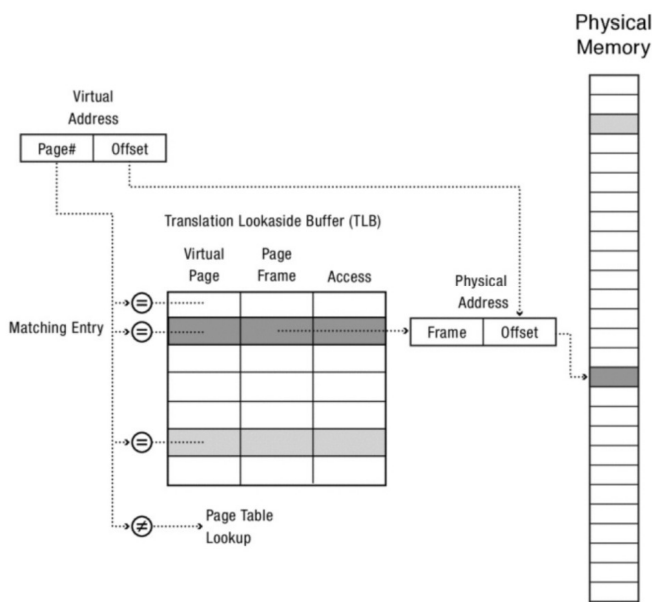


Pareciera que este proceso es más costoso, pero a nivel procesador tiene una ganancia muy buena en prestaciones ya que a nivel hardware está implementado en **memoria estática on chip** que está localizada muy cerca del procesador.

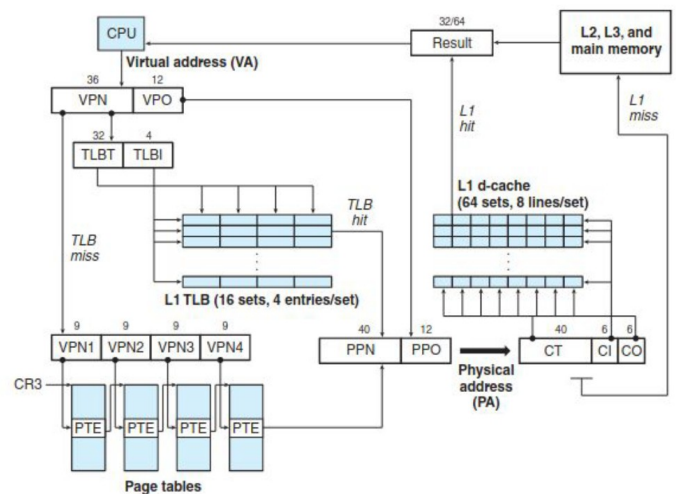
Para que sea útil, la búsqueda de la TLB necesita ser mucho más rápido que realizar una traducción completa de una dirección de memoria. Por ende, las entradas de la tabla de la TLB son implementadas en una memoria muy rápida, memoria estática on-chip, situada muy cerca del procesador. De hecho, para mantener esta búsqueda rápida, muchos sistemas en la actualidad incluyen múltiples niveles de TLB. En general cuanto más pequeña es la memoria, más rápida es la búsqueda.

Si el primer nivel de la TLB produce un TLB miss existe otro nivel, que guarda más datos, y este es consultado y la traducción se realiza si se falla en ambos niveles

La TLB



Todo completo



Consistencia de la TLB

Cada vez que se introduce un cache en el sistema, se necesita considerar la forma de asegurar la consistencia del cache con los datos originales cuando las entradas en el mismo son modificadas. Una TLB no es la excepción.

Para una ejecución correcta y segura de un programa, el sistema operativo tiene que asegurarse que cada programa ve su propia memoria y la de nadie más. Existen tres puntos a considerar:

- El **context switch**: Las direcciones virtuales del viejo proceso ya no son más válidas, y no deben ser válidas, para el nuevo proceso. De otra forma, el nuevo proceso sería capaz de leer las direcciones del viejo proceso. Frente a un context switch, se necesita descartar el contenido de TLB en cada context switch. Este approach se denomina **flush de TLB**. Debido a que este proceso acarrearía una penalidad, los procesadores taguean la TLB de forma tal que la misma contenga el id del proceso que produce cada transacción.
- **Reducción de Permiso**: ¿Qué sucede cuando el sistema operativo modifica una entrada en una page table? Normalmente no se provee consistencia por hardware para la TLB; mantener la TLB consistente con la page table es responsabilidad del sistema operativo.

- **TLB shutdown:** En un sistema multiprocesador cada uno puede tener cacheada una copia de una transacción en su TLB. Por ende, para seguridad y correctitud, cada vez que una entrada en la page table es modificada, la correspondiente entrada en todas las TLB de los procesadores tiene que ser descartada antes que los cambios tomen efecto. Típicamente sólo el procesador actual puede invalidar su propia TLB, por ello, para eliminar una entrada en todos los procesadores del sistema, se requiere que el sistema operativo mande una interrupción a cada procesador y pida que esa entrada de la TLB sea eliminada. Esta es una operación muy costosa y por ende tiene su propio nombre y se denomina TLB shutdown.

La TLB

Teniendo en cuenta los siguientes supuestos: una page table lineal (un arreglo) y una TLB que es manejada esencialmente por el hardware, el mecanismo de funcionamiento a groso modo debería ser el siguiente:

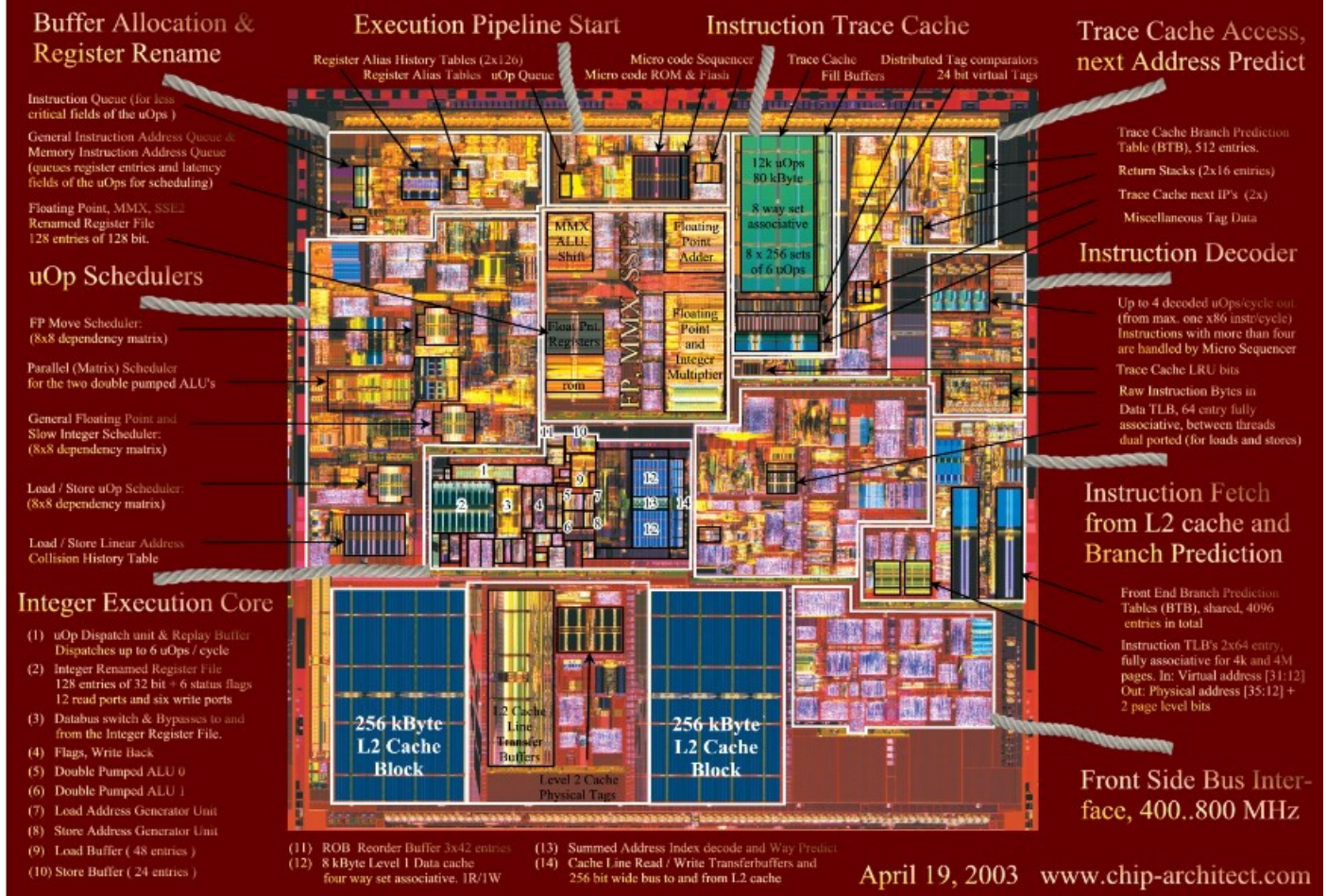
- Primero se extrae la virtual page number (VPN) de la virtual address y se chequea si la TLB tiene esa traducción para esa VPN. Si existe esa traducción se tiene un TLB HIT que significa que la TLB posee esa traducción.
- A partir de ahí, se puede extraer el Page Frame Number (PFN) de la entrada en la TLB
- Se concatena con el offset de la virtual address original y con ello se forma la physical address deseada y se accede a la memoria (suponiendo que todos los chequeos de protección no fallaron).

Si la CPU no encuentra esa traducción en la TLB (se tiene un TLB MISS), hay que hacer un poco más de trabajo para obtener la physical address. En este ejemplo:

- El hardware accede la page table para encontrar la traducción
- Suponiendo que la referencia a la memoria virtual generada por el proceso es válida y accesible se actualiza la TLB con la traducción hecha por el hardware. Este conjunto de acciones es costoso debido a que existen accesos extras a memoria.
- Finalmente, una vez que la TLB fue actualizada, el hardware vuelve a buscar la instrucción en la TLB y por ende la referencia a la memoria es procesada rápidamente.

Como todo cache, la TLB, está construida bajo la premisa que el **** caso común **** es que la traducción sea encontrada en el cache. Por ende poco es el costo que se agrega ya que la TLB está ubicada en cercanías del núcleo que está realizando el procesamiento y de hecho está diseñada para ser muy rápida.

Intel Pentium 4 Northwood



Cuando un MISS ocurre se incurre en el alto costo de la paginación; la page table tiene que ser accedida para encontrar la traducción correspondiente incurriendo en referencias extras a memoria. Si esto sucede mucho el programa va a parecer que se está ejecutando lentamente.

Que contiene la TLB

Una TLB típica puede tener entre 32, 64, o 128 entradas y ser lo que se llama fully associative. Básicamente, esto quiere decir que cualquier traducción puede encontrarse en cualquier lado de la TLB.

Una entrada de la TLB contiene los siguientes datos:

- VPN
- PFN
- Otros bits. Algunos de los "otros bits" son:
 - o bit de validez
 - o bits de protección (ejecución lectura)
 - o dirty bit
 - o entre otros

7 - Sistema de archivos (File system)

La mayoría de los usuarios de una computadora han tenido que lidiar con el concepto, de alto nivel, de lo que es un **file system**. Si bien el concepto puede parecer fácil de comprender, a muchos usuarios no expertos a veces se les complica entender el concepto.

¿Qué es un file system?

Un **file system o sistema de archivos** permite a los usuarios organizar sus datos para que se persistan a través de un largo período de tiempo.

De hecho, organizar un sistema de archivos de forma que sea funcional a un usuario no es tarea sencilla. Existen varias convenciones de cómo poner nombres en un file system.

Formalmente un **file system** es:

Una abstracción del sistema operativo que provee **datos persistentes con un nombre**.

Datos persistentes son aquellos que se almacenan hasta que son explícitamente (o accidentalmente) borrados, incluso si la computadora tiene un desperfecto con la alimentación eléctrica.

El hecho de que los datos tengan un nombre es con la intención de que un ser humano pueda acceder a ellos por un identificador que el sistema de archivos le asocia al archivo en cuestión.

Además, esta posibilidad de identificación permite que una vez que un programa termina de generar un archivo otro lo pueda utilizar, permitiendo así compartir la información entre los mismos.

Existen dos partes fundamentales de esta abstracción:

- 1) Los **archivos**, que están compuestos por un conjunto de datos.
- 2) Los **directorios**, que definen nombres para los archivos.

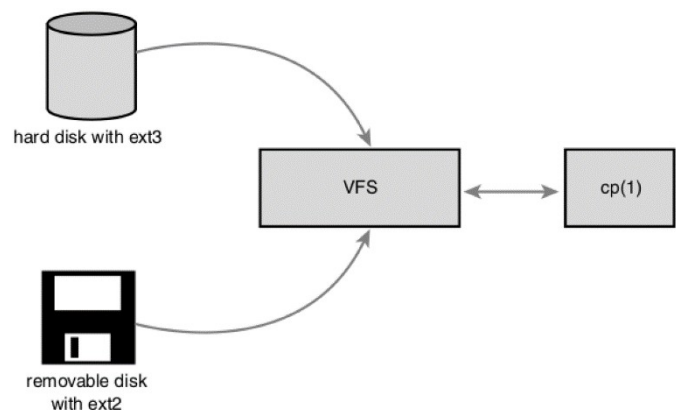
El virtual File System

El **virtual Files System (VFS)** es el subsistema del kernel que implementa la interfaces que tiene que ver con los archivos y el sistema de archivos provistos a los programas corriendo en modo usuario. Todos los sistemas de archivos deben basarse en VFS para:

- coexistir
- inter-operar

Esto habilita a los programas a utilizar las system calls de unix para leer y escribir en diferentes sistemas de archivos y diferentes medios.

VFS es el pegamento que habilita a las system calls como por ejemplo `open()`, `read()` y `write()` a funcionar sin que estas necesiten tener en cuenta el hardware subyacente.



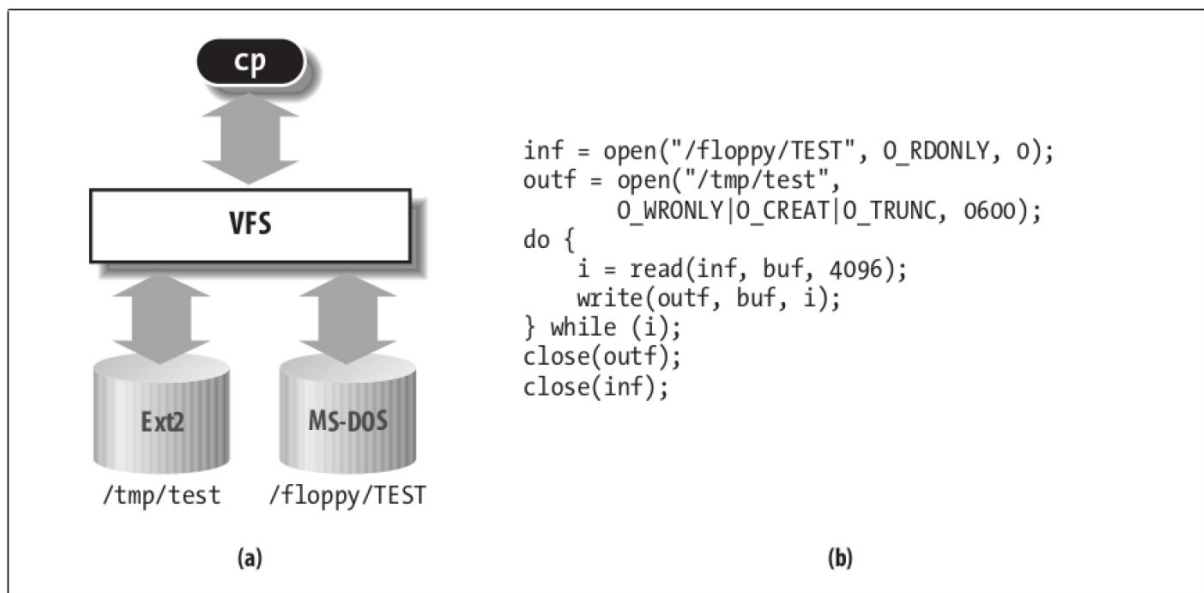
Ejemplo:

File System Abstraction Layer

Es un tipo genérico de interfaz para cualquier tipo de file system que es posible sólo porque el kernel implementa una capa de abstracción que rodea esta interface para con el sistema de archivo de bajo nivel.

Esta capa de abstracción habilita a Linux a soportar sistemas de archivos deferentes, incluso si estos difieren en características y comportamiento.

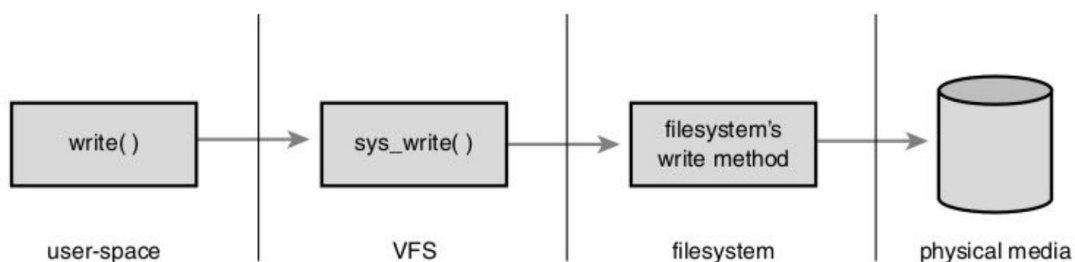
Esto es posible porque VFS provee un modelo común de archivos que pueda representar cualquier característica y comportamiento general de cualquier sistema de archivos.

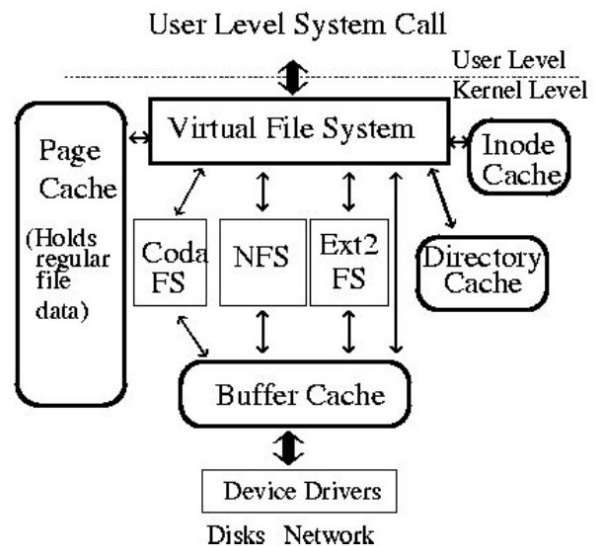
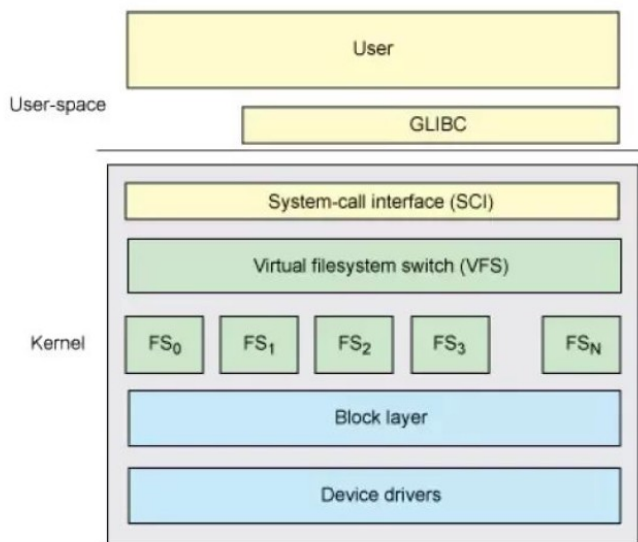


Esta capa de abstracción trabaja mediante la definición de interfaces conceptualmente básicas y de estructuras que cualquier sistema de archivos soporta.

Los filesystems amoldan su visión de conceptos como *"esta es la forma de como abro un archivo"* para matchear las expectativas del VFS, todos estos sistemas de archivos soportan nociones tales como **archivos**, **directorios** y además todos soportan un conjunto de operaciones básicas sobre estos.

El resultado es una **capa de abstracción general** que le permite al kernel manejar muchos tipos de sistemas de archivos de forma fácil y limpia.





Linux Kernel Map

VFS: Sus objetos

VFS presenta una serie de estructuras que modelan un filesystem, estas estructuras se denominan **objetos** (programadas en C). Estos Objetos son:

- El **super bloque**, que representa a un sistema de archivos.
- El **inodo**, que representa a un determinado archivo.
- El **dentry**, que representa una entrada de directorio, que es un componente *simple de un path* (no es todo el path).
- El **file** que representa a un archivo asociado a un determinado proceso.

VFS: dentry

Hay que tener en cuenta que un directorio es tratado como un archivo normal, no hay un objeto específico para directorios. En unix los directorios son archivos normales que listan los los archivos contenidos en ellos.

Ejemplo

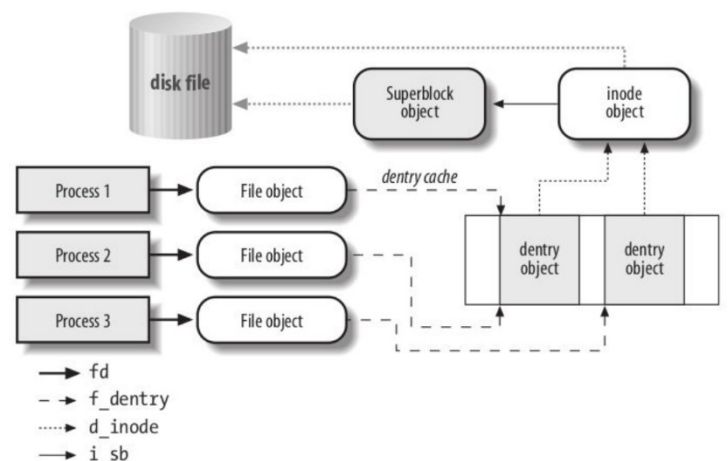
:: /home/darthmendez/hola.txt

Los dentries en ese path son: / (root), home, darthmendes y hola.txt

	inode	rec_len	file_type	name_len	name
0	21	12	1	2	· \0 \0 \0
12	22	12	2	2	· · \0 \0
24	53	16	5	2	h o m e 1 \0 \0 \0
40	67	28	3	2	u s r \0
52	0	16	7	1	o l d f i l e \0
68	34	12	4	2	s b i n

VFS: Operaciones

- Las **super_operations**: métodos que aplica el kernel sobre un determinado sistema de archivos. Por ejemplo: `write_inode()` o `sync_fs()`.



- ✂ Las **inode_operations**: métodos que aplica el kernel sobre un archivo determinado. Por ejemplo: create() o link().
- ✂ Las **dentry_operations**: métodos que se aplican directamente por el kernel a una determinada directory entry. Por ejemplo: d_compare() y d_delete().
- ✂ Las **file_operations**: métodos que el kernel aplica directamente sobre un archivo abierto por un proceso. Por ejemplo: read() y write().

Archivo

Un **archivo** es una colección de datos con un nombre específico.

Por ejemplo /home/mariano/MisDatos.txt.

Los archivos proveen una abstracción de más alto nivel que la que subyace en el dispositivo de almacenamiento; un archivo proporciona un nombre único y con significado para referirse a una cantidad arbitraria de datos.

Por ejemplo, /home/mariano/MisDatos.txt podría estar guardada en el disco en los bloques 0x0A23D42F, 0xE3A2540F y en 0x5567Ae34; es evidente que es muchísimo más fácil recordar /home/mariano/MisDatos.txt que esa lista de bloques en los que se almacena el archivo los datos.

Además, la información que se almacena en un archivo se divide en dos partes:

1. **Metadata**: Información acerca del archivo que es comprendida por el Sistema Operativo, esta información es:
 - o tamaño
 - o fecha de modificación
 - o propietario
 - o información de seguridad (que se puede hacer con el archivo).

Para poder ver la metadata de un archivo existen varias opciones:

```
$ ls -lisan prueba
```

o de una forma más human-readable:

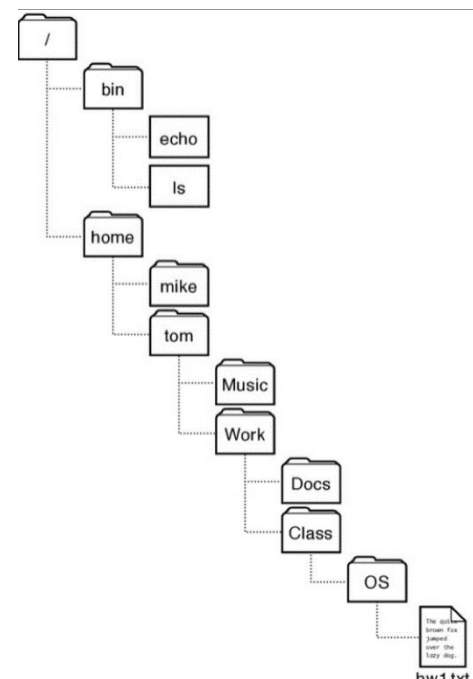
```
$ stat prueba
```

2. **Datos**: Son los datos propiamente dichos que quieren ser almacenados. Desde el punto de vista del Sistema Operativo, un archivo o file no es más que un arreglo de bytes sin tipo.

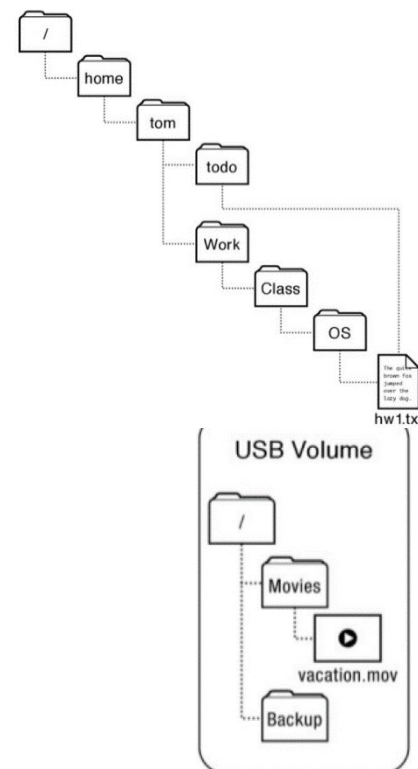
Directorios o Directories

Mientras que los archivos almacenan metadatos definidos por el S.O. y datos arbitrarios de los programas; los **directorios** proveen los nombres para estos archivos. En particular un directorio es una lista de nombre human-friendly y su mapeo a un archivo o a otro directorio.

- ▮ **path o ruta**: es el string que identifica unívocamente a un directorio o archivo dentro de un dispositivo.



- ▮ **root directory o directorio raíz:** es el directorio de que cuelgan todos los demás.
- ▮ **absolute path:** es la ruta desde el directorio raíz e.i. “/home/mariano/prueba”.
- ▮ **relative path:** es el path relativo que se interpreta a partir del directorio actual.
- ▮ **current directory:** es el directorio actual en el cual se está ejecutando el proceso.
- ▮ **hard link:** es el mapeo entre el nombre y el archivo subyacente, esto implica que la estructura de un file system que permite múltiples hard links ya no es de árbol invertido. Aquellos S.O. que lo permiten se cuidan de no crear ciclos asegurándose que la estructura sea un grafo dirigido acíclico.
- ▮ **soft links:** Se da cuando un archivo puede ser llamado por distintos nombres.
- ▮ **volumen:** Es una abstracción que corresponde a un disco lógico. En el caso más simple un disco corresponde a un disco físico. Es una colección de recursos físico de almacenamiento.
- ▮ **mount point:** Es un punto en el cual el root de un volumen se engancha dentro de la estructura existente de otro file system.



El API

Unix File Systems System Calls

Las *System Calls* de archivos pueden dividirse en dos clases:

1. Las que operan sobre los **archivos** propiamente dichos.
2. Las que operan sobre los **metadatos de los archivos**.

open()

La *System Call* **open()** convierte el nombre de un archivo en una entrada de la tabla de descriptores de archivos, y devuelve dicho valor. Siempre devuelve el descriptor más pequeño que no está abierto.

Los flags, estos flags pueden combinarse:

- ◆ O_RDONLY: modo solo lectura.
- ◆ O_WRONLY: modo solo escritura.
- ◆ O_RDWR: modo lectura y escritura.
- ◆ O_APPEND: el archivo se abre en modo lectura y el offset se setea al final, de forma tal que este pueda agregar al final.
- ◆ **O_CREATE: si el archivo no existe se crea con los permisos seteados en el parámetro mode:**
 - o S_IRWXU 00700 user (file owner) el usuario tiene permisos para leer, escribir y ejecutar
 - o S_IRUSR 00400 el usuario tiene permisos para leer.
 - o S_IWUSR 00200 el usuario tiene permisos para escribir.

- o S_IXUSR 00100 el usuario tiene permisos para ejecutar.
- o S_IRWXG 00070 el grupo tiene permisos para leer, escribir y ejecutar
- o S_IRGRP 00040 el grupo tiene permisos para leer.
- o S_IWGRP 00020 el grupo tiene permisos para escribir
- o S_IXGRP 00010 el grupo tiene permisos para ejecutar.
- o S_IRWXO 00007 otros tienen permisos para leer, escribir y ejecutar
- o S_IROTH 00004 otros tienen permisos para leer
- o S_IWOTH 00002 otros tienen permisos para escribir.
- o S_IXOTH 00001 otros tienen permisos para ejecutar

creat()

La *System Call* **creat()** equivale a llamar a **open()** con los flags O_CREAT|O_WRONLY|O_TRUNC.

close()

La *System Call* close cierra un file descriptor. Si este ya está cerrado devuelve un error.

Devuelve 0 si OK, -1 si hubo error.

read()

La llamada read se utiliza para hacer intentos de lecturas hasta un número dado de bytes de un archivo. La lectura comienza en la posición señalada por el file descriptor, y tras ella se incrementa ésta en el número de bytes leídos.

write()

la *System Call* **write()** escribe hasta una determinada cantidad (count) de bytes desde un buffer que comienza en buf al archivo referenciado por el file descriptor.

lseek()

La *System Call* **lseek()** reposiciona el desplazamiento (offset) de un archivo abierto cuyo filedescriptor es fd de acuerdo con el parámetro whence (de donde):

- ❖ SEEK_SET: el desplazamiento.
- ❖ SEEK_CUR: el desplazamiento es sumado a la posición actual del archivo.
- ❖ SEEK_END: el desplazamiento se suma a partir del final del archivo.

dup() y dup2()

Esta *System Call* crea una copia del file descriptor del archivo cuyo nombre es oldfd.

Después de que retorna en forma exitosa, el viejo y nuevo file descriptor pueden ser usados de forma intercambiable. Estos se refieren al mismo archivo abierto y por ende comparten el offset y los flags de estado.

dup2() hace lo mismo pero en vez de usar la política de seleccionar el file descriptor más pequeño utiliza a newfd como nuevo file descriptor.

Hasta el momento, podría parecer que existe una correspondencia uno a uno entre un filedescriptor y un archivo abierto. Sin embargo, no es así. Es a veces muy útil y necesario tener varios file descriptors referenciando al mismo archivo abierto. Estos file descriptors pueden haber sido abiertos en un mismo proceso o en otros. Para ello existen tres tablas de descriptores de archivos en el kernel:

- ❖ Per-process file descriptor table
- ❖ Una tabla system-wide de open file descriptors

❖ La file system i-node table

Para cada proceso, el kernel mantiene una tabla de *open file descriptors*. Cada entrada de esta tabla registra la información sobre un único fd:

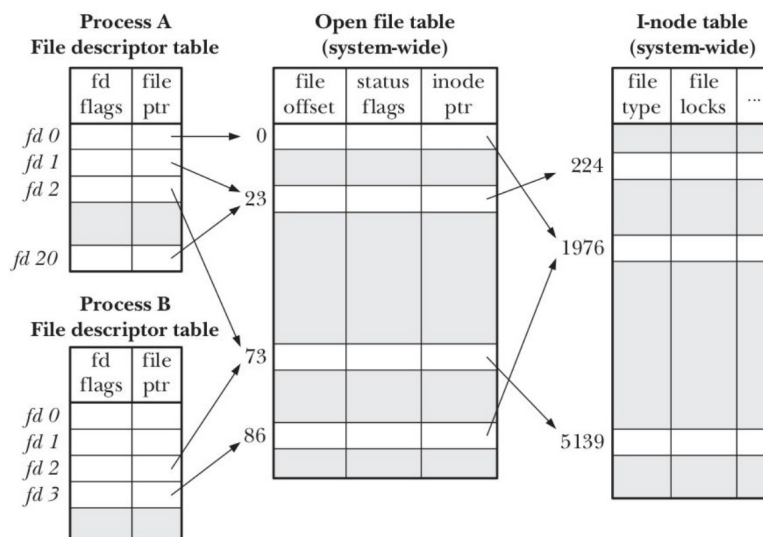
- ❖ Un conjunto de flags que controlan las operaciones del fd;
- ❖ Una referencia al open file descriptor.

Por otro lado, el kernel mantiene una tabla general para todo el sistema de todos los *open filedescriptor* (también conocida como la *open files table*), esta tabla almacena:

- El offset actual del archivo (que se modifica por *read()*, *write()* o por *lseek()*);
- los flags de estado que se especificaron en la apertura del archivo (los flags arguments de *open()*);
- el modo de acceso (solo lectura, solo escritura, escritura-lectura);
- una referencia al objeto i-nodo para este archivo.

Además cada sistema de archivos posee una tabla de todos los i-nodos que se encuentran en el sistema de archivos. En esta tabla se almacenarán:

- el tipo de archivo;
- un puntero a la lista de los locks que se mantienen sobre ese archivo;
- otras propiedades del archivo



link()

La *System Call* *link()* crea un nuevo nombre para un archivo. Esto también se conoce como un *link(hard link)*.

Nota: Este nuevo Nombre puede ser usado exactamente como el viejo nombre para cualquier operación, es más ambos nombres se refieren exactamente al mismo archivo y es imposible determinar cual era el nombre original.

unlink()

Esta *System Call* elimina un nombre de un archivo del sistema de archivos. Si además ese nombre era el último nombre o link del archivo y no hay nadie que tenga el archivo abierto **lo borra** completamente del sistema de archivos.

System Calls sobre Directorios

`mkdir()`

`rmdir()`

`chdir()`

`dirent.h`

Esta biblioteca permite el manejo de las entradas de directorios de linux.

`struct dirent`

Esta es la estructura de datos provista para poder leer las entradas a los directorios.

- ▮ `char d_name[]`: Este es el componente del nombre null-terminated. Es el único campo que está garantizado en todos los sistemas posix
- ▮ `ino_t d_fileno`: este es el número de serie del archivo.

`opendir()`

La función `opendir` abre y devuelve un stream que corresponde al directorio que se está leyendo en `dirname`. El stream es de tipo `DIR*`.

`readdir()`

Esta función lee la próxima entrada de un directorio. Normalmente devuelve un puntero a una estructura que contiene la información sobre el archivo.

`closedir()`

Cierra el stream de tipo `DIR *` cuyo nombre es `dirstream`.

System Call sobre los Metadatos

`stat()`

Esta familia de *System Calls* devuelven información sobre un archivo, en el buffer apuntado por `statbuf`. No se requiere ningún permiso sobre el archivo en cuestión, pero si en los directorios que conforma el path hasta llegar al archivo.

La estructura apuntada por `statbuf` se describe de la siguiente manera:

`access()`

La *System Call* `access` chequea si un proceso tiene o no los permisos para utilizar el archivo con un determinado `pathname`. El argumento `mode` determina el tipo de permiso a ser chequeado.

El modo (`mode`) especifica el tipo de accesibilidad a ser chequeada, los valores pueden conjugarse como una máscara de bits con el operador `|`:

- `F_OK`: el archivo existe.
- `R_OK`: el archivo puede ser leído.
- `W_OK`: el archivo puede ser escrito.
- `X_OK`: el archivo puede ser ejecutado.

`chmod()`

Estas *System Calls* cambian los bits de modos de acceso.

- S_IRUSR (00400) read by owner
- S_IWUSR (00200) write by owner
- S_IXUSR (00100) execute/search by owner (“search” applies for directories, and means that entries within the directory can be accessed)
- S_IRGRP (00040) read by group
- S_IWGRP (00020) write by group
- S_IXGRP (00010) execute/search by group
- S_IROTH (00004) read by others
- S_IWOTH (00002) write by others
- S_IXOTH (00001) execute/search by others
- La única diferencia entre ambas system calls es la forma en que se accede al archivo en cuestión.

chown()

Estas *system Calls* cambian el id del propietario del archivo y el grupo de un archivo.

umask()

Esta *System Call* setea la máscara con el modo de permisos para la creación de archivos. La máscara por defecto es la 022.

Some File System Unix Commands

Fyle System Commands

Comma nd	System Call
cat	read() write() open() close()
rm	unlink() stat()
fstqat	stat()
tee	open() close() read() write()
tousc	creat() stat()
ls	readir() write()

Implementación de File System

8 - Scheduling

Cuando hay múltiples cosas qué hacer ¿Cómo se elige cuál de ellas hacer primero?

Debe existir algún mecanismo que permita determinar cuánto tiempo de CPU le toca a cada proceso. Ese período de tiempo que el kernel le otorga a un proceso se denomina **time slice** o **timequantum**.

Esto sucede en los sistemas operativos de tipo Time Sharing.

Time sharing

Los Sistemas Operativos llamados **time sharing** surgen de la idea de los programadores de tener **“toda una computadora para uno mismo”**. Esta idea que en la actualidad puede parecer trivial, pero en la década de los 60's era algo impensable.

Inicialmente IBM fue la primera empresa en utilizar **circuitos integrados** para sus computadoras, la línea IBM 360 (https://en.wikipedia.org/wiki/IBM_System/360_Model_20) , y su mayor fortaleza era la idea de tener una sola familia de computadoras.

El sistema operativo que corría en estas máquinas es el conocido OS 360. Esta familia de computadoras introdujo varias técnicas innovadoras en el desarrollo de los sistemas operativos. Una de estas técnicas es la de *multiprogramación*.

Multiprogramación

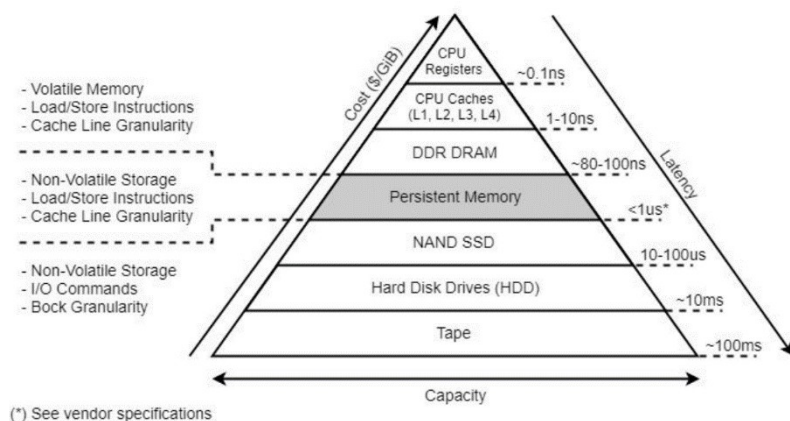
La idea básica de la multiprogramación es la siguiente: **si un determinado programa que se estaba ejecutando debía realizar operaciones de entrada y salida de datos, debía interactuar con un dispositivo de entrada y salida. Esta interacción en términos de tiempos de ejecución de instrucciones del procesador podía parecer infinito. Además, con el agravante de que el tiempo de E/S solía ser en promedio de entre el 80% y 90% del tiempo total del programa, la CPU permanecía inactiva un gran período de tiempo.**

Utilización de la CPU

Si se asume que el 20% del **tiempo de ejecución** de un programa es solo cómputo y el 80% son **operaciones de entrada y salida**, con tener 5 procesos en memoria se estaría utilizando el 100% de la CPU.

Siendo un poco más realista se supone que las operaciones de E/S son **bloqueantes** (una operación de lectura a disco tarda 10 miliseg y una instrucción registro registro tarda 1-2 nanoseg), es decir, paran el procesamiento hasta que se haya realizado la operación de E/S.

Entonces, el cálculo es más realista si se supone que un proceso gasta una **fracción p**, bloqueado en E/S. De esta manera, si tenemos **n procesos** esperando para hacer operaciones de entrada y salida, la probabilidad de que los **n procesos** estén haciendo E/S es p^n .



Por ende la probabilidad de que se esté ejecutando algún proceso es $1 - p^n$, esta fórmula es conocida como **utilización de CPU**.

Ejemplo:

si se tiene un solo proceso en memoria y este tarda un 80% del tiempo en operaciones de E/S el tiempo de utilización de CPU es $1 - 0.8 = 0.2$ que es el 20%.

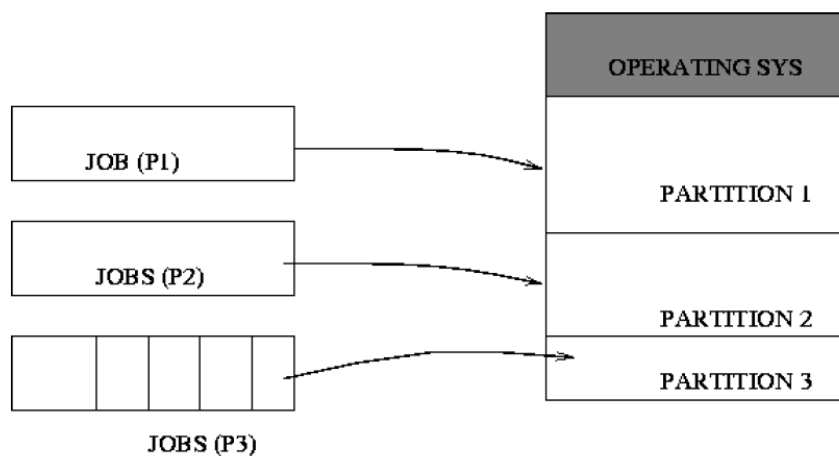
Ahora bien, si se tienen 3 procesos con la misma propiedad, el grado de utilización de la CPU es $= 0.488$ es decir el 48 de ocupación de la cpu.

Si se supone que se tienen 10 procesos, entonces la fórmula cambia a $= 0.89$ el 89% de utilización, aquí es donde se ve la **IMPORTANCIA** de la **Multiprogramación**.

Hay que tener en cuenta que estos tiempos varían de proceso en proceso.

Multiple Fixed partitions

1967 - IBM System /360 (OS/MFT)



Multi tasking with a Fixed number of Tasks _MFT_: varias particiones de memoria, cada una de un tamaño fijo, se establecían cuando el sistema operativo se instalaba o podían ser redefinidas por el operador.

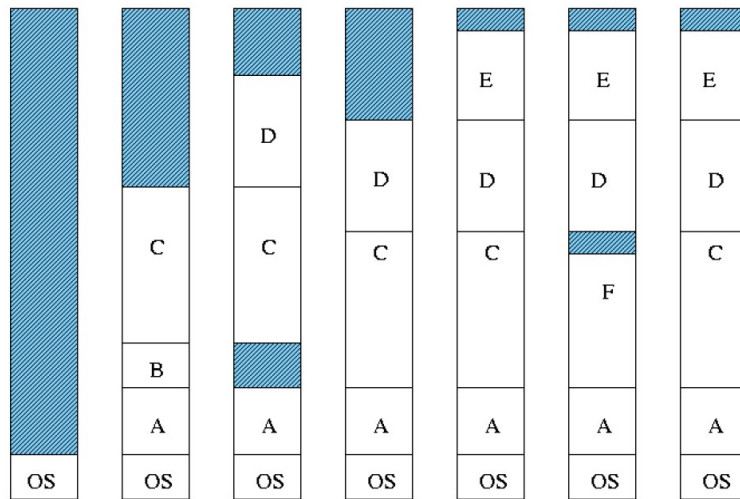
Por ejemplo, podría haber una partición pequeña, dos particiones medianas y una partición grande.

Si había dos programas grandes listos para funcionar, uno tendía que esperar hasta que el otro terminara y desocupara la partición grande.

1. Cada partición era monoprogramada, la multiprogramación ocurría entre las particiones.
2. Los límites de las particiones no eran móviles.
3. Las particiones eran de tamaño fijo.
4. La fragmentación era un problema.
5. No existía el concepto de memoria compartida.

Multiple variable Partitions

IBM System /360 OS/MVT



Multitasking with a Variable number of Tasks MVT: trataba a toda la memoria no utilizada por el sistema operativo como un único gran espacio desde el cual se podían asignar “regiones” contiguas tanto como lo requiriera un número ilimitado de aplicaciones y programas simultáneamente.

1. Cada tarea podía tener cualquier tamaño, la limitante era la memoria de la máquina.
2. Una única cola de procesos listo para correr.
3. La tarea podía ser movida.
4. Se elimina la fragmentación interna.
5. Introduce la fragmentación externa.

Time sharing

Si bien la idea de multitarea agilizó el procesamiento de los procesos, aún las computadoras utilizaban su procesador para ejecutar un único programa a la vez. La noción de tener una computadora para un solo programador era un poco descabellada aún.

A medida que los tiempos de respuesta entre procesos se fueron haciendo cada vez más pequeños, más procesos podían ser cargados en memoria para su ejecución.

Una variante de la técnica de multiprogramación consistió en asignar una terminal a cada usuario en línea.

Teniendo en cuenta que los seres humanos tienen un tiempo de respuesta lento (0.25 seg para estímulos visuales) en comparación a una computadora (operación en nanosegundos). Debido a esta diferencia de tiempos y a que no todos los usuarios necesitan de la cpu al mismo tiempo, este sistema daba la sensación de asignar toda la computadora a un usuario determinado Corbató y otros. Este concepto fue popularizado por MULTICS.

Cuando un Sistema Operativo se dice que realiza multi-programación de varios procesos debe existir una entidad que se encargue de coordinar la forma en que estos se ejecutan, el momento en que estos se ejecutan y el momento en el cual paran de ejecutarse. En un sistema operativo esta tarea es realizada por el **Planificador o Scheduler** que forma parte del Kernel del Sistema Operativo

Números y el workload

El Workload es carga de trabajo de un proceso corriendo en el sistema.

Determinar cómo se calcula el **workload** es fundamental para determinar partes de las políticas de planificación. Cuanto mejor es el cálculo, mejor es la política. Las suposiciones que se harán para el cálculo del workload son más que irreales.

Los supuestos sobre los procesos o **jobs** que se encuentran en ejecución son:

1. Cada proceso se ejecuta la misma cantidad de tiempo.
2. Todos los jobs llegan al mismo tiempo para ser ejecutados.
3. Una vez que empieza un job sigue hasta completarse.
4. Todos los jobs usan **únicamente CPU**.
5. El tiempo de ejecución (run-time) de cada job es conocido.

Métricas de planificación

Para poder hacer algún tipo de análisis se debe tener algún tipo de métrica estandarizada para comparar las distintas **políticas de planificación o scheduling**. Bajo estas premisas, por ahora, para que todo sea simple se utilizará una única métrica llamada **turnaround time**. Que se define como el tiempo en el cual el proceso se completa menos el tiempo de arribo al sistema:

$$T_{turnaround} = T_{completion} - T_{arrival}$$

Debido a 2 el $T_{arrival} = 0$

Hay que notar que el turnaround time es una métrica que mide performance.

Políticas para sistemas mono-procesador

Se estudiarán las políticas de planificación para un sistema que posea un solo procesador o CPU con un solo núcleo de procesamiento.

First In, First Out (FIFO)

El algoritmo más básico para implementar como política de planificaciones es el First In First Out o First Come, First Served.

Ventajas:

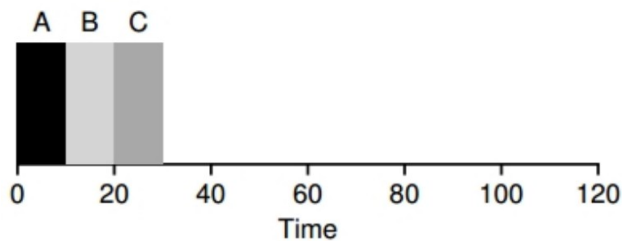
- Es simple.
- Por 1 es fácil de implementar.
- Funciona bárbaro para las suposiciones iniciales.

Ejemplo

a) Se tiene tres procesos A, B y C con $T_{arrival} = 0$. Si bien llegan todos al mismo tiempo llegaron con un insignificante retraso de forma tal que llegó A, B y C. Si se asume que todos tardan 10 segundos en ejecutarse... ¿cuánto es el T_{around} ?

$$(10+20+30)/3=20$$

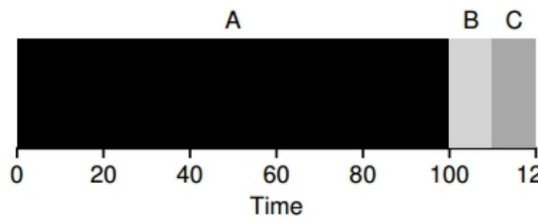
Esto se puede apreciar en la imagen:



- b) Ahora relajemos la suposición 1 y no se asume que todas las tareas duran el mismo tiempo. Ahora A dura 100 segundos. ¿Cuánto es el T_{around} ?

$$(100+110+120)/3=110$$

Segundos en promedio... un desastre esto se llama **convoy effect**.



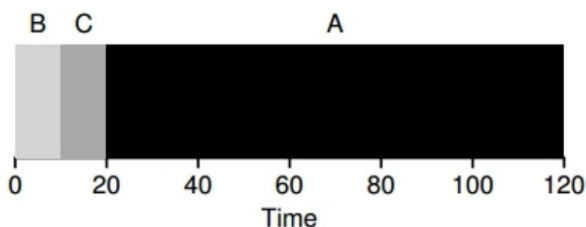
Shortest Job First (SJF)

Para resolver el problema que se presenta en la política FIFO, se modifica la política para que se ejecute el proceso de duración mínima, una vez finalizado esto se ejecuta el proceso de duración mínima y así sucesivamente.

Ejemplo

- a) En el mismo caso de arriba, se mejora el turnaround time con el sencillo hecho de ejecutar B, C y A en ese orden:

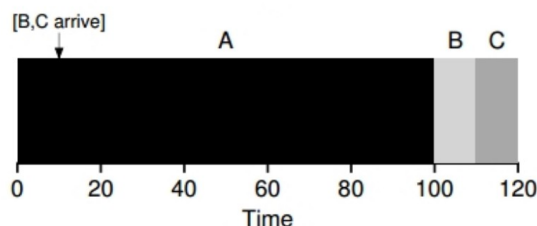
$$(10+20+120)/3=50$$



Utilizando SJF se obtuvo una significativa mejora... pero con las suposiciones iniciales que son muy poco realistas.

- b) Si se relaja la suposición 2, en la cual no todos los procesos llegan al mismo tiempo, por ejemplo, llega el proceso A y a los 10 segundos llegan el proceso B y el proceso C. ¿Cómo sería el cálculo, ahora? $t_a=10$ seg

$$(100+110-10+120-10)/3=103.33$$



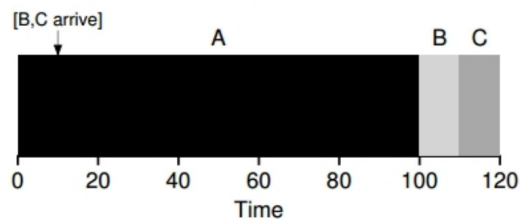
Shortest Time-to-Completion (STCF)

Para poder solucionar este problema se necesita relajar la suposición 3 (los procesos se tienen que terminar hasta el final). La idea es que el planificador o scheduler pueda

adelantarse y determinar qué proceso debe ser ejecutado. Entonces cuando los procesos B y C llegan se puede desalojar (preempt) al proceso A y decidir que otro proceso se ejecute y luego retomar la ejecución del proceso A.

Ejemplo

c) El caso anterior el de SFJ es una política **non-preemptive**



El cálculo del **turnaround time** sería:

$$(120 - 0 + 20 - 10 + 30 - 10) / 3 = 50$$

Una nueva métrica: Tiempo de Respuesta

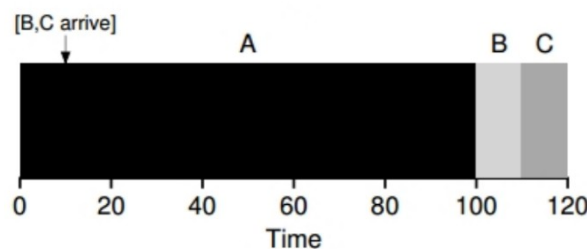
El tiempo de respuesta o response time surge con el advenimiento del **time-sharing** ya que los usuarios se sientan en una terminal de una computadora y pretenden una interacción con rapidez. Por eso nace el **response time** como métrica:

$$T_{response} = T_{firstrun} - T_{arrival}$$

Para entender la definición veamos el caso anterior:

Ejemplo

c)



- El $T_{response}$ del proceso A es 0.
- El $T_{response}$ del proceso B es... 0... llega en 10 pero tarda 10 ($\Rightarrow 10 - 10 = 0$).
- El $T_{response}$ del proceso C es... 10... llega en 10 pero termina en 20 ($\Rightarrow 20 - 10 = 10$).

El promedio del $T_{response}$ es de 3.33 seg. Entonces, ¿cómo escribir un planificador que tenga noción del tiempo de respuesta?

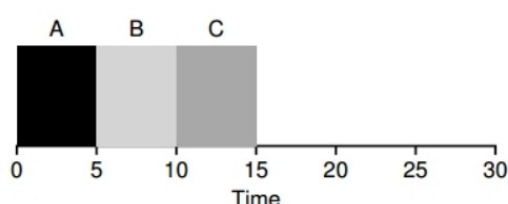
Round Robin (RR)

La idea del algoritmo es bastante simple, se ejecuta un proceso por un período determinado de tiempo (slice) y transcurrido el período se pasa a otro proceso, y así sucesivamente cambiando de proceso en la cola de ejecución.

Ejemplo

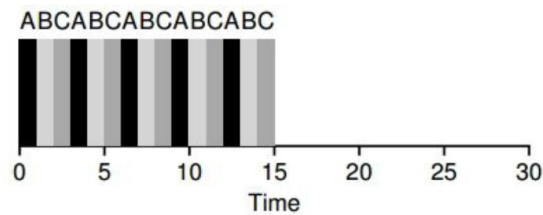
c) Los procesos A, B y C llegan a ejecutarse en el mismo instante y tardan 5 segundos

I. si se utiliza SJF:



$$(0 + 5 + 10) / 3 = 5$$

II. Para RR con time slice de 1 seg:



$$(0+1+2)/3=1$$

El **tiempo de respuesta promedio** sería 1.

Lo importante de RR es la elección de un buen time slice, se dice que el time slice tiene que amortizar el cambio de contexto sin hacer que esto resulte en que el sistema no responda más.

Por ejemplo, si el tiempo de cambio de contexto está seteado en 1 ms y el time slice está seteado en 10 ms, el 10% del tiempo se estará utilizando para cambio de contexto.

Sin embargo, si el time slice se setea en 100 ms, solo el 1% del tiempo será dedicado al cambio de contexto:

- I. ¿Qué pasa si se trae a colación a la métrica del turnaround time?
- II. ¿Qué pasa cuando hay Entrada y salida de datos? (Cap 7 del Arpachi y Cap 7 del Dahlin)

La planificación en la vida real

1. ¿Qué debería proporcionar un marco de trabajo básico que permita pensar en políticas de planificaciones?
2. ¿Cuáles deberían ser las suposiciones a tener en cuenta?
3. ¿Cuáles son las métricas importantes?

Multi-Level Feedback Queue (MLFQ)

Esta técnica llamada **Multi-Level Feedback Queue** de planificación fue descrita inicialmente en los años 60 en un sistema conocido como **Compatible Time Sharing System CTSS**. Este trabajo en conjunto con el realizado sobre **MULTICS** llevó a que su creador ganara el **Turing Award**.

Este planificador ha sido refinado con el paso del tiempo hasta llegar a las implementaciones que se encuentran hoy en un sistema moderno.

MLQF intenta atacar principalmente 2 problemas:

1. Intenta optimizar el turnaround time, que se realiza mediante la ejecución de la tarea más corta primero, desafortunadamente el sistema operativo nunca sabe a priori cuanto va a tardar en correr una tarea.
2. MLFQ intenta que el planificador haga sentir al sistema con un tiempo de respuesta interactivo para los usuarios por ende minimizar el **response time**; desafortunadamente los algoritmos como round-robin reducen el **response time** pero tienen un terrible **turnaround time**.

Entonces:

- ¿Cómo se hace para que un planificador pueda lograr estos dos objetivos si generalmente no se sabe nada sobre el proceso a priori?

- ¿Cómo se planifica sin tener un conocimiento acabado?
- ¿Cómo se construye un planificador que minimice el tiempo de respuesta para las tareas interactivas y también minimice el **timearound** time sin un conocimiento a priori de cuánto dura la tarea?

MLQF: Las reglas básicas

MLFQ tiene un conjunto de distintas colas, cada una de estas colas tiene asignado un nivel de prioridad.

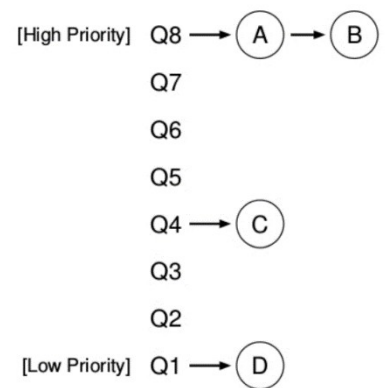
En un determinado tiempo, una tarea que está lista para ser corrida está en una única cola. MLFQ usa las prioridades para decidir cual tarea debería correr en un determinado tiempo t_0 : la tarea con mayor prioridad o la tarea en la cola más alta será elegida para ser corrida.

Dado el caso que existan más de una tarea con la misma prioridad entonces se utilizara el algoritmo de **Round Robin** para planificar estas tareas entre ellas.

Las 2 reglas básicas de MLFQ:

- I. **REGLA 1:** si la prioridad (A) **es mayor** que la prioridad de (B), (A) se ejecuta y (B) no.
- II. **REGLA 2:** si la prioridad de (A) **es igual** a la prioridad de (B), (A) y (B) se ejecutan en **Round-Robin**.

La clave para la planificación MLFQ subyace entonces en cómo el planificador setea las prioridades. En vez de dar una prioridad fija a cada tarea, MLFQ varia la prioridad de la tarea basándose en su comportamiento observado.



Por ejemplo, si una determinada tarea repetidamente no utiliza la CPU mientras espera que un dato sea ingresado por el teclado, MLFQ va a mantener su prioridad alta, así es como un proceso interactivo debería comportarse.

Si por lo contrario, una tarea usa intensivamente por largos periodos de tiempo la CPU, MLFQ reducirá su prioridad. De esta forma MLFQ va a aprender mientras los procesos se ejecutan y entonces va a usar la historia de esa tarea para predecir su futuro comportamiento

9- Concurrencia

Existen **dos** cosas muy difíciles de hacer en Ciencias de la Computación:

1. Poner Nombres a las cosas.
2. La Concurrencia.
3. Errar por uno.

La concurrencia

El mundo de la concurrencia se refiere a un conjunto de actividades que pueden suceder al mismo tiempo. [Anderson-Dahlin pag. 129].

El correcto manejo de la concurrencia es una de las claves en el desarrollo de los Sistemas Operativos Modernos. En este tema se verá cuál es la abstracción que maneja y disminuye la complejidad del problema de la Concurrencia.

Uno de los aspectos más interesantes al enfrentarse con la Concurrencia, es la manera en que nuestra mente es capaz de atacar un problema:

1. En el caso de la mayoría de los programadores la forma más común de construir programas es la llamada forma secuencial: ejecutar una acción detrás de la otra. Estos es lo que la mayoría de los programadores realizan cotidianamente.
2. Pensar en atacar un problema en el cual decenas de eventos pueden desencadenarse al mismo tiempo es aún muy complejo...

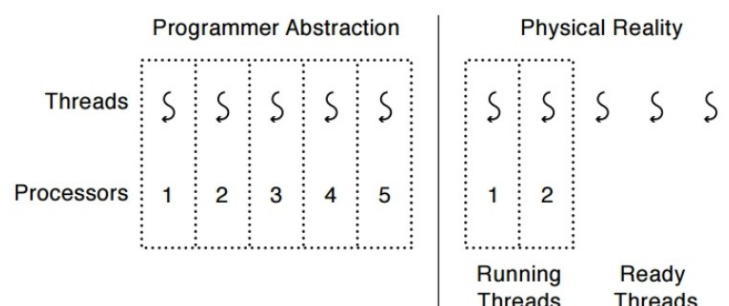
Además, un problema de los seres humanos se encuentra en nuestra limitación para manejar distinta información al mismo tiempo. Según el artículo de G.A. Miller "Magical Seven Psych Review" de 1955, los seres humanos podemos manejar 7 ± 2 **chunks of data**.

Una parte muy interesante del mismo está en la sección de conclusiones:

"And finally, what about the magical number seven? What about the seven wonders of the world, the seven seas, the seven deadly sins, the seven daughters of Atlas in the Pleiades, the seven ages of man, the seven levels of hell, the seven primary colors, the seven notes of the musical scale, and the seven days of the week? What about the seven-point rating scale, the seven categories for absolute judgment, the seven objects in the span of attention, and the seven digits in the span of immediate memory? For the present I propose to withhold judgment. Perhaps there is something deep and profound behind all these sevens, something just calling out for us to discover it. But I suspect that it is only a pernicious, Pythagorean coincidence." (Miller, 1955)

¿A qué apuntamos?

El concepto clave es escribir un programa concurrente como una secuencia de **streams de ejecución o threads** que interactúan y comparten datos en una manera muy precisa. El concepto básico es el siguiente:



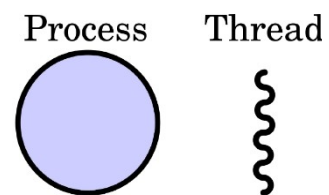
La abstracción

Un *thread* es una **secuencia de ejecución atómica que representa una tarea planificable de ejecución**

- **Secuencia de ejecución atómica:** Cada thread ejecuta una secuencia de instrucciones como lo hace un bloque de código en el modelo de programación secuencial.
- **Tarea planificable de ejecución:** El sistema operativo tiene injerencia sobre el mismo en cualquier momento y puede ejecutarlo, suspenderlo y continuarlo cuando él desee.

Threads vs. procesos

- ❖ **Proceso:** un programa en ejecución con derechos restringidos.
- ❖ **Thread:** una secuencia independiente de instrucciones ejecutándose dentro de un programa.

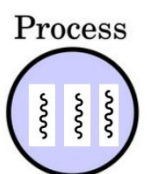
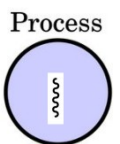


Esta abstracción, el thread, se caracteriza por:

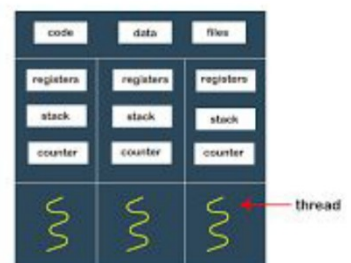
- o Thread id
- o un conjunto los valores de registros
- o stack propio
- o una política y prioridad de ejecución
- o un propio errno
- o datos específicos del thread

Threads

1. **One thread per process:** un proceso con una única secuencia de instrucciones ejecutándose de inicio a fin. Para los que vieron Pascal o C sería el equivalente a un bloque de instrucciones delimitado por begin-end o { }. Lo que todos los programadores de modelo secuencial conocemos.
2. **Many thread per process:** un programa es visto como threads ejecutándose dentro de un proceso con derechos restringidos. En dado un t_i algunos threads pueden estar corriendo y otros estar suspendidos. Cuando se detecta por ejemplo una operación de I/O por alguna interrupción, el kernel desaloja (preempt) a algunos de los threads que están corriendo, atiende la interrupción, y al terminar de manejar la interrupción vuelve a correr el thread nuevamente.
3. **Many single-threaded processes:** limitación de algunos sistemas operativos que permitían varios procesos, pero cada uno con un único thread, lo que implica que puede haber varios threads ejecutándose en kernel model.
4. **Many kernel threads:** para aprovechar recursos, también el kernel puede ejecutar varios threads en kernel mode.



Single-threaded process



Multi-threaded process

Thread scheduler

¿Cómo hace el S.O. para crear la ilusión de muchos threads con un número fijo de procesadores?

Obviamente es necesario un **planificador de thread o threads scheduler**, ya que el S.O. podría estar trabajando con un único procesador. El cambio entre threads es transparente, es decir, que el programador debe preocuparse de la secuencia de instrucciones y no de cuándo este debe ser suspendido o no.

Por ende los Threads proveen un modelo de ejecución en el cual cada thread corre en un procesador virtual dedicado (exclusivo) con una velocidad variable e impredecible [AndersonDahlin, pag 138].

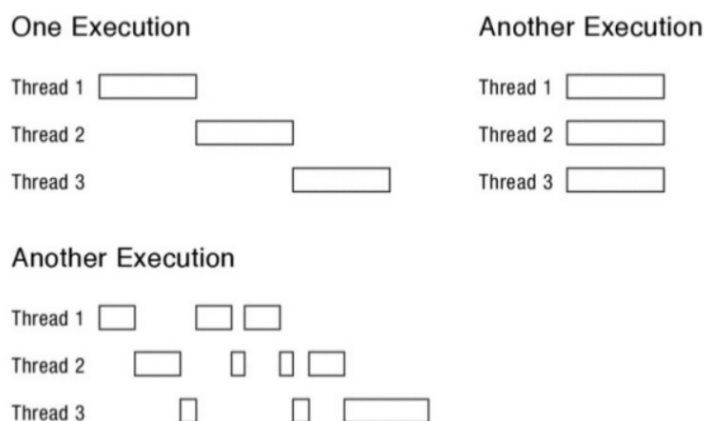
Esto quiere decir que desde el punto de vista del thread cada instrucción se ejecuta inmediatamente una detrás de otra. Pero el que decide cuando se ejecuta es el planificador de threads o thread scheduler. Por ejemplo:

```
...
...
x = x + 1;
y = x + y;
z = x + 5y;
...
...
```

Entonces en base a lo antedicho, se pueden encontrar los siguientes escenarios de ejecución:

Programmer's View	Possible Execution #1	Possible Execution #2	Possible Execution #3
<pre> x = x + 1; y = y + x; z = x + 5y;</pre>	<pre> x = x + 1; y = y + x; z = x + 5y;</pre>	<pre> x = x + 1; Thread is suspended. Other thread(s) run. Thread is resumed. y = y + x; z = x + 5y;</pre>	<pre> x = x + 1; y = y + x; Thread is suspended. Other thread(s) run. Thread is resumed. z = x + 5y;</pre>

Y los siguientes interleaves o entrelazado pueden suceder, con estos distintos threads:



En la actualidad hay dos formas de que los threads se relacionen entre sí:

1. Multi-threading Cooperativo: no hay interrupción a menos que se solicite. -> ¿Problemas?
2. Multi-threading Preemptivo: Es el más usado en la actualidad. Consiste en que un threads en estado de *running* puede ser movido en cualquier momento.

El API de Threads

Para la programación utilizando threads se utilizará la biblioteca pthread donde la p es de POSIX Threads. El API de pthreads es muy completa y la iremos viendo a medida que se la necesite.

Creación de un thread

```
#include <pthread.h>

int pthread_create(pthread_t * thread, const pthread_attr_t * attr,
                  void * (start_routine) (void *), void * arg)
```

Esta función tiene cuatro argumentos:

6. **thread**: Es un puntero a la estructura de tipo pthread_t, que se utiliza para interactuar con el threads.
7. **attr**: Se utiliza para especificar los ciertos atributos que el thread debería tener, por ejemplo, el tamaño del stack, o la prioridad de scheduling del thread. En la mayoría de los casos es NULL.
8. **start_routine**: Sea tal vez el argumento más complejo, pero no es más que un puntero a una función, en este caso que devuelve void.
9. **arg**: Es un puntero a void que debe apuntar a los argumentos de la función.

devuelve 0 si se ha creado el thread con éxito, si hubo error devuelve otro valor.

Terminación de un thread

```
int pthread_join(pthread_t thread, void **value_ptr)
```

Muchas veces es necesario esperar a que un determinado thread finalice su ejecución, para ello se utilizan la función pthread_join(), que toma dos argumentos:

1. **thread**: es el thread por el que hay que esperar y es de tipo pthread_t.
2. **value_ptr**: es el puntero al valor esperado de retorno.

Estructura y ciclo de vida de un thread

Como se ha visto, cada thread es la representación de una secuencia de ejecución de un conjunto de instrucciones. El S.O. provee la ilusión de que cada uno de estos threads se ejecutan en su propio procesador, haciendo de forma transparente que se ejecuten o paren su ejecución.

Para que la ilusión sea creíble, el sistema operativo debe guardar y cargar el estado de cada thread. Como cualquier thread puede correr en el procesador o en el kernel, también debe haber estados compartidos, que no deberían cambiar entre los modos.

Para poder entender la abstracción hay que comprender que existen dos estados:

1. El estado **per thread**.
2. El estado **compartido** entre varios threads.

El Estado Per-thread y Threads Control Block (TCB)

Cada thread debe tener una estructura que represente su estado. Esta estructura se denomina **Thread Control Block (TCB)**, se crea una entrada por cada thread. La TCB almacena el estado per-thread de un thread:

El estado del Cómputo que debe ser realizado por el thread.

Para poder crear múltiples threads y pararlos y reanuncarlos, el S.O. debe poder almacenar en la TCB el estado actual del bloque de ejecución:

1. El puntero al stack del thread.
2. Una copia de sus registros en el procesador.

Metadata referente al thread que es utilizada para su administración

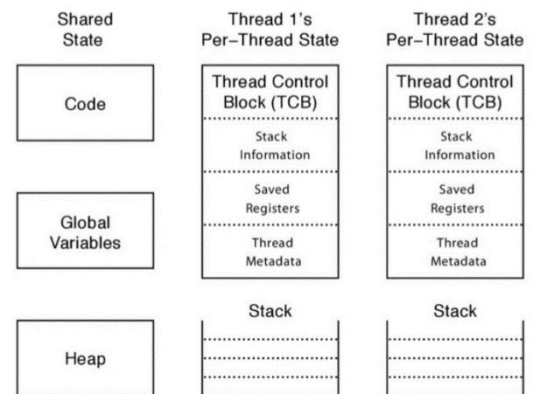
Por cada thread se debe guardar determinada información sobre el mismo:

- ID
- Prioridad de scheduling
- Status

Shared State o Estado Compartido

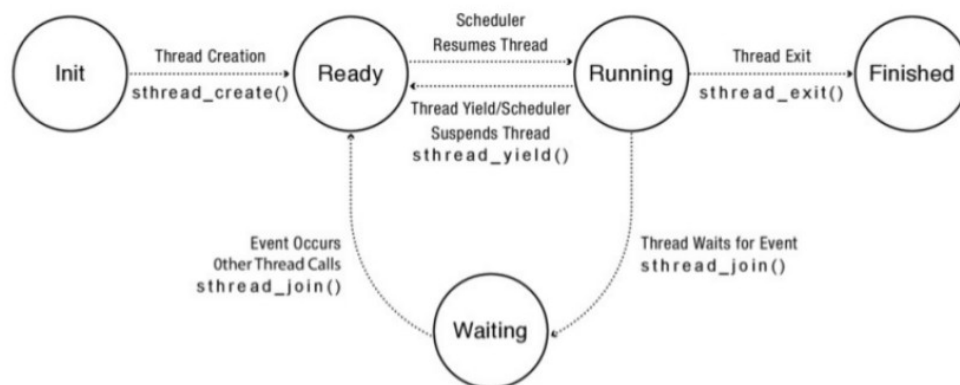
De forma contraria al per-thread state se debe guardar cierta información que es compartida por varios Threads:

- El Código
- Variables Globales
- Variables del Heap



Estados de un thread

Los estados de un Thread son:



- **Init:** Un thread se encuentra en estado **INIT** mientras se está inicializando el estado per-thread y se está reservando el espacio de memoria necesario para estas estructuras. Una vez que esto se ha realizado el estado del thread se setea en **READY**. Además se lo pone en una lista llamada *ready list* en la cual están esperando todos los thread listos para ser ejecutados en el procesador.
- **Ready:** Un thread en este estado está listo para ser ejecutado pero no está siendo ejecutado en ese instante. La TCB está en la **ready list** y los valores de los registros está

en la TCB. En cualquier momento el **thread scheduler** puede transicionar al estado **RUNNING**.

- **Running:** Un thread en este estado está siendo ejecutado en este mismo instante por el procesador. En este mismo instante los valores de los registros están en el procesador. En este estado un **RUNNING THREAD** puede pasar a **READY** de dos formas:
 1. El **scheduler** puede pasar un thread de su estado **RUNNING** a **READY** mediante el desalojo o **preemption** del mismo mediante el guardado de los valores de los registros y cambiando el thread que se está ejecutando por el próximo de la lista.
 2. Voluntariamente un thread puede solicitar abandonar la ejecución mediante la utilización de `thread_yield`, por ejemplo.
- **Waiting:** En este estado el Thread está esperando que algún determinado evento suceda. Dado que un thread en **WAITING** no puede pasar a **RUNNING** directamente, estos thread se almacenan en la lista llamada *waiting list*. Una vez que el evento ocurre el scheduler se encarga de pasar el thread del estado **WAITING** a **RUNNING**, moviendo la TCB desde el *waiting list* a la *ready list*.
- **Finished:** Un thread que se encuentra en estado **FINISHED** nunca más podrá volver a ser ejecutado. Existe una lista llamada *finished list* en la que se encuentran las TCB de los threads que han terminado.

Tabla de equivalencia entre procesos y threads

Process primitive	Thread primitive	Description
fork	pthread_create	crea un nuevo flujo de control
exit	pthread_exit	sale de un flujo de control existente
waitpid	pthread_join	obtiene el estado de salida de un flujo de control
atexit	pthread_cleanup	función a ser llamada en el momento de salida de un flujo de control
getpid	pthread_self	obtiene el id de un determinado flujo de control
abort	pthread_cancel	terminación anormal de un flujo de control